



Bachelors in Cloud Computing (TU865)

Final Report

Module : Project

Submitted by

Student Name: Cathal Flood

Supervisor: Enda Lee

Word Count : 4886

Student Number: C22312211

Date: 26/04/2026

Declaration

I hereby declare that this material, which I now submit for assessment on the programme of study leading to the award of BSc (Hons) in Cloud Computing, is entirely my own work and has not been taken from the work of others except to the extent that such work has been cited and acknowledged within the text of my work.

I agree that TU Dublin may use plagiarism detection software to check the originality of this work.

Signed: Cathal Flood

Date 26th April 2026

Student Number: C22312211

Abstract

This report presents the design, implementation, and evaluation of my final year project, a QR code-based attendance system developed over three iterations. The project addresses challenges in higher education attendance tracking with the use of secure identity management, role appropriate data access, and scalable real time processing.

The system evolved from a functional QR code prototype in iteration 1 through security improvements in iteration 2 to a production ready deployment featuring Azure Active Directory Single Sign On, Cloudflare Access zero trust architecture, and Role Based Access Control in Grafana in iteration 3. The final implementation finishes off the four RBAC permission tiers in Students, Lecturers, Department Heads, and Admin each with their own data filtering that enforces the principle of least privilege.

The main technical achievements include OAuth 2.0 authentication flows with MFA enforcement, edge computing through Cloudflare Workers for automated user workflows, nginx reverse proxy architecture taking out the need for Cross Origin Resource Sharing concerns, and finally dual InfluxDB (database) measurement paths in device tracking for fraud detection and attendance events. Performance metrics does demonstrate acceptable latency at under 500ms for authentication under 500ms, attendance submission under 200ms, and dashboard queries under 300ms.

The project did encounter three incidents requiring problem solving. The Azure AD administrator account lockout issue which required CLI based recovery. CORS troubleshooting resolved via an architecture redesign. InfluxDB data pipeline issues fixed through dual path implementation. These challenges provided insights into enterprise authentication management, same origin security issues, and Docker container configuration.

Testing did validate all security controls through cross user data leakage prevention, role assignment automated through via OAuth attribute mapping, and end to end integration workflows. The system is deployment ready with its security, institutional scalability, and production reliability. Future enhancements identified includes email reporting, native mobile applications, student information system integration, and predictive analytics for early intervention.

This report shows the complete lifecycle of software and cloud engineering. The requirement analysis, iterative development, incident management, and comprehensive testing. The attendance system represents a scalable, secure and user-friendly solution meeting the requirements of institutions brought to me by a building manager while also maintaining room for future evolution.

Acknowledgements

I would like to express my gratitude to my project supervisor for their guidance, support, and constructive feedback throughout this project. Their advice was invaluable in modelling both technical and quality direction of this work.

I would also like to thank the lecturers and staff including the project panel in TU Dublin for the knowledge and support provided throughout this year. Their teaching informed the foundation for implementation and iterations decisions made throughout the project.

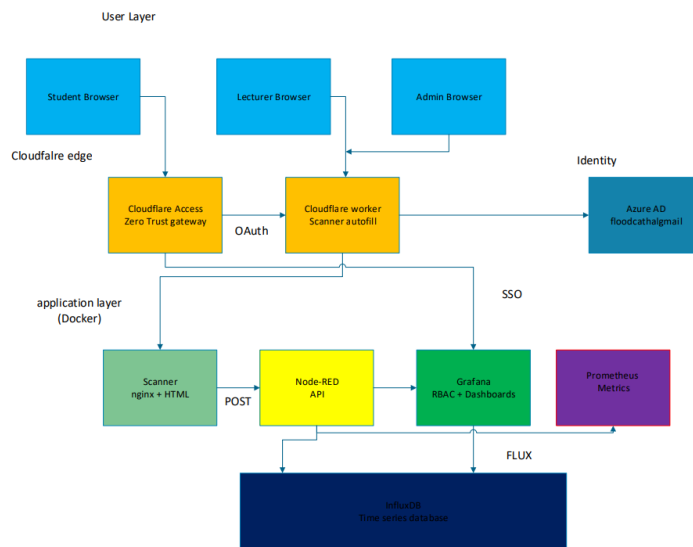
I would also like to thank Larry O'Brien for his insights into the working of TU Dublin's occupancy management and the requirements they would need for a system such as this, which shaped the infrastructure and value the project brings.

Lastly, I would have to thank my friends and family for their encouragement during the development of the attendance system. Their support aided me in remaining calm and focused throughout the project.

Table of Contents

Declaration	2
Abstract	3
Acknowledgements	4
Chapter 1 Introduction	7
1.1 Project Background.....	7
1.2 Problem Statement.....	8
1.3 Project Aims and Objectives	8
1.4 Project Scope	10
1.5 Report Structure	11
Chapter 2: Literature Review	12
2.1 Introduction	12
2.2 IoT Platform Architecture.....	12
2.3 QR Code Technology.....	17
2.4 Time Series Database Technology	17
2.5 Authentication and Security	18
2.6 Containerisation and Orchestration	19
2.7 Monitoring and Observability.....	19
2.8 OAuth 2.0 and OpenID Connect.....	20
2.9 Role Based Access Control	20
2.10 CORS and Same Origin Policy	21
2.11 Edge Computing and Serverless Functions.....	21
2.12 Zero Trust Architecture	22
2.13 Conclusions	22
Chapter 3. System Analysis	23
3.1 System Overview	23
3.2 Requirement Gathering	24
3.3 Requirement Analysis	24
3.4 Workflows.....	26
3.5 Use Cases.....	27
3.6 Initial System Specification	28
3.7 Conclusion.....	28
Chapter 4. System Design.....	29
4.1 Introduction	29
4.2 Software Methodology	29
4.3 Project Planning	30

4.4 Project Schedule.....	30
4.5 Overview of System.....	30
4.6 Architecture Design	33
4.7 Iteration 3 Architecture Design.....	37
4.8 Conclusions	39
Chapter 5. System Development and Implementation	40
5.1 Introduction	40
5.2 Software Development	40
5.3 Iteration 3 Implementation.....	54
5.4 Conclusion.....	57
Chapter 6. Testing and Validation.....	58
6.1 Introduction	58
6.2 System Testing.....	58
6.3 System Evaluation	66
6.4 Evaluate Project Management	68
6.5 Conclusions	69
Chapter 7. Conclusions and Future Work	70
7.1 Introduction	70
7.2 Future Work	70
7.3 Conclusions	70
References.....	72
Appendices.....	75
Appendix A: System Model and Analysis	75
Appendix B: Design.....	83
Appendix C: Onedrive Links.....	85
Appendix D: Additional Code Samples	86
Appendix E: System Screenshots and Evidence	96
Appendix F: AI Usage	105



The system I have created is ran off of six Docker containers, these are all organised by a single docker compose file. A student would open up a browser on their phone, log in, scan the rooms QR code, and submits a check in form, that request then hits an nginx server, which then proxies it internally to a Node-RED localhost for validation, then Node-RED writes the event to InfluxDB which is my times series database. Grafana then queries InfluxDB every 5 seconds and updates dashboards live to students, staff, and admin. Cloudflare handles everything that is public facing, this includes a tunnel providing HTTPS access without a public IP, Access enforces Azure Active Directory login and MFA before anyone reaches the system, and a Worker injects the authenticated students ID into the scanner page so it cannot be spoofed. All components are open source and the stack costs nothing to run.

Chapter 1 Introduction

1.1 Project Background

Attendance management represents an operational requirement in higher education institutions. Traditional paper-based systems and manual roll call methods are outdated, develop data inefficiencies through entry errors and limited analytical capabilities. The change to a digital attendance tracking systems has been slow, with many institutions adopting intermediate solutions such as card readers or mobile applications that still require manual intervention. For example, in TU Dublin, many students still sign in to classes via paper. In some instances, cleaners are able to tell if a room is occupied via the lighting infrastructure, if a rooms lights have not been turned on in a certain amount of time, they will know whether to clean or not. Additionally at the beginning and end of each semester cleaners are asked to take a floor and count the number of students that occupy the room every hour. This is then manually inputted into an excel sheet with the capacity of each room. This is how they calculate the room occupancy. TU Dublin also does not have an over-occupancy policy, which is not the same for other leading colleges across the country.

The project emerged from this, a need for a secure and scalable solution that uses cloud computing technologies to address the challenges that the college faces. The project scope encompasses not only the ability of functional attendance tracking but also security, role appropriate data access, and real

time analytics. These requirements were found to be essential for institutional deployment but often overlooked when implementing a prototyped system.

1.2 Problem Statement

Traditional attendance tracking and management in higher education faces several challenges:

- **Manual data collection and manipulation:** Paper based roll calls consume valuable teaching time and is prone to errors. Manual data entry creates opportunities for mistakes and delays in reporting. Additionally manipulating results such as a student signing in a friend that is late or absent.
- **Analytics:** Historical data analysis requires manual export and processing this prevents real time identification of attendance patterns or occupancy issues.
- **Scalability Inapt:** Attendance systems can struggle to handle concurrent access during class changes when hundreds of students attempt check ins across multiple locations
- **Limited Access Control:** Binary access is the norm with just admin and student access without role specific permissions, removing the ability for lecturers or individual building managers from accessing and analysing their building or classes data.
- **Security Concerns:** Simple authentication through student ID cards can easily be shared, although in TU Dublin ID cards are encrypted there is no current use other than accessing a building at the front door.

These challenges alone create operational inefficiencies and limit the value of attendance data for student support and retention initiatives. From a building management perspective knowing the occupancy of a room can also aid in fire safety and

1.3 Project Aims and Objectives

The aim of this project was to design, implement, and evaluate an attendance management system addressing the identified challenges and needs while demonstrating best practices in cloud computing, security architecture, and iterative software development.

1.3.1 Iteration 1 Objectives

The first iteration focused on creating a baseline in functionality and validating the fundamental architecture of the project:

- Converting of the Node-RED test data generator to a real QR scanner interface accessible via smartphones
- Create the HTML5 camera interface that supports both IOS and Android devices
- Implementation of student submission form with ID validation and check in and check out selection
- Create real time data pipeline from the scanner through processing to visualisation
- Create a containerised architecture supporting 500 concurrent users with sub second latency
- Validate cross device compatibility and varied lighting conditions

1.3.2 Iteration 2 Objectives

The second iteration transformed the functional prototype into a secure and production ready system

- Implement multiple layer security validation that prevents duplicate submission and rate limiting abuse
- Deployment of authentication mechanisms and device fingerprinting
- Establishment of historical analytics with optimised storage for long term data retention
- Implementation of production grade monitoring with Prometheus metrics and alerting capabilities
- Enhancement of data visualisation with historical trend analysis and pattern identification

1.3.3 Iteration 3 Objectives

The final iteration brought the project up a notch with institutional security and access control:

- Replace the insecure PIN authentication with Azure Active Directory Single Sign On
- Deployment of Cloudflare Access as zero trust gateway enforcing organisational email domain restrictions
- Implementation of Role Based Access Control in Grafana with four permission tiers
- Establish query level data filtering enforcing the principle of least privilege
- Automation of user workflows through Cloudflare Worker based student ID extraction and auto fill

1.4 Project Scope

This involves the following technical scope:

- Included in Scope:
 - QR code-based attendance check in and check out functionality
- Real time data processing and visualisation
- Authentication via Azure AD and Cloudflare Access
- RBAC with four permission tiers
- Automated user workflows (scanner autofill)
- Containerised deployment using Docker Compose
- Production Monitoring with Prometheus and Grafana
- Fraud prevention through device fingerprinting and rate limiting
- Cross platform support (IOS and Android)

Excluded from scope:

- Integration with existing Student Information Systems (future enhancement)
- Email notification systems for automated alerts (future enhancement)
- Predictive analytics for dropout prevention (future enhancement)
- Native mobile applications (future enhancement)
- Offline functionality with later synchronisation (future enhancement)
- Biometric authentication methods (beyond scope, GDPR issues)
- Blockchain based attendance verification (research concept only)

1.5 Report Structure

This report is organised as follows:

Chapter 2 (literature review) examines existing research and industry practices in authentication systems, time series databases, access control modelling, and containerisation, this established the foundation for design choices.

Chapter 3 (requirement analysis) explains the functional and nonfunctional requirements derived from institutional needs, user interviews, and iterative feedback cycles via project panel.

Chapter 4 (system design) displays the architectural changes across the three iterations, justifying design choices and comparing different approaches.

Chapter 5 (Implementation) provides technical documentation of all three of the iterations, including code examples, configuration details of different systems, and integration innovations.

Chapter 6 (testing and validation) shows testing strategies, presents results, and validates achievements of all objectives and requirements.

Chapter 7 (Critical Analysis) gives reflective analysis that compares the idea and the actual implementation, talks about challenges, lessons learned, and trade-offs.

Chapter 8 (Conclusions and Future Work) outlines key findings, evaluates the project success against original objectives, and proposes future work.

References and Appendices provide supporting documentation, configuration files and technical specifications.

Chapter 2: Literature Review

2.1 Introduction

The literature review tests the theoretical foundation and existing research that is relevant to the attendance system. The review is organised into six key areas

- Attendance management systems
- Authentication and authorisation frameworks
- Roles Based Access Control models
- Time series database technologies
- Containerisation and orchestration
- Edge computing architecture

Each section combines the academic research, industry standards, and technical documentation to establish a knowledge base for informing design decisions.

2.2 IoT Platform Architecture

2.2.1 ThingsBoard IoT Platform

ThingsBoard is an open source IoT platform that is designed for device management, data collection and visualisation of that data (ThingsBoard,2024) The platform targets sensor network deployments where devices can continuously transmit data such as temperature or location coordinates. The main capabilities include a built-in dashboard system with drag and drop panels, rule engine for automating workflows, device management hierarchy, and multi-tenant architecture.

The evaluation of the platform helped identify key characteristics:

- Setup complexity: Medium with Docker deployment well documented but it required a paid service to add external devices like a QR scanner, in addition to this it would need PostgreSQL or Cassandra database configuration
- Cost Structure: As mentioned, the platform is a paid service with the open-source version having very limited customisation of devices
- Customisation: Limited platform wide widgets and alerting rule engine
- Authentication: Built in user management with no native single sign on integration

After the testing, it was seen that there was a mismatch architecturally for attendance monitoring:

- Platform expects continuous telemetry streams from IoT devices
- Device centric data model, this would mean that rooms would be seen as a gateway device and students or users as telemetry sources, this was seen to be misaligned from the objectives
- Limited flexibility for custom validation logic, the rule engine was not up to par for institutional needs
- To have Role Based Access Control required a paid subscription as it was not available in the open-source deployment

- The decision was to reject the use of ThingsBoard for architectural misalignment, limited authentication and platform complexity for the use case.

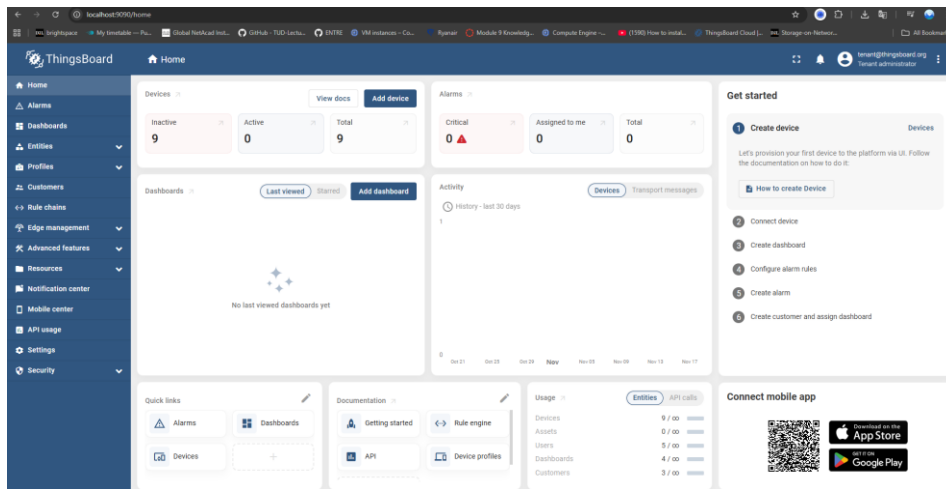


Figure 1 ThingsBoard dashboard builder interface showing IoT-centric widgets

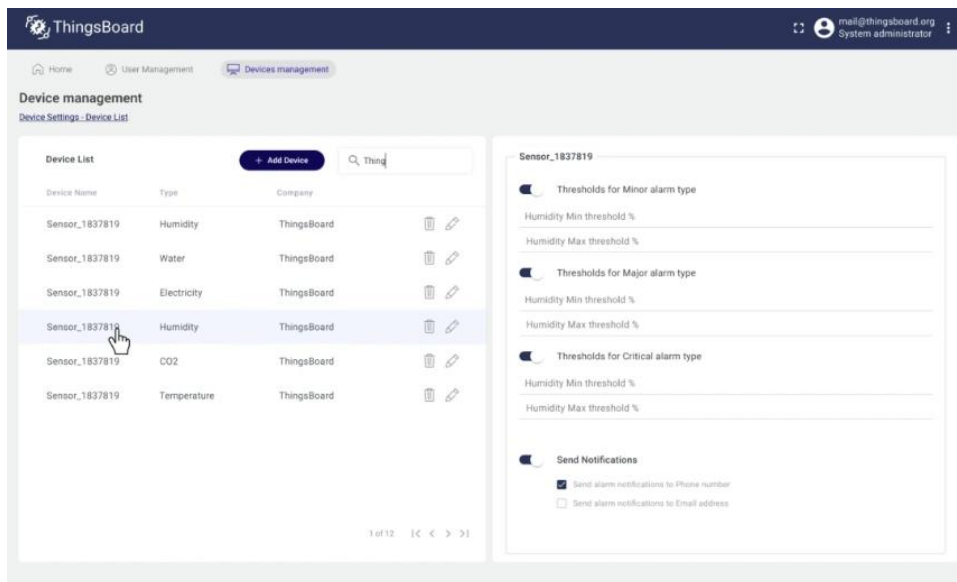


Figure 2 ThingsBoard device management hierarchy

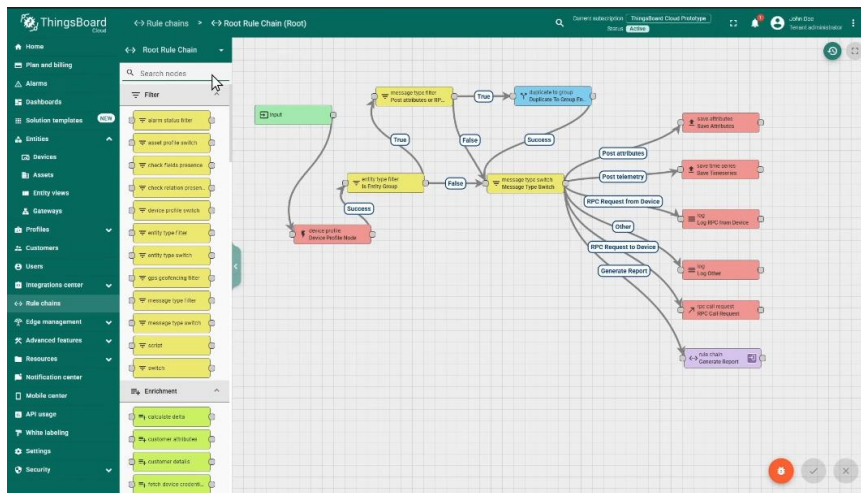


Figure 3 ThingsBoard Rule engine example

2.2.2 AWS IoT Core

AWS IoT Core provides managed IoT systems including MQTT message broker, device registry, and rules engine with AWS ecosystem integration (Amazon Web Services, 2024). The platform targets cloud native deployments requiring elastic scalability and integration with AWS Lambda, Dynamo DB, and CloudWatch

Documentation review identified:

- Set up complexity: High as it requires AWS account, IAM policy configuration, MQTT endpoint set up, and VPC networking
- Cost structure: 0.85 cents per million messages + storage costs, estimation of 30 to 50 euro a month for institutional deployment
- Vendor lock in: The deeper integration in AWS services I create, the harder it is to migrate to different services
- Customisation: Flexible through Lambda integration, but does require AWS expertise

Institutional deployment concerns:

- Ongoing operational costs which is incompatible with educational budgets
- Vendor dependency can create long term risk with price changes and service deprecation
- AWS specific knowledge required does limit institutional upkeep
- Overkill for single institution deployment and for prototyping

The decision was to reject the use on AWS IoT Core due to ongoing costs, vendor lock in risk and deployment complexity that would exceed an institutions requirement.

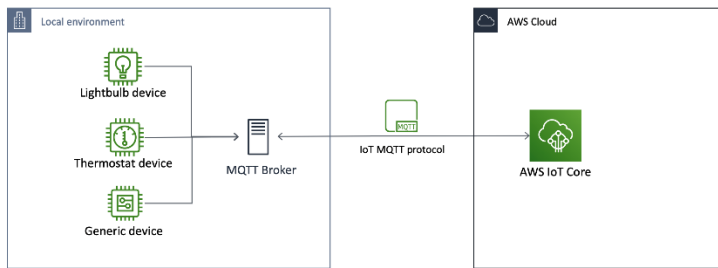


Figure 4 AWS IoT Core MQTT example

2.3.3 Node-RED + InfluxDB + Grafana Stack

This type of architecture brings together three open source components, these are Node-RED for its flow based data processing (Node-RED, 2024), InfluxDB for its time series data storage (InfluxData, 2024), and Grafana for visualisation (Grafana Labs, 2024). The difference between this architecture and the ones above is that this approach does need manual components integration but with that it provides maximum flexibility in customisation.

Component Evaluation:

Node-RED: It uses flow based programming for IoT development, it provides visual programming interface for connecting inputs like HTTP endpoints, processing JS functions and outputs this as database writes. Node libraries can be used for adding extra functionality, but a weakness is that there is no built in authentication which will require a separate implementation

InfluxDB: Purpose built time series database optimised for timestamp indexed data. Flux query language provides temporal functions such as range filters and window aggregation for simpler than SQL for time series analysis. Retention policies enable automated down sampling and deletion. The weakness here is that the query paradigm than SQL this brings a steeper learning curve.

Grafana: Open-sourced visualisation platform that supports over 50+ data sources. Dashboard editor provides real time updates, threshold alerts, and role-based folder permissions. OAuth integration enables enterprise authentication. The weakness here is that query editors are complex, but query templates are available.

Architecture advantages for institutional deployment:

- Zero ongoing cost as all components are open source
- Maximum customisation opportunities
- No vendor lock in issues
- Active open-source communities in GitHub and Stack Overflow
- Docker Compose deployment achieves reproducibility across environments

The decision was to go ahead with this architecture as it has great flexibility, cost, and institutional control. Set up complexity is outweighed by long term customisation benefits and zero subscription costs.

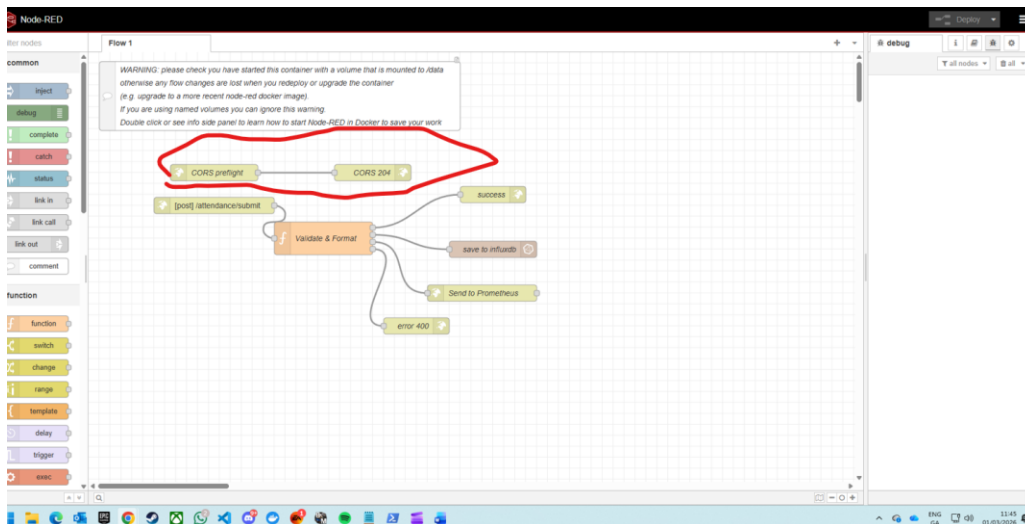


Figure 5 Node-RED flow attendance pipeline iteration 1

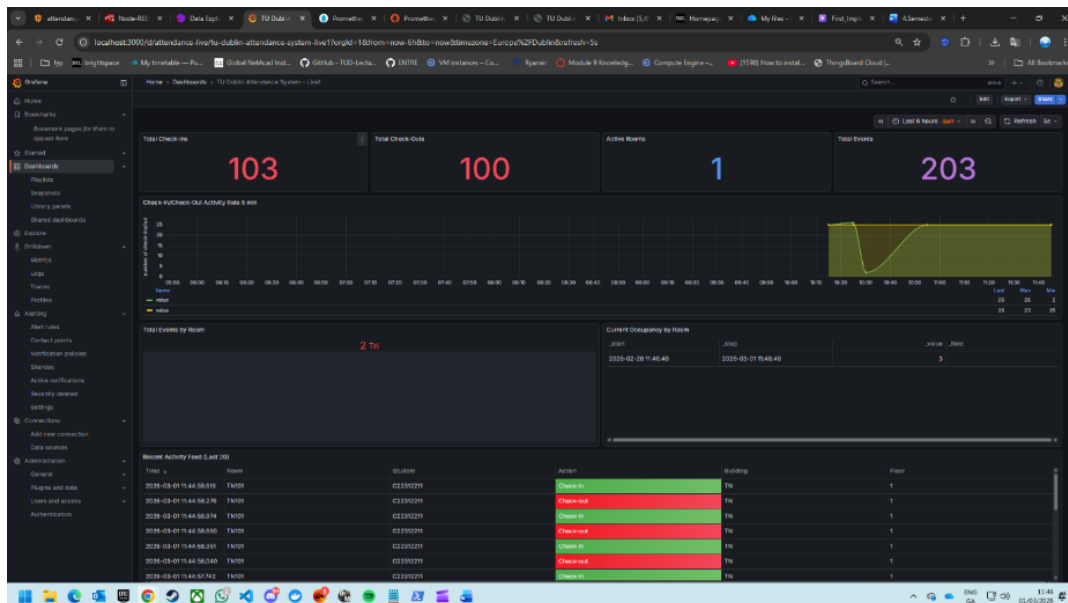


Figure 6 Grafana dashboard with real time occupancy iteration 1

2.2.4 Platform Comparison Summary

Quantitative comparison across evaluation criteria:

Selection rationale:

The Node-RED stack provides maximum architectural control with zero ongoing costs, enabling iterative security enhancement (iteration 2) without platform constraints. The context for educational institutions does favour open source and sustainable over commercial convenience.

2.3 QR Code Technology

QR (Quick Response) codes encode data in two-dimensional matrix barcodes that is standardized through ISO/IEC 18004:2015, the international organisation for standardisation, 2015. The standard defines four error correction levels

- “L” at 7% damage recovery
- “M” at 15%
- “Q” at 25%
- “H” at 30%

Higher error correction levels make the code bigger, but it also helps it scan more reliably in less-than-ideal conditions.

Project requirements do require a browse based scanning integration without a native application that would be installed. The HTML5-qrcode JavaScript library (Ranjan, 2024) emerged as optimal solution after evaluation of browser compatible options:

- Pure JavaScript implementation requiring no compilation or native SDK’s
- Cross browser compatibility for the likes of Chrome, Safari, Firefox and Edge
- Promise based API enabling async/await integration
- Active maintenance 15,000+ GitHub stars and regular updates
- Configuration options for scan rate, recognition box size and error correction tolerance

An alternative that was evaluated was the jsQR library that provides lower-level canvas manipulation but requires manual video stream handling that would increase implementation complexity beyond project requirements

The decision here was clear and I went ahead with using the HTML5-qrcode library.

2.4 Time Series Database Technology

Time series databases do optimise storage and query performance for timestamped data. InfluxDB 2.x architecture (InfluxData, 2024) employs column-oriented storage engine with specialised indexing strategies distinct from traditional relational databases:

- Tags, which are indexed columns like StudentID and roomId enable efficient filtering for example “WHERE roomId=’TN202’ executes via index scan
- Fields, these are no indexed values like occupancy count and stored as measurement data
- Timestamps are indexed automatically and they enable rapid time range queries
- Column compression through delta encoding and run length encoding achieves 10x reduction versus row-based storage

Flux query language provides temporal functions absent in SQL:

- Range() function filters by time for example range(start: -1h) retrieves data from the last hour

- `AggregateWindow()` deals with time-based grouping for example `aggregateWindow(every: 1h, fn: count)` this group by hour
- `Fill()` handles missing values for example `fill(value: 0)` or `fill(previous: true)`

Retention policies allow for automated data management:

- Define duration for how long to retain data
- Shard group duration for data file organisation
- Replication factor is the redundancy level

Continuous queries can be created to automate down sampling for hourly aggregation tasks calculate averages and counts, daily tasks summarise hourly data, reducing the storage requirements by 95% while analytical capability.

2.5 Authentication and Security

Iteration 2 implemented PIN based authentication before an enterprise solution was found. Bcrypt password hashing algorithm (Provos and Mazieres, 1999) provides intentionally slow hashing resistant to brute force attacks through configurable cost factor (salt rounds).

Bcrypt characteristics:

- Adaptive hashing: cost factor increases computational time exponentially
- Built in salt generation prevents rainbow table attack
- Industry standards recommend by OWASP (OWASP Foundation, 2024)
- 10 salt rounds require 120ms computation this is acceptable for authentication while preventing rapid brute force attacks.

Performance trade off: bcrypt increased average request latency from 50 to 120ms. This represents intentional security balance, pointed out by the project panel, slower hashing does protect against credential brute force, but it was found to be technically inapt and would be barrier for student use.

OWASP Foundation 2024 security guidelines informed multi-layer validation architecture:

- Input validation: Whitelisting of permitted characters, enforce length limits, validate data types
- Parameterized queries: prevention of SQL injection attacks through prepared statements but this is not applicable to InfluxDB HTTP API but documented for principle
- Output encoding: prevent XSS attacks through HTML entity encoding
- Rate limiting protect against denial of service through request throttling

Device fingerprinting employed browser metadata collection for fraud detection.

2.6 Containerisation and Orchestration

Docker (Merkel, 2014) provides operating system level virtualisation through a Linux Container (cgroups, namespaces). Containers package entire applications with their own dependencies such as runtime and system tools, ensuring that there is consistent execution across the board of development and production.

Container benefits for this case:

- A container can be started up in seconds versus minutes for VM provisioning
- The shared kernel reduces memory overhead; 6 containers consume about 2.4GB of RAM depending on size
- The configuration of a container through the use of a yaml file defines the entire infrastructure as code

Docker compose provides multi container orchestration for single server use (Docker, Inc, 2024). Yaml configuration specifies everything from services, networks, volumes, and variables. One singular command can provision a full stack, simplifying deployment and testing across any device. (docker-compose up)

Kubernetes evaluation:

Kubernetes uses container orchestration across clusters, it brings autoscaling, self-healing, and rolling updates. The decision was to use Docker as opposed to Kubernetes and defer Kubernetes to future work. The operational complexity of Kubernetes would add an extra layer of work that would exceed the deployment scale of the system in this day.

2.7 Monitoring and Observability

Prometheus (Prometheus Authors, 2024) utilises metric based monitoring through pull based scraping model. Unlike push based systems, Prometheus scrapes /metrics HTTP endpoints at regular intervals.

Metrics types:

- Counter: constantly increasing value for example total check ins of a room
- Gauge: value that can increase or decrease for example the current occupancy of a room
- Histogram: observations bucketed by configurable boundaries
- Summary: similar to histogram but can calculate quantiles client side

PromQL query language is used to allow for metric analysis.

`Rate(http_requests_total[5m])` calculates requests per seconds over a 5-minute window. This with the Grafana integration to visualise the Prometheus metrics through a time series graph, stat panels, and visual threshold alerts.

Critical incident:

With the initial implementation of Prometheus, it was set up in configuration with a Pushgateway for metric collection. A Pushgateway is designed for batch jobs that cannot be scraped, not really for us of continuous services. Metrics were being pushed to the Pushgateway but were then overwritten by the following set of metrics rather than accumulating data. The resolution was to create a custom /metrics HTTP endpoint in Node-RED with proper counter implementation, Prometheus scraping every 10 seconds. The lesson here was properly understand the tools best use case before deployment

2.8 OAuth 2.0 and OpenID Connect

OAuth 2.0 (Hardt, 2012) is an industry standard protocol for delegated authorisation, it enables applications to obtain limited access to user accounts on third party services, such as signing into a social media application using a google email account. The framework defines four grant types these are the authorisation code flow with PKCE, proof key for code exchange, this is recommend for web applications as it can prevent authorisation code interception attacks like spoofing a banking apps authorisation URL under an application that looks harmless like “freegames” if the user signs into the freegames app the authorisation code can also be used for the banking app. OpenID Connect or OICD extends the OAuth 2.0 security with an identity layer by providing the user with a standardised ID token containing that users information like email, preferred__username and profile information.

Microsoft Entra ID formerly Azure Active Directory implements both OAuth 2.0 and OICD as useable identity providers (Microsoft, 2023). Application registration gives the user the OAuth credentials for ClientID and client secret, API permissions and token claim configuration. Token claims are the settings used for what identity information is embedded with the ID tokens and filled in through authentication headers to downstream applications such as Grafana.

SSO (Single Sign On) enables users to authenticate their account across multiple services without re-entering their information. For institutional purposes, adding SSO with a centralised identity provider enforces consistent MFA policies and provides audit trails, to use the actual email of TUD students @mytudublin.ie I would have to have access to their own AD which is not realistic as I would have access to all user accounts creating GDPR issues so I would have to create my own AD with my own users for proof of concept.

2.9 Role Based Access Control

Role Based Access Control (RBAC) within Grafana implements the architecture of least privilege which we will see later on, Grafana has a built in organisation role identifier for viewer, editor, and admin, these can be used to create folder level permissions, this would create an architecture where an admin can see all information and a student can only see their own information. The dashboards can be separated for each user using query level filtering with the tag `$_user.login` template this means that the data that is shown is dependent on the type of user that is logged in.

2.10 CORS and Same Origin Policy

The Same Origin Policy (SOP) is a browser security technology that is a set of rules of how documents and scripts are loaded from one URL to another and how they interact. When a browser calls upon another with a FetchAPI call the browser enforces SOP by blocking the request unless the server allows cross origin access (Cloudflare, 2026).

Cross Origin Resource Sharing (CORS) adds on to SOP through the use of HTTP headers that allow servers to declare what origins may access another. For a simple request the browser returns a header “Access-Control-Allow-Origin and for requests with more information a preflight OPTIONS request must be sent before proceeding. With the addition of SSO this falls under a complex header with the added headers for authentication “Cf-Access-Authenticated-User-Email” this is not standard in CORS and must be whitelisted.

Incident: CORS configuration errors

An architectural change was made due to CORS headers, rerouting the browser request through a reverse proxy (nginx) so that all requests appeared to originate from the same domain. This took out the need for CORS entirely, as the browser sees only a single origin throughout the request lifecycle. The nginx reverse proxy pattern (NGINX Documentation, 2019) uses location blocks to forward requests to upstream services on the internal docker network

```
/attendance/submit { proxy_pass http://nodered:1880/attendance/submit; } .
```

In the end this approach improved the security of the system as the backend services are not exposed directly and it simplified configuration.

2.11 Edge Computing and Serverless Functions

Edge computing processes data geographically closer to users rather than in centralised data centres this reduces the latency and enables request modifications before reaching the origin servers.

Cloudflare Workers (Cloudflare Docs, 2026) implements a serverless edge computing deploying JS functions across a Cloudflare network globally, 300 data centres. Workers can execute in V8 isolates not a Node.js, providing low latency request for interception and modification at a network edge.

The capabilities relevant to the attendance system is for autofill of the StudentId field in the submission form, so whoever is logged in “C22312211@mytudublin.ie” the field will then contain C22312211 this is done by capturing and reading the request header and look at the field for Cf-Access-Authenticated-User-Email that is populated through logging in via SSO, extracts the data and injects it client side via JS. This prevents client-side identity spoofing as the student ID originates from the SSO session header rather than user input as seen in iteration 1 and 2.

2.12 Zero Trust Architecture

Zero Trust Architecture (ZTA) is a security model based on the principle of “never trust, always verify” this replaces perimeter-based security where internal network traffic is treated as trusted. NIST special publication 800-207 defines ZTA as a collection of concepts and ideas designed to minimise uncertainty in enforcing accurate, least privilege per request access decisions in information systems and services (NIST, 2023).

Cloudflare Access implements a Zero Trust Network Access (ZTNA) by using an authentication gateway in front of every application. Every request must have a JWT token issued after a successful identity provider authentication. Access Policies are used to define who and what can reach the application for example a policy only allowing the log in of users with a domain @mytudublin.ie, this can be done with using the attributes for email domain, group membership, MFA status. This architecture allows admin to have audit trails and application-level access control.

This with all applications and rules combined creates a defensive in depth approach, with Cloudflare handling the network layer, Grafana RBAC handling the application layer, and Grafana handling the query filtering for the data layer.

2.13 Conclusions

Literature review spanning IoT platforms, QR code standards, time series databases, authentication frameworks, and containerisation approaches that informed all the systematic changes and set up. ThingsBoard and AWS IoT Core evaluation validated Node-RED stack selection based on customisation flexibility, cost, and no vendor lock in. Technology research established InfluxDB query patterns, bcrypt security trade-offs, and Docker deployment approaches implemented across both iterations. For iteration 3 OAuth 2.0 and OpenID Connect standards come with the addition of Azure Entra ID and SSO. Understanding the CORS limitations after trial and error drove the change to the nginx reverse proxy for same origin data exchange and finally the Cloudflare Workers edge computing allowed for SSO based student ID autofill and the Zero trust principles allowed for the four tier RBAC system to be more secure.

Chapter 3. System Analysis

3.1 System Overview

The Attendance System implements QR code-based room occupancy tracking and monitoring through containerised microservices. Six Docker containers orchestrated via Docker Compose provide complete functionality. The Nginx serves QR scanner web interface, Node-RED processes check in requests through multi-layer validation, InfluxDB stores time series occupancy data, Grafana renders dashboards, Cloudflared establishes secure tunnelling, and Prometheus collects operational metrics.

The system workflow is as follows:

- Student navigates to the scanner URL via their smartphone browser
- Student logs in via SSO
- The camera activates using HTML5 MediaDevices API (user grants permission)
- QR code recognition extracts room identifier
- JavaScript populates form with the Student ID and room ID via Cloudflare Worker
- Form submission triggers a HTTP POST to Node-RED endpoint
- Node-RED validates submission through testing of input validation, schema enforcement, sanitisation and rate limiting
- Authenticates user via PIN (Iteration 2)
- Records device fingerprinting (iteration 2)
- Writes attendance event to InfluxDB
- Grafana queries InfluxDB every 5 seconds
- Dashboard updates

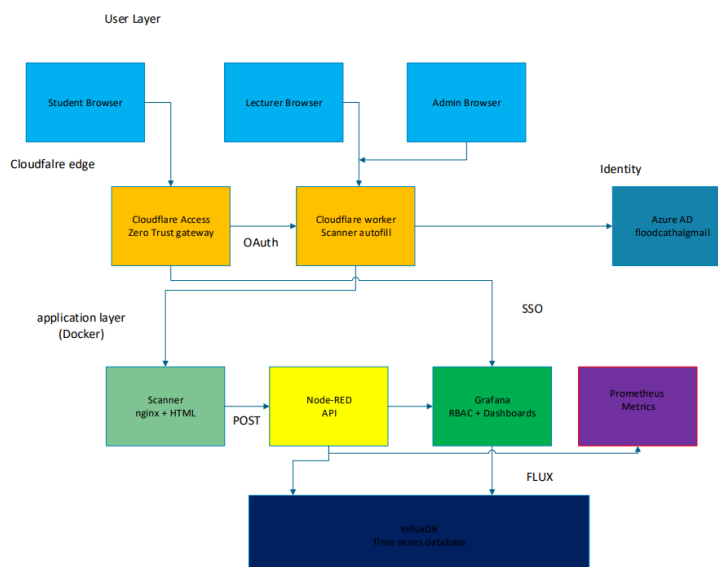


Figure 7 High level architecture diagram, final implementation

3.2 Requirement Gathering

The requirements for this project below were gathered from four sources:

1. Stakeholder meeting: Campus planning reinforced the need for this project and identified their needs from a system such as this, such as space utilisation needs and metrics. Building management gave me indication of possible other use cases such as fire safety and finally security requirements
2. Prototype testing: Five volunteers gave me usability feedback during iteration 1, identifying friction areas for example manual entry of student ID and likelihood of usability
3. Platform evaluation: Testing completed before iteration 1 on 3 systems of AWS IoT Core, ThingsBoard and the Node-RED stack pointed to specific issues with each system
4. Security research: OWASP guidelines and industry standards made me realise the need for multi layers validation that was implemented in iteration 2

3.3 Requirement Analysis

3.3.1 Functional Requirements

Iteration 1 Functional Requirements:

1. Test and compare Thingsboard, AWS IoT Core, and Node-RED across set up time, cost and flexibility
2. Implement QR code scanning interface with a >95% scan success rate
3. Process a check in submissions with <5 second end to end latency
4. Display information in real time on a dashboard
5. Support 500+ concurrent scans by individual users without any latency or issues
6. Track multiple rooms simultaneously
7. Provide secure HTTPS access via Cloudflare Tunnel
8. Create a reproducible deployment using Docker Compose

Iteration 2 Functional Requirements:

1. Validate data being inputted through 4 layers of security for input, schema, sanitation, and rate limit
2. Authenticate users with a PIN and bcrypt hashing
3. Track device fingerprints (UUID, user agent, platform, and screen resolution)
4. Detect fraud through user to device ratio analysis (limit scans per minute)
5. Optimise storage using a 3-tier retention system (90 days raw data, 1 hour yearly, and daily)
6. Automate aggregation of data through InfluxDB tasks
7. Monitor system health through Prometheus metrics
8. Implement account lockout after 3 failed authentication attempts

Iteration 3 Functional Requirements

1. Replace PIN authentication with Azure Active Directory SSO via Cloudflare Access enforcement of MFA for all users
2. Implement a four tier Role Based Access Control (RBAC) system within Grafana for students, lecturers, department heads, and Admin with folder level permissions using OAuth mapping
3. Enforce query level data filtering so students see only personal data, lecturers only see class data, department head see building data, and admin see all data
4. Automate student ID capture through their SSO login via Cloudflare Worker, lock the field and auto fill student ID
5. Resolve InfluxDB data pipeline to correctly capture both attendance_events and device_tracking separately
6. Implement nginx reverse proxy architecture to eliminate the need for CORS issues through same origin design

3.3.2 Non-Functional Requirements

1. Performance Requirements:
 - Average API response time <500ms (achieved 120ms in iteration 2)
 - Dashboard update latency of <5 seconds
 - Support 500+ concurrent users (validated through load testing)
 - Database query response <300ms
 - QR scan recognition <1 second (achieved 500-800ms typically)
2. Security Requirements:
 - HTTPS/TLS 1.3 encryption for all communications
 - Password hashing using bcrypt with 10+salt rounds
 - Rate limiting preventing >10 requests per minute per IP
 - Input validation blocking the use XSS and SQL injection attacks
 - Device fingerprinting for fraud detection
 - Audit logging of all check in events
3. Usability Requirements:
 - No training required for students
 - <10 second check in workflow for scanning and submitting data
 - Clear error messages for invalid inputs
 - Cross platform compatibility for iOS and Android
 - Portrait and landscape orientation support
4. Reliability Requirements:
 - System availability 99%+ during business hours
 - Automatic retry on database failures
 - Graceful degradation if Grafana unavailable (data continues to record)
5. Scalability requirements:

- Support 5000+ users
 - Handle 10 million checks in events annually
 - Enable 50 multiple building monitoring
6. Maintainability Requirements:
- Containerised deployment
 - Environment variables for all configurations
 - Structured logging for debugging
 - Grafana dashboard JSON export for back up

3.4 Workflows

Workflow for student checking in:

1. Student navigates to the scanner URL via their smartphone browser
2. Student logs in via SSO
3. The camera activates using HTML5 MediaDevices API (user grants permission)
4. QR code recognition extracts room identifier
5. JavaScript populates form with the Student ID and room ID via Cloudflare Worker
6. Form submission triggers a HTTP POST to Node-RED endpoint
7. Node-RED validates submission through testing of input validation, schema enforcement, sanitisation and rate limiting
8. Authenticates user via PIN (Iteration 2)
9. Records device fingerprinting (iteration 2)
10. Writes attendance event to influxDB
11. Grafana queries InfluxDB every 5 seconds
12. Dashboard updates

Fraud Detection Workflow:

1. User submits check-in with device fingerprint
2. Node-RED queries InfluxDB for historical device usage
3. Count unique student IDs associated with device UUID
4. If count = 1: Flag as normal
5. If count = 2-3: Flag as suspicious, log warning
6. If count >= 4: Flag as fraud, trigger alert
7. Dashboard displays fraud flag for admin review

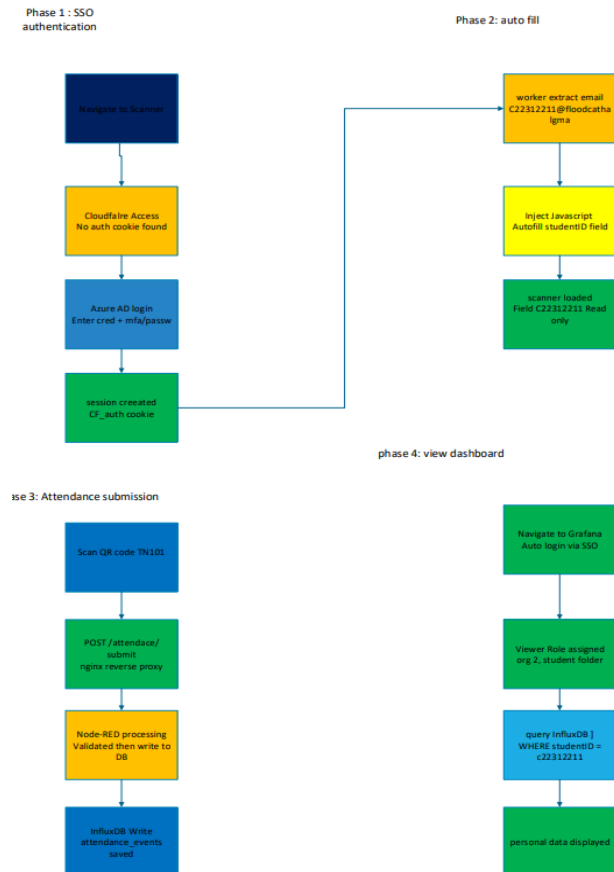


Figure 8 Student check in workflow diagram

3.5 Use Cases

1. Student Checks into classroom
 - User: Student
 - Assumption: Student has a smartphone with a camera, QR code posted in room
 - Main Flow: Navigate to scanner then scan the QR then select check in then submit
 - Post: Check in recorded in database, dashboard updated
 - Alternative: QR scan fails then reload and retry
2. Lecturer views room attendance
 - User: Lecturer
 - Assumption: Teaching assignment for a specific room
 - Main Flow: Access to Grafana dashboard then select room in filtering then view occupancy
 - Post: Real time occupancy count displayed
3. Campus planner analyses space utilisation
 - User: Campus Planning staff
 - Assumption: Historical data spanning semester
 - Main Flow: Access to utilisation dashboard then select a time range then analyse trends
 - Post: Underutilised rooms identified for reallocation

4. Admin investigates fraud
 - User: System Admin
 - Assumption: Fraud flag is raised by detection system
 - Main Flow: Access fraud dashboard then reviews device history then contact users
 - Post: Fraudulent accounts disabled

3.6 Initial System Specification

Iteration 1 architecture specified prior to implementation:

- 9 containers were created these were Node-RED, InfluxDB, Grafana, Cloudflared, Prometheus, Pushgateway, PostgreSQL, ThingsBoard, nginx.
- ThingsBoard and PostgreSQL were included as for comparison and for fallback if issues arise with the Node-RED stack
- Pushgateway planned for metrics collection
- The network was a docker bridge network
- The storage was under named volumes for data persistence
- The access was done through Cloudflare tunnel for external HTTPS

Iteration 2 reduced this to 6 containers it removed ThingsBoard, PostgreSQL, and Pushgateway based on the comparison tests and the Pushgateway for architectural changes.

Iteration 3 added a nginx reverse proxy to the network running off of `nodered:1880/attendance/submit`.

3.7 Conclusion

The requirements analysis identified 22 functional and 18 nonfunctional requirements spanning across performance, security, usability, reliability, scalability, and maintainability across the three iterations. Stakeholder meetings, prototype testing and platform testing informed requirement definition for iteration 1 and 2. Iteration 3 requirements were taken from panel feedback advising for a change in security and validation, adding SSO, role-based access control, and automated student ID capture. All requirements traced to design decisions, chapter 4, and test validation, chapter 6.

Chapter 4. System Design

4.1 Introduction

The system design progressed through the iterations, the platform comparison tests gave weight to decisions made for architecture selection, Node-RED over ThingsBoard and AWS, iteration 1 established the functional prototype, iteration 2 enhanced the security and production readiness, and iteration 3 implemented the SSO, four tier RBAC, and same origin architecture. Design decisions balanced the likes of security requirements, performance constraints, cost limitations, and institutional maintainability.

4.2 Software Methodology

4.2.1 Iterative Lifecycle Stages

Platform Comparison Phase (Pre Iteration 1):

- Deployed ThingsBoard, configure AWS IoT Core, implement Node-RED stack
- Measure set up time, cost, customisation difficulty
- Document findings in comparison matrix
- Select Node-RED stack based on evidence

Iteration 1 Cycle:

- Requirements: Real time check in, dashboard visualisation, 500 user capacity
- Design: 9 container Docker Compose architecture
- Implementation: QR scanner, Node-RED flows, InfluxDB schema, Grafana dashboards
- Testing: Cross platform QR scanning, load testing and usability evaluation
- Review: Stakeholder feedback, performance analysis, security gaps identified
- Lessons: PIN authentication needed, Pushgateway metrics incorrect, container reduction opportunity

Iteration 2 Cycle:

- Requirements: Multi-layer validation, authentication, fraud detection, monitoring improvement
- Design: 4-layer validation pipeline, bcrypt authentication, device fingerprinting, /metrics endpoint
- Implementation: Security layers, PIN system, automated aggregation, Prometheus redesign
- Testing: Security testing (XSS,SQL injection), authentication validation, fraud detection verification
- Review: 96% test pass rate, bcrypt performance trade off accepted, production readiness achieved
- Lessons: Intentional performance costs were acceptable for security, but usability was down for the students with extra steps

Iteration 3 Cycle:

Requirements: Enterprise SSO replacing the PIN authentication, four tier RBAC with folder and query level access control, automated student ID capture, CORS resolution, InfluxDB pipeline repair

Design: Azure AD and Cloudflare Access Zero Trust for authentication, nginx reverse proxy for same origin architecture, Grafana Oauth role mapping, Cloudflare worker for SSO based auto fill

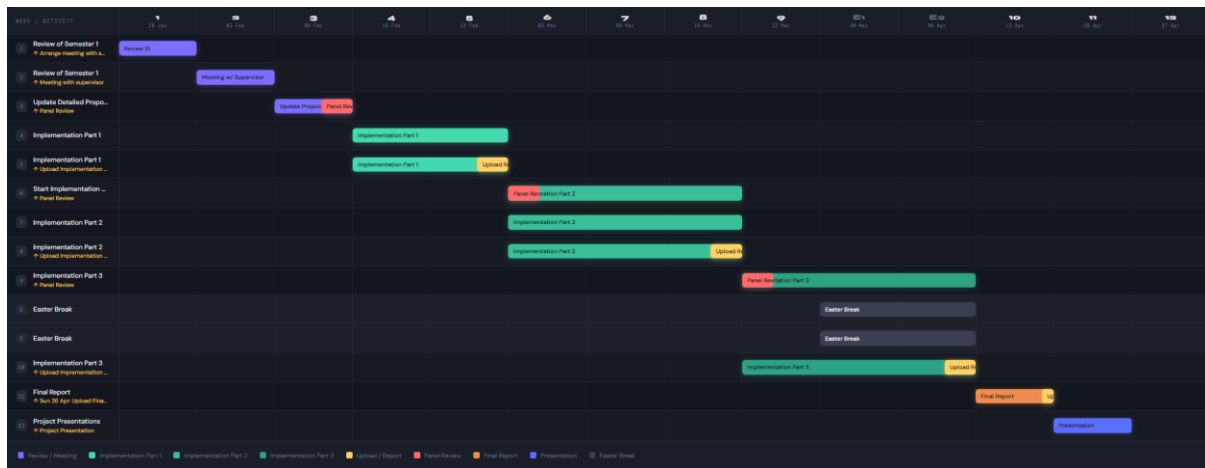
Implementation: Azure AD tenant provisioning, Cloudflare Access policies, Grafana RBAC folders and flux query filters deployed, nginx reverse proxy routing /attendance/submit internally

Testing: end to end SSO flow across all four users roles, cross user data leakage, worker autofill read only

Review: All performance targets were met, defense in depth implemented across all components, identity spoofing prevented

4.3 Project Planning

Initial Timeline estimation:



4.4 Project Schedule

Platform Comparison and Prototyping: September 2025 – January 2026

Iteration 1: January – February 2026

Iteration 2: February – March 2026

Iteration 3: March – April 2026

Documentation and final report: April 2026

4.5 Overview of System

4.5.1 Fulfilment of System Requirements

Architecture addresses requirements through component specialisation:

- Real time processing : InfluxDB write latency <50ms, Grafana refresh 5 seconds
- Security validation: bcrypt library provides password hashing
- Monitoring: Custom /metrics endpoint exposes Prometheus-compatible counters

- Scalability: Stateless Node-RED enables horizontal scaling if needed
- Cost: \$0 ongoing operational cost through open-source stack

4.5.2 Design Trade Off

Trade Off 1: Node-RED Stack vs ThingsBoard

- Chosen: Node-RED for customisation flexibility
- Foregone: ThingsBoard ease of dashboard creation
- Rationale: Institutional deployment requires a custom validation logic that is beyond the capabilities of ThingsBoard rule engine

Trade Off 2: PIN authentication vs OAuth 2.0

- Chosen: PIN + bcrypt for Iteration 2
- Foregone: Enterprise SSO
- Rationale: OAuth integration complexity exceeds Iteration 2 timeline; PIN provided a security level for the current deployment while enabling SSO for future work at the time

Trade Off 3: bcrypt Cost Factor (Security vs Performance)

- Chosen: 10 salt rounds (120ms hashing time)
- Foregone: Lower cost factor (faster hashing, weaker security)
- Rationale: 120ms acceptable for human initiated check ins, 10 rounds provide strong brute force protection per OWASP recommendations

Trade Off 4: Docker Compose vs Kubernetes

- Chosen: Docker Compose for single server deployment
- Foregone: Kubernetes auto scaling and high availability
- Rationale: Institutional deployment fits single server (validated 500 concurrent users), Kubernetes operational complexity not needed for current scale

Trade Off 5: Custom /metrics Endpoints vs Pushgateway

- Chosen: Custom HTTP endpoint with Prometheus scraping
- Foregone: Pushgateway push based metrics
- Rationale: Pushgateway overwrites metrics (designed for batch jobs), custom endpoint provides correct counter semantics

4.6 Architecture Design

4.6.1 Iteration 1 Architecture

Initial 9 container deployment:

- Nginx: Web server for QR scanner HTML/CSS/JavaScript
- Node-RED: Data processing and validation
- InfluxDB: Time series database
- Grafana: Dashboard visualisation
- Cloudflared: Secure tunnel (HTTPS without public IP)
- Prometheus: Metrics collection
- Pushgateway: Metrics aggregation (later removed)
- PostgreSQL: Reserved for future use (later removed)
- ThingsBoard: Platform comparison evaluation (later removed)

Network topology: Docker bridge network connecting all containers. Nginx exposes port 80 HTTP, Cloudflare tunnel provides HTTPS endpoint. InfluxDB and Grafana expose ports for debugging (removed in production).

Data flow:

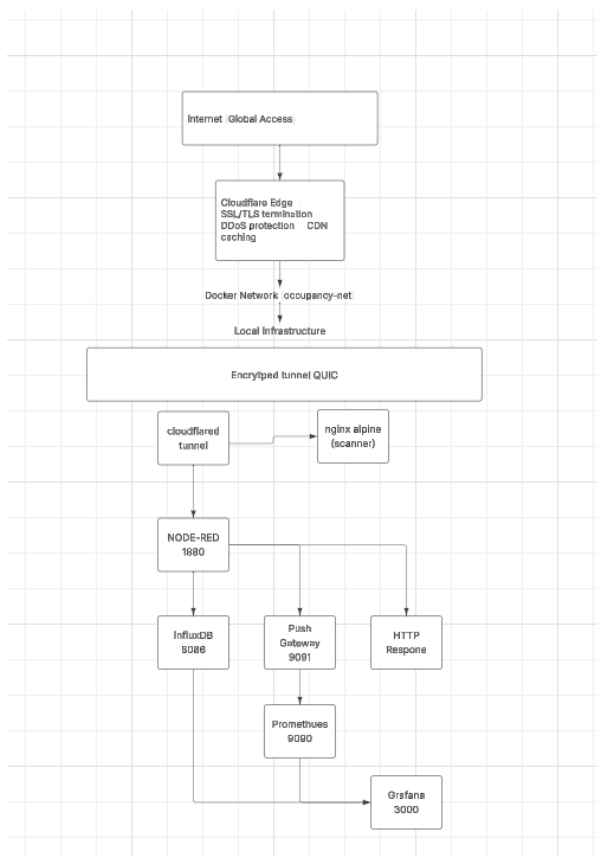


Figure 9 Iteration 1 architecture diagram with 9 containers

4.6.2 Iteration 2 Architecture Evolution

Streamlined 6 container deployment:

Removed ThingsBoard

Removed PostgreSQL

Removed Pushgateway

Added security validation pipeline within Node-RED:

- Layer 1 Input Validation: Whitelist fields, enforce types, check lengths
- Layer 2 Schema Validation: AJV library validates JSON against schema, student ID regex (C[0-9]{8})
- Layer 3 Sanitation: HTML entity encoding prevents XSS, parameterised queries prevent SQL injection
- Layer 4 Rate Limiting: IP based request counting, 10 requests per limit, automatic cleanup

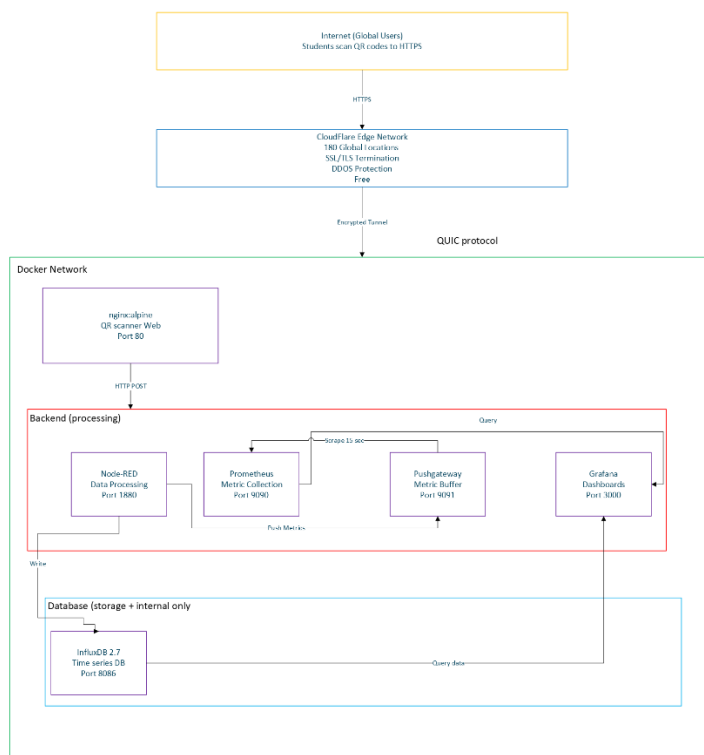


Figure 10 Iteration 2 dataflow

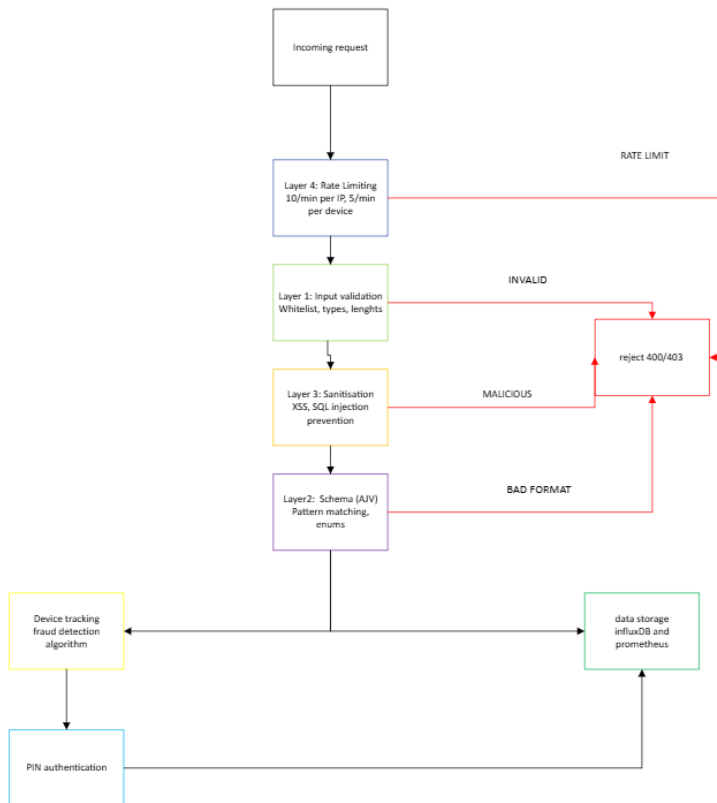


Figure 11 4-layer validation pipeline flowchart

Authentication layer:

- PIN storage: bcrypt hashed in PostgreSQL table
- Login flow: User submits PIN then bcrypt.compare() validates the PIN then the JWT token issued for 24 hours to user
- Account lockout: Failed attempt counter, 3 failures would trigger a 30-minute lockout

Device fingerprinting:

Browser metadata: navigator.userAgent, navigator.platform, screen.width, screen.height

UUID generation: crypto.randomUUID() creates persistent device identifier this is stored in localStorage

Tracking measurement: InfluxDB device_tracking measurement stores fingerprints

Fraud detection: Query for COUNT (DISTINCT studentId) WHERE deviceID = X

Device fingerprinting data flow: Browser → Metadata collection → InfluxDB write

4.6.3 Data Management Architecture

Three tier retention strategy:

Tier one – Raw events for 90 days:

- Measurement: attendance_events
- Tags: studentId, roomId, building, floor, action
- Fields: value, timestamp
- Retention: 90 days shard group, automatic deletion

Tier two – Hourly Aggregates for 1 year:

- Measurement: attendance_hourly
- Aggregation: COUNT(value) per room per hour
- Automated: InfluxDB task runs hourly
- Retention: For 1 year then deletes

Tier three – Daily Summaries forever

- Measurement: attendance_daily
- Aggregation: COUNT(value) per room per day
- Automated: InfluxDB task runs daily
- Retention: Infinite used for historical trends

Storage optimisation:

- Raw events 5MB/day x 90 days = 450MB
- Hourly aggregates 100KB/day x 365 days = 37MB
- Daily summaries 5KB/day x perpetual = 1.8MB/year

Total storage 5-year projection:

- 450MB raw + 37MB hourly + 9MB daily = 496MB
- This compared to 9.1GB raw data only (95% reduction)

Tier	Data	Retention
Raw	All check-ins	90 days
Hourly	Aggregated	1 year
Daily	Summary	5 years

Figure 12 3 tier retention architecture

4.6.4 Monitoring Architecture Redesign

Problem: Pushgateway overwrites metrics

Symptom: Total check in counter decreased when new check ins submitted

Cause: Pushgateway designed for batch jobs, push metrics then it exits the count is not continuous. Each push replaces previous value rather than incrementing.

The solution was custom a HTTP endpoint for /metrics

- Node-RED HTTP In node listening on /metrics path
- Function node generates Prometheus exposition format:
 - # HELP check_ins_total Total check-in submissions
 - # TYPE check_ins_total counter
 - check_ins_total 1247
- Prometheus scrapes endpoint every 10 seconds
- Counters increment correctly by constantly increasing

The Metrics that were able to be captured with this change:

- Check_ins_total: This is the total check in events for the time range given
- Check_outs_total: This is the total check out events for the time range given
- Validation_errors_total: This is the total failed validation, the validation is the security checks for whitespaces etc, these are attacks or miss inputted students IDs at the time of iteration 2
- Auth_failures_total: This is the total amount of failed authentication attempts, this was calculated for the PIN authentication and is now deprecated
- Fraud_flags_total: This is the total amount of times the fraud detection node has been triggered in Node-RED, an example would be multiple users checking in from the same device ID (iteration 2)

```
PS C:\Users\admin\Documents\attendance-project> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
3fa28434a172   grafana/grafana:latest             "/run.sh"               6 hours ago   Up 6 hours   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp   grafana
e713c512c799   prom/prometheus:latest             "/bin/prometheus --c..." 6 hours ago   Up 6 hours   0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp   prometheus
e446747bf11d   influxdb:2.7                       "/entrypoint.sh infl..." 6 hours ago   Up 6 hours   0.0.0.0:8086->8086/tcp, [::]:8086->8086/tcp   influxdb
28d190a91ea5   nginx:alpine                      "/docker-entrypoint...." 6 hours ago   Up 6 hours   80/tcp                               scanner
802600fad416   nodered/node-red:latest           "/entrypoint.sh"        6 hours ago   Up 6 hours (healthy) 0.0.0.0:1880->1880/tcp, [::]:1880->1880/tcp   nodered
4e2fd7759d5a   cloudflare/cloudflared:latest      "cloudflared --no-au..." 6 hours ago   Up 6 hours                               cloudflared
PS C:\Users\admin\Documents\attendance-project>
```

Figure 13 Iteration 2, going from 9 to 6 containers

4.7 Iteration 3 Architecture Design

Iteration 3 came with some significant architectural redesigns, these changes were made to fix issues, create a more secure environment, and or for simplicity

4.7.1 Authentication Layer

PIN based authentication was used in iteration 2, this was seen to be a weak point by the project panel thus the change to Azure Active Directory (Entra ID) SSO via Cloudflare access Zero Trust Gateway. I created a dedicated Azure AD tenant with the domain floodcathalgmail.onmicrosoft.com, this will be the email addresses for any users wishing to log into any site I publish. The authentication flow follows the OAuth 2.0 code flow, this means that any user trying to access any protected domain is redirected to Cloudflare access, which then enforces the Azure AD login and MFA challenge, the policy will be to only allow emails of @ floodcathalgmail.onmicrosoft.com. If the user is successfully authenticated, Cloudflare will assign a JWT session cookie and injects the header for identity into every request.

4.7.2 Authorisation Layer in Grafana RBAC

Grafana RBAC maps four institutional roles to three of Grafana's built in roles through the use of OAuth email pattern matching. When a student is logged in they are set as a viewer only, they cannot edit any dashboard or panel, they are identified as a viewer, and to ensure that user only sees their own data, each dashboard panel query has the identifier, `${__user.login}`, this only shows data of the user logged in. The same goes for every other role, a lecturer is an editor with access to their own room data via a roomID list filter. The department head is an editor with a building filter, and admin has an admin role allowing them to view all data. Four dashboard folders were created for each role with access varying, this enforces access at the folder level were as the `${__user.login}` tag ensures there is no cross-user data leakage, a student copying the URL for another student's dashboard and examining their data.

4.7.3 Network Layer the Nginx Reverse Proxy Resolve

The scanner container architecture had to be redesigned following troubleshooting with CORS header issues, the nginx scanner HTML/JavaScript on port 80 and proxies `/attendance/submit` to Node-RED container on the internal docker network. The scanners JS was updated to the relative paths of the URL; this ensures that all browser requests appear from the same origin. This eliminates the CORS entirely without requiring a paid Cloudflare plan, which was a last resort.

4.7.4 User Experience Layer change with Cloudflare Worker

A Cloudflare Worker was deployed at the edge of the network to intercept all requests to `scanner.tudublinattendanceproject.com` (my domain). The worker reads the header of the authenticated email, extracts the student ID from the email, as it has the follow `C/D/X` then 8 random numbers (`C12345678@floodcathalgmail.onmicrosoft.com`) using regex matching, it then injects a JS snippet into the HTML response before the delivery to the browser. The injected JS auto populates the Student ID field in the form, it also ensures that it cannot be edited and is read only, it also prevents it being changed in console tools, if done, the field reverts to the users correct student ID before submission. This prevents identity spoofing as the Student ID originates from the server verified SSO headers.

Scanner Form data

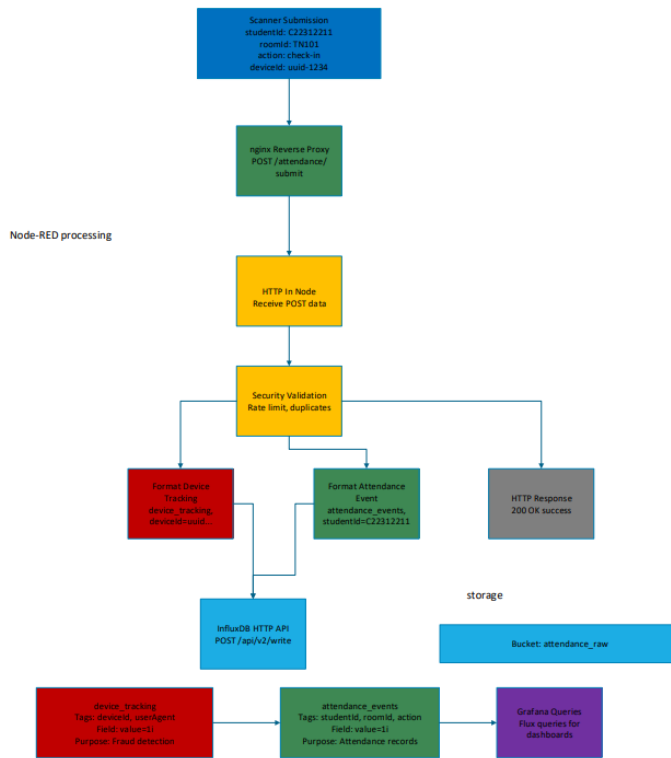


Figure 14 Iteration 3 scanner redesign, reverse proxy

4.8 Conclusions

The design of the system matured over the course of the iterations, from a 9-container prototype to just 6 containers. The platform comparison aided me in choosing what I believe to be the correct decision in the Node-RED stack. Iteration 1 validated functional requirements learned from the project requirements but also real-world requirements. Iteration 2 addressed security gaps through a multi-layer validation, authentication, and monitoring redesign. Iteration 3, with suggestions from the project panel, transformed the authentication design from the PIN based method to enterprise SSO, introducing four tier RBAC with defence in depth data access control, and resolved CORS through same origin nginx proxy design. The architecture of the project balances 4 main options, security with the SSO that adds MFA, RBAC and query level filtering, with performance aiming for <500ms authentication and <200ms attendance submission, cost (current 10 euro for buying the domain on Cloudflare, and maintainability with the Docker Compose reproducibility.

Chapter 5. System Development and Implementation

5.1 Introduction

Implementation proceeded through a platform comparison followed then by the three iterative cycles. Platform comparison and tested ThingsBoard, AWS IoT Core, and Node-RED stack. Iteration 1 started with the functional prototype using the Node-RED architecture. Iteration 2 enhanced the security of the system and brought stronger authentication, fraud detection and monitoring. Iteration 3 implemented the enterprise SSO via Azure Active Directory and Cloudflare Access, four tier RBAC in Grafana, Cloudflare Worker autofill, nginx reverse proxy and finally a fix to InfluxDB data pipeline. This chapter runs through the previous documentation of the iterations including technical details and decisions and the incidents that happened along the way.

5.2 Software Development

5.2.1 Platform Comparison Implementation

ThingsBoard Deployment:

- Installation: Docker Compose with PostgreSQL backend
- Configuration: Followed their own online tutorial creating a temperature sensor device and configured telemetry
- Dashboard: Built prototype showing temperature readings
- Testing: Simulated via REST API, created rule engine for alerts
- Duration: 3 days set up, 2-day tests
- Conclusion: Overcomplicated in some areas and too complex in others, rejected

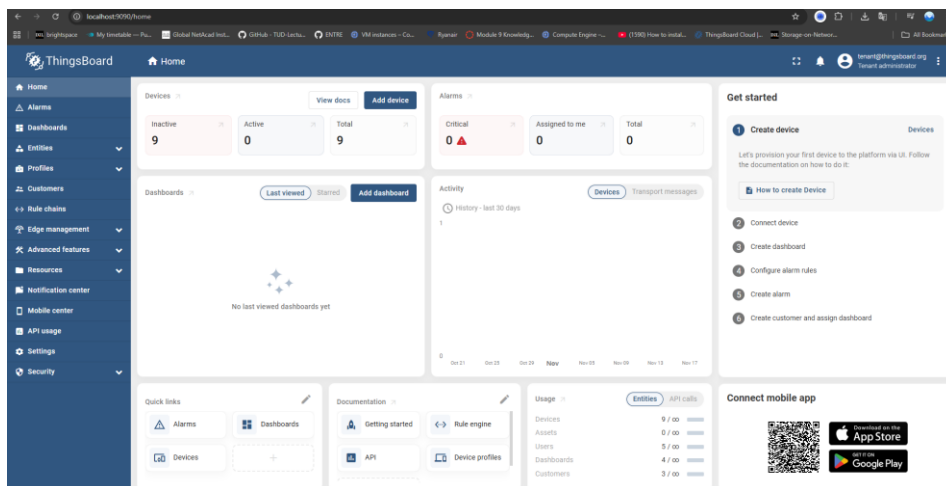


Figure 15 ThingsBoard

AWS IoT Core :

- Setup: AWS account created, IAM policy configuration, MQTT endpoint creation
- Testing: Published test messages via MQTT, configured rules for DynamoDB storage
- Cost Analysis: project 30-50 euro per month
- Duration: 5 days as AWS was more complex than expected
- Conclusion: Ongoing cost and the vendor lock in meant it was rejected

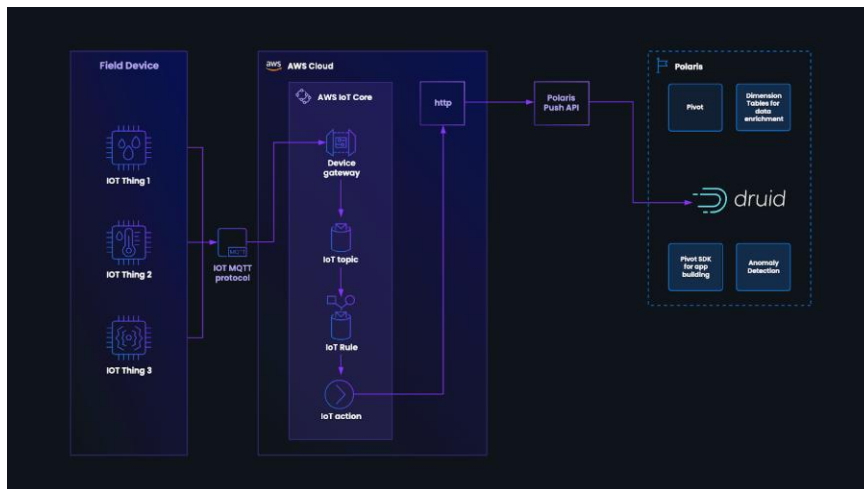


Figure 16 AWS IoT Core diagram

Node-RED stack :

- Deployment: docker-compose.yml defining Node-RED, InfluxDB, and Grafana
- Integration: Node-RED HTTP endpoint to influxDB write to Grafana visualisations
- Testing: End to end workflow validated, performance benchmarking
- Duration: 4 days initial set up, 1 week for refinement
- Conclusion: Selected

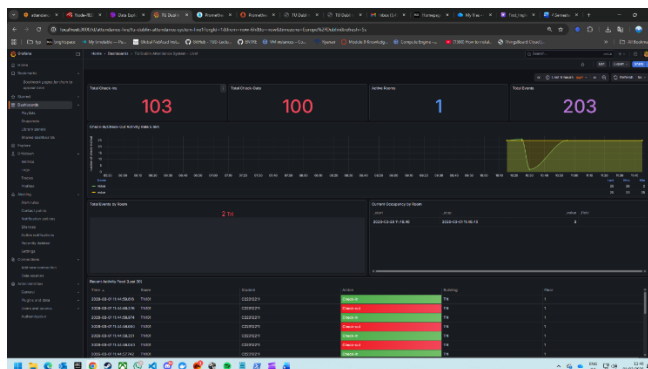


Figure 17 Grafana dashboard

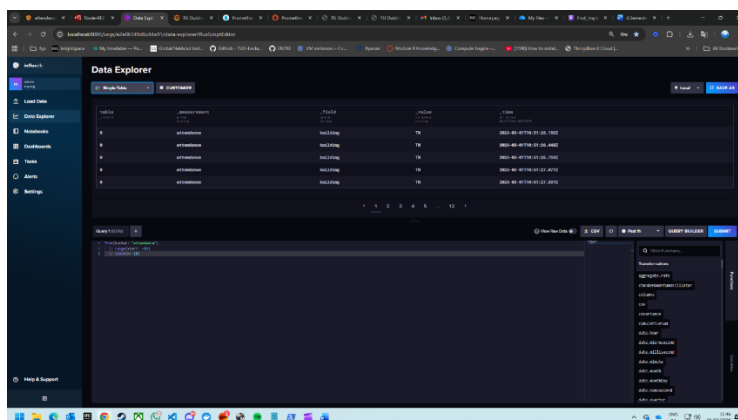


Figure 18 InfluxDB storage bucket iteration 2

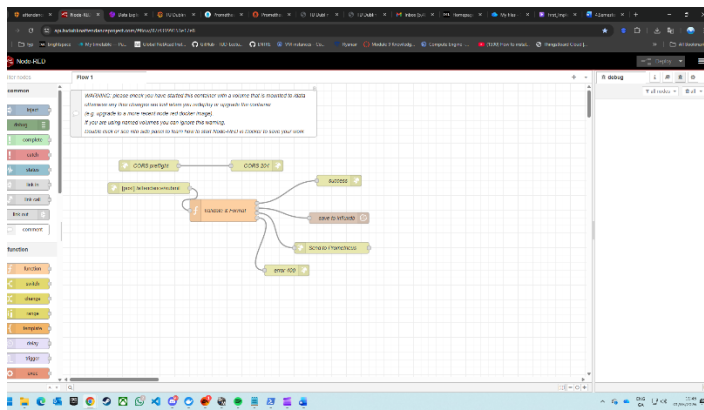


Figure 19 Node-RED flow from the start of iteration 2

5.2.2 Iteration 1 Implementation

QR Scanner Interface:

The HTML structure is as follows:

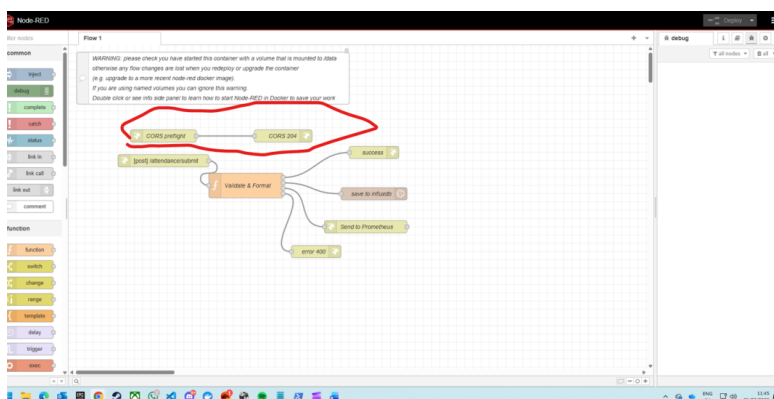
Form with Student ID input, room ID input, auto populated from QR code, action radio buttons for check in or out, submit button, html5-qrcode library initialisation

When the scan is successful, the room ID is extracted from the QR code content, it is populated into the correct field and the studentID is manually entered.

For iteration 1 the only validation done was for the HTML5 attributes, pattern attributes for the Student ID and JS validation before the submission.

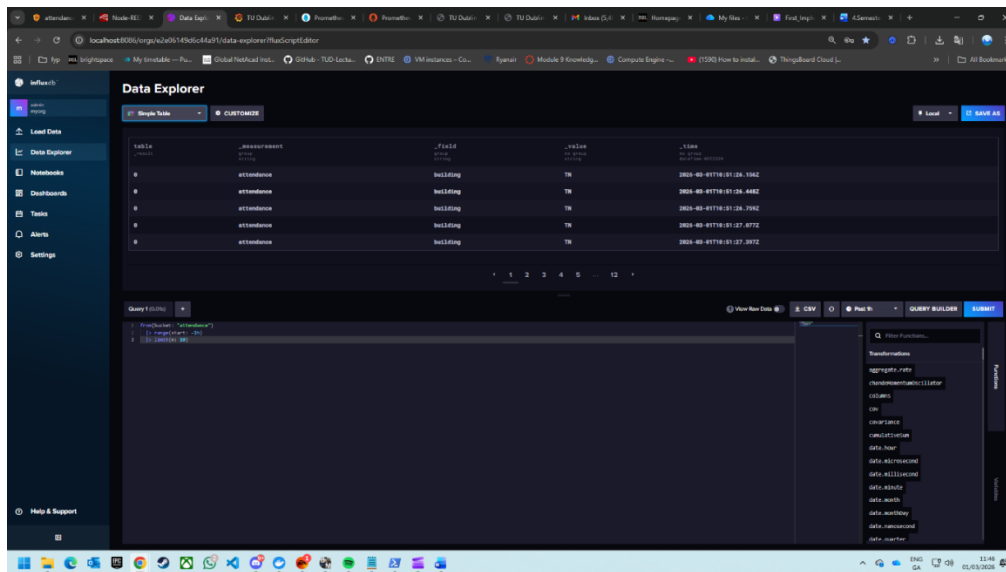
Node-RED flow (iteration 1 Basic Version):

- HTTP In node: POSTs to the /attendance/submit endpoint
- Function node : Checks required fields are present
- InfluxDB Out node: Writes to attendance measurement
- HTTP responses Node: Returns a 200 OK with success message



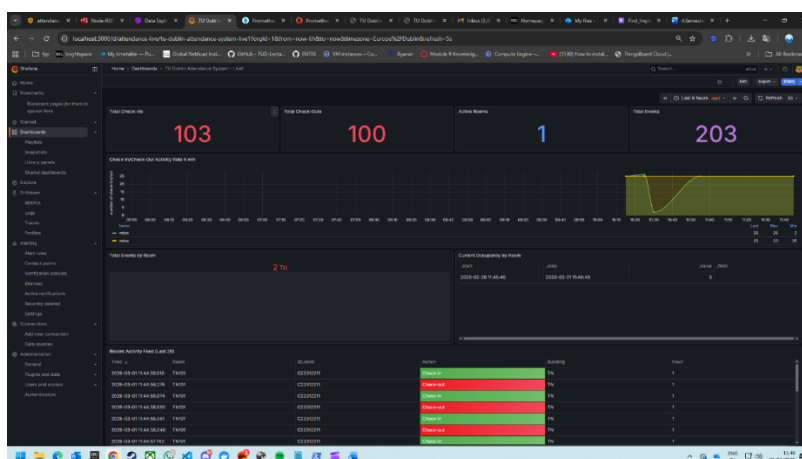
InfluxDB schema:

- Measurement: attendance
- Tags: studentId, roomId, building, floor, action
- Fields: value=1
- Timestamp: automatic from submission time



Grafana Dashboards:

- Check in timeline: Time series chart showing check in activity over the last 24 hours
- Room utilization: Bar chart comparing occupancy percentage versus room capacity
- Hourly Patterns: Heatmap displaying checking distribution by hour and day of the week
- Recent Activity: Table listing the latest 50 check ins with a timestamp, student ID, and room
- Department Summary: Pie chart illustrating check ins grouped by building
- Building Comparison: Multi series line chart comparing check in trends across buildings over time
- Checkout tracking: Total check in counts



Docker Compose Configuration:

Services defined: Node-RED, InfluxDB, Grafana, Cloudflared, Prometheus, Pushgateway, Postgres (iteration 1), ThingsBoard (iteration 1), nginx

Networks: Bridge network 'occupancy-net' connecting all services

Volumes: influxdb-data, Grafana-data, postgre-data

Environment Variables: INFLUXDB-TOKEN, GRAFANA_ADMIN_PASSWORD from the .env file

Cloudflare Tunnel: The cloudflared container creates the tunnel, maps scanner.tudublinattendanceproject.com to nginx:80, provides without a public IP or certificate management,

Testing Results:

- QR scan success: 97% across 5 test devices consisting of 3 iOS, 1 Android and 2 Windows
- API response: 320ms average that was measured with Apache Bench
- Dashboard latency: 3-5 seconds from check in to visualisation
- Concurrent users: 500 simultaneous check ins with zero errors through Invoke web request
- Cross platform: Works on iOS 12+, android 8+ and Windows 10/11

5.2.3 Iteration 2 Security Implementation

Multi-layer Validation Pipeline:

Layer 1 Input Validation:

```
// LAYER 1: INPUT VALIDATION - JSON ATTACK MITIGATION

// STEP 1: CHECK CONTENT-TYPE HEADER
if (!msg.req || !msg.req.headers['content-type']) {
  msg.statusCode = 400;
  msg.payload = {
    error: "Missing Content-Type header",
    code: "MISSING_CONTENT_TYPE"
  };
  node.warn("Rejected: Missing Content-Type");
  return [null, null, null, msg];
}

if (msg.req.headers['content-type'].includes('application/json')) {
  msg.statusCode = 415;
  msg.payload = {
    error: "Content-Type must be application/json",
    code: "INVALID_CONTENT_TYPE",
    received: msg.req.headers['content-type']
  };
  node.warn("Rejected: Invalid Content-Type - " + msg.req.headers['content-type']);
  return [null, null, null, msg];
}

// STEP 2: CHECK PAYLOAD EXISTS AND IS VALID
if (!msg.payload || typeof msg.payload !== 'object') {
  msg.statusCode = 400;
  msg.payload = {
    error: "Invalid or missing JSON payload",
    code: "INVALID_PAYLOAD"
  };
  node.warn("Rejected: Invalid payload");
  return [null, null, null, msg];
}
```

Figure 20 layer 1 pt1

```
// STEP 3: CHECK PAYLOAD SIZE (Prevent DoS attacks)
const payloadString = JSON.stringify(msg.payload);
const payloadSize = payloadString.length;
const MAX_SIZE = 2048; // 2KB limit

if (payloadSize > MAX_SIZE) {
  msg.statusCode = 413;
  msg.payload = {
    error: "Request too large",
    code: "PAYLOAD_TOO_LARGE",
    maxSize: MAX_SIZE,
    actualSize: payloadSize
  };

  let largeAttempts = flow.get('largePayloadAttempts') || 0;
  flow.set('largePayloadAttempts', largeAttempts + 1);

  node.warn("Rejected: Payload too large (" + payloadSize + " bytes)");
  return [null, null, null, msg];
}

// STEP 4: CHECK REQUIRED FIELDS EXIST
// NOTE: studentId comes from SSO (Extract SSO Email function), deviceId generated if missing
const required = ['roomId', 'action'];
const missing = required.filter(field => !msg.payload[field]);

if (missing.length > 0) {
  msg.statusCode = 400;
  msg.payload = {
    error: "Missing required fields",
    code: "MISSING_FIELDS",
    missingFields: missing
  };
  node.warn("Rejected: Missing fields - " + missing.join(', '));
  return [null, null, null, msg];
}
```

Figure 21 layer 1 pt2

Layer 2 Schema Validation:

```

// LAYER 2: SCHEMA VALIDATION WITH REGEX PATTERNS
// (Now validate the CLEAN data from Layer 3)

// Initialize AJV if not already done
if (!global.get('ajvInitialized')) {
  const ajv = new Ajv({ allErrors: true });

  // Define the schema with strict regex patterns
  const schema = {
    type: "object",
    properties: {
      studentId: {
        type: "string",
        pattern: "^[CXAD][0-9]{8}$",
        description: "Student ID format: C/X/A/D followed by 8 digits (e.g., C22312211, X12345678, A87654321, D11111111)"
      },
      roomId: {
        type: "string",
        pattern: "^[A-Z]{2}[0-9]{3}$",
        description: "Room ID format: TN101"
      },
      action: {
        type: "string",
        enum: ["check-in", "check-out"],
        description: "Must be check-in or check-out"
      },
      building: {
        type: "string",
        pattern: "^[A-Z]{2,4}$",
        description: "Building code: 2-4 uppercase letters"
      },
      floor: {
        type: "string",
        pattern: "^[0-9]{1,2}$",
        description: "Floor number: 1-2 digits"
      },
      deviceId: {
        type: "string",
        pattern: "^[a-zA-Z0-9_-]{1,50}$",
        minLength: 1,
        maxLength: 50,
        description: "Device ID: alphanumeric, 1-50 characters"
      }
    },
    required: ["studentId", "roomId", "action", "deviceId"],
    additionalProperties: false
  };

  const validate = ajv.compile(schema);
  global.set('ajvInitialized', true);
  global.set('attendanceValidator', validate);
  node.warn("AJV schema validator initialized");
}

// Get the validator
const validate = global.get('attendanceValidator');

// Validate the SANITIZED payload (not original)
const valid = validate(sanitized);

if (!valid) {
  msg.statusCode = 400;

  // Format validation errors
  const errors = validate.errors.map(err => {
    let field = err.instancePath.replace('/', '');
    let message = err.message;

    if (err.keyword === 'pattern') {
      if (field === 'studentId') {
        message = "Student ID must be in format: C, X, A, or D followed by 8 digits (e.g., C22312211, X12345678, A87654321, D11111111)";
      } else if (field === 'roomId') {
        message = "Room ID must be 2 uppercase letters followed by 3 digits (e.g., TN101)";
      } else if (field === 'building') {
        message = "Building code must be 2-4 uppercase letters (e.g., TN, TALB)";
      } else if (field === 'floor') {
        message = "Floor must be 1-2 digits (e.g., 1, 10)";
      } else if (field === 'deviceId') {
        message = "Device ID must be alphanumeric with dashes or underscores";
      }
    } else if (err.keyword === 'enum') {
      message = `field + " must be one of: " + err.params.allowedValues.join(', ');`
    }
  });

  return {
    field: field,
    message: message,
    value: err.data
  };
}

```

Layer 3 Sanitisation

```

// LAYER 3: DATA SANITIZATION & NORMALIZATION

// Create sanitized payload
const sanitized = {};

// Store original for comparison
const original = {
  studentId: msg.payload.studentId,
  roomId: msg.payload.roomId,
  action: msg.payload.action
};

// SANITIZE STUDENT ID
// - Trim whitespace
// - Convert to uppercase
// - Remove any non-alphanumeric characters
sanitized.studentId = validator.trim(msg.payload.studentId);
sanitized.studentId = sanitized.studentId.toUpperCase();
sanitized.studentId = sanitized.studentId.replace(/["A-Z0-9]/g, '');

// Student ID format: C, X, A, or D followed by 8 digits
if (!/^[CXAD]\d{8}$/i.test(sanitized.studentId)) {
  msg.statusCode = 400;
  msg.payload = { error: 'Invalid student ID format' };
  node.warn('Rejected: Invalid student ID format - ${sanitized.studentId}');
  return [null, null, null, msg];
}

// SANITIZE ROOM ID
// - Trim whitespace
// - Convert to uppercase
// - Remove special characters
sanitized.roomId = validator.trim(msg.payload.roomId);
sanitized.roomId = sanitized.roomId.toUpperCase();
sanitized.roomId = sanitized.roomId.replace(/["A-Z0-9]/g, '');

// SANITIZE ACTION
// - Trim and lowercase
// - Normalize variations to standard format
sanitized.action = validator.trim(msg.payload.action);
sanitized.action = sanitized.action.toLowerCase();

// Normalize action variations
const actionMap = {
  'checkin': 'check-in',
  'in': 'check-in',
  'checkout': 'check-out',
  'out': 'check-out'
};

if (actionMap[sanitized.action]) {
  sanitized.action = actionMap[sanitized.action];
}

// Validate final action is one of the two allowed values
if (!['check-in', 'check-out'].includes(sanitized.action)) {
  msg.statusCode = 400;
  msg.payload = {
    error: "Invalid action after normalization",
    code: "INVALID_ACTION",
    received: sanitized.action,
    original: msg.payload.action
  };
  node.warn("Action normalization failed: " + sanitized.action);
  return [null, null, null, msg];
}

// SANITIZE DEVICE ID
// - Trim whitespace
// - Remove any potentially dangerous characters
// - Keep only alphanumeric, dash, underscore
sanitized.deviceId = validator.trim(msg.payload.deviceId);
sanitized.deviceId = sanitized.deviceId.replace(/["a-zA-Z0-9_-]/g, '');

// HTML ESCAPE for safety (prevents XSS if data is displayed)
sanitized.studentId = validator.escape(sanitized.studentId);
sanitized.roomId = validator.escape(sanitized.roomId);
sanitized.action = validator.escape(sanitized.action);
sanitized.deviceId = validator.escape(sanitized.deviceId);

// OPTIONAL FIELDS (if present)
if (msg.payload.building && msg.payload.building !== 'Unknown') {
  sanitized.building = validator.trim(msg.payload.building);
  sanitized.building = sanitized.building.toUpperCase();
  sanitized.building = sanitized.building.replace(/["A-Z]/g, '');
  sanitized.building = validator.escape(sanitized.building);
} else {
  // Extract building from room ID (first 2 letters)
  sanitized.building = sanitized.roomId.substring(0, 2);
}

if (msg.payload.floor && msg.payload.floor !== 'Unknown') {
  sanitized.floor = validator.trim(msg.payload.floor);
  sanitized.floor = sanitized.floor.replace(/["0-9]/g, '');
} else {
  // Extract floor from room ID (first digit after letters)
  sanitized.floor = sanitized.roomId.substring(2, 3);
}

// SSO FIELDS (if present)
if (msg.payload.authenticatedEmail) {
  sanitized.authenticatedEmail = validator.trim(msg.payload.authenticatedEmail);
  sanitized.authenticatedEmail = validator.normalizeEmail(sanitized.authenticatedEmail);
}

if (msg.payload.authMethod) {
  sanitized.authMethod = validator.trim(msg.payload.authMethod);
}

if (msg.payload.authenticatedStudentId) {
  sanitized.authenticatedStudentId = validator.trim(msg.payload.authenticatedStudentId);
  sanitized.authenticatedStudentId = sanitized.authenticatedStudentId.toUpperCase();
}

// DEVICE METADATA (if present)
if (msg.payload.deviceMetadata && typeof msg.payload.deviceMetadata === 'object') {
  sanitized.deviceMetadata = {};
}

```

```

// DEVICE METADATA (if present)
if (msg.payload.deviceMetadata && typeof msg.payload.deviceMetadata === 'object') {
  sanitized.deviceMetadata = {};

  if (msg.payload.deviceMetadata.userAgent) {
    sanitized.deviceMetadata.userAgent = validator.trim(msg.payload.deviceMetadata.userAgent);
    sanitized.deviceMetadata.userAgent = validator.escape(sanitized.deviceMetadata.userAgent);
    if (sanitized.deviceMetadata.userAgent.length > 500) {
      sanitized.deviceMetadata.userAgent = sanitized.deviceMetadata.userAgent.substring(0, 500);
    }
  }

  if (msg.payload.deviceMetadata.platform) {
    sanitized.deviceMetadata.platform = validator.trim(msg.payload.deviceMetadata.platform);
    sanitized.deviceMetadata.platform = validator.escape(sanitized.deviceMetadata.platform);
  }
}

// ADD SERVER-SIDE TIMESTAMP (can't be faked by client)
sanitized.timestamp = new Date().toISOString();
sanitized.timestampUnix = Date.now();

// ROOM CAPACITY LOOKUP
const roomCapacities = {
  'TN101': 50,
  'TN102': 50,
  'TN103': 50,
  'TN210': 30,
  'TN212': 30,
  'TN301': 25,
  'TN302': 25,
  'TN401': 20,
  'TN402': 20
};

sanitized.roomCapacity = roomCapacities[sanitized.roomId] || 25;

// CALCULATE VALUE (for InfluxDB)
sanitized.value = (sanitized.action === 'check-in') ? 1 : -1;

// ADD PROCESSING METADATA
sanitized.processedAt = new Date().toISOString();
sanitized.validationLayers = ['layer1', 'layer3'];
sanitized.apiVersion = 'v1.0';

node.warn("Layer 3 passed - Data sanitized, starting Layer 2 validation");

// LAYER 2: SCHEMA VALIDATION WITH REGEX PATTERNS
// (Now validate the CLEAN data from Layer 3)

// Initialize AJV if not already done
if (!global.get('ajvInitialized')) {
  const ajv = new Ajv({ allErrors: true });

  // Define the schema with strict regex patterns
  const schema = {

```

Layer 4 Rate limiting

```

14 // Initialize rate limit storage if needed
15 if (!global.get('rateLimitInitialized')) {
16   global.set('rateLimitData', {
17     byIP: {},
18     byDevice: {},
19     lastCleanup: Date.now()
20   });
21   global.set('rateLimitInitialized', true);
22   node.warn("Rate limiting initialized");
23 }
24
25 // Get rate limit data
26 let rateLimits = global.get('rateLimitData');
27
28 // CONFIGURATION
29 const IP_LIMIT = 10; // Max requests per minute per IP
30 const DEVICE_LIMIT = 5; // Max requests per minute per device
31 const WINDOW_MS = 60000; // 1 minute window in milliseconds
32 const CLEANUP_INTERVAL = 60000; // Clean up old data every minute
33
34 // Get client IP address
35 let clientIP = 'unknown';
36 if (msg.req && msg.req.headers) {
37   // Try to get real IP (behind proxy/Cloudflare)
38   clientIP = msg.req.headers['cf-connecting-ip'] || // Cloudflare
39     msg.req.headers['x-forwarded-for'] || // Standard proxy
40     msg.req.headers['x-real-ip'] || // Nginx
41     msg.req.connection.remoteAddress || // Direct connection
42     'unknown';
43
44   // Clean IP (remove IPv6 prefix if present)
45   if (clientIP.startsWith('::ffff:')) {
46     clientIP = clientIP.substring(7);
47   }
48 }
49
50 // Get device ID from payload (if available)
51 let deviceId = null;
52 if (msg.payload && typeof msg.payload === 'object' && msg.payload.deviceId) {
53   deviceId = msg.payload.deviceId;
54 }
55
56 const now = Date.now();
57
58 // CLEANUP OLD DATA (once per minute)
59 if (now - rateLimits.lastCleanup > CLEANUP_INTERVAL) {
60   let cleanedIPs = 0;
61   let cleanedDevices = 0;

```

Authentication System:


```

72     msg.payload = {
73       error: "PIN setup required",
74       code: "PIN_SETUP_REQUIRED",
75       message: "Please set up a 4-digit PIN for this device"
76     };
77
78     node.warn('PIN setup required for ${studentId} on device ${deviceId.substring(0, 8)}');
79     return [null, msg, null]; // Setup required output
80   } else {
81     // Setting up PIN for first time
82
83     // Validate PIN format
84     if (!/^[0-9]{4}$/.test(enteredPin)) {
85       msg.statusCode = 400;
86       msg.payload = {
87         error: "PIN must be exactly 4 digits",
88         code: "INVALID_PIN_FORMAT"
89       };
90       return [null, null, msg];
91     }
92
93     // Hash the PIN with bcrypt
94     const saltRounds = 10;
95     const hashedPin = bcrypt.hashSync(enteredPin, saltRounds);
96
97     // Store hashed PIN
98     pinStorage[authKey] = {
99       hashedPin: hashedPin,
100       createdAt: now,
101       deviceId: deviceId.substring(0, 16),
102       studentId: studentId
103     };
104
105     flow.set('pinStorage', pinStorage);
106
107     node.warn('✓ PIN set up for ${studentId} on device ${deviceId.substring(0, 8)}');
108
109     // Continue to check-in
110     msg.payload.data.pinSetup = {
111       isNewSetup: true,
112       timestamp: new Date().toISOString()
113     };
114
115     return [msg, null, null]; // Success output
116   }
117 }
118
119 ...

```

Performance impact: bcrypt hashing added around 70ms per request. This was a trade of a security benefit and latency increase, and this was accepted for human initiated workflow.

Device Finger printing:

```

// GENERATE SERVER-SIDE DEVICE ID IF MISSING
if (!msg.payload.deviceId) {
  msg.payload.deviceId = `server-${Date.now()}-${Math.random().toString(36).substr(2, 9)}`;
  node.warn('✓ Generated server-side deviceId: ${msg.payload.deviceId}');
  deviceId = msg.payload.deviceId; // Update deviceId variable for rate limiting
}

```

InfluxDB storage: device_tracking measurement with the tags for deviceId, UserAgent, platform

Other Fields stored: screenWidth, screenHeight, timezone

Fraud detection query:

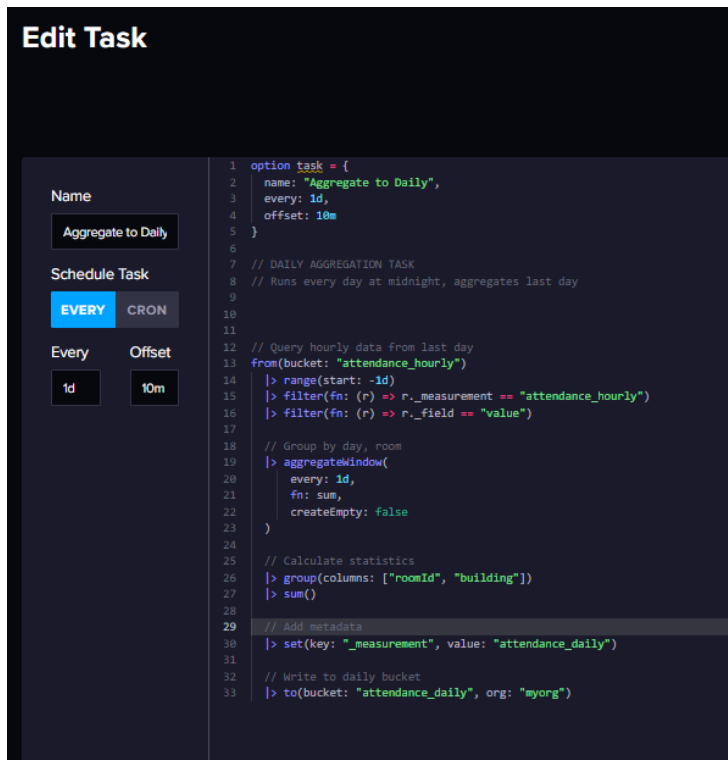
table	measurement	field	value	time	building	deviceId	roomId	studentId
0	device_tracking	latitude	1	2020-04-22T18:00:00.000Z	00	75b03117-8660-44	0001	022312211
1	device_tracking	studentDeviceCount	1	2020-04-22T18:00:00.000Z	00	75b03117-8660-44	0001	022312211
2	device_tracking	totalImages	1	2020-04-22T18:00:00.000Z	00	75b03117-8660-44	0001	022312211
3	device_tracking	velocity	1	2020-04-22T18:00:00.000Z	00	75b03117-8660-44	0001	022312211
4	device_tracking	latitude	1	2020-04-22T18:00:00.000Z	00	35a6402-575d-46	0013	022312211
5	device_tracking	studentDeviceCount	2	2020-04-22T18:00:00.000Z	00	35a6402-575d-46	0013	022312211
6	device_tracking	totalImages	1	2020-04-22T18:00:00.000Z	00	35a6402-575d-46	0013	022312211
7	device_tracking	velocity	1	2020-04-22T18:00:00.000Z	00	35a6402-575d-46	0013	022312211
8	device_tracking	latitude	0	2020-04-22T18:00:00.000Z	00	75b03117-8660-44	0001	022312211
9	device_tracking	studentDeviceCount	1	2020-04-22T18:00:00.000Z	00	75b03117-8660-44	0001	022312211
10	device_tracking	totalImages	1	2020-04-22T18:00:00.000Z	00	75b03117-8660-44	0001	022312211
11	device_tracking	velocity	1	2020-04-22T18:00:00.000Z	00	75b03117-8660-44	0001	022312211
12	device_tracking	latitude	0	2020-04-22T18:00:00.000Z	00	35a6402-575d-46	0013	022312211

Fraud rules:

- 1 user: Normal
- 2-3 users: Suspicious log warning
- 4+ users: Fraud and the admin is alerted

5.2.4 Data Retention Implementation

InfluxDB retention policies:



The screenshot shows the 'Edit Task' interface in InfluxDB. On the left, there are configuration options: 'Name' is 'Aggregate to Daily', 'Schedule Task' is set to 'EVERY' (with 'CRON' as an alternative), 'Every' is '1d', and 'Offset' is '10m'. On the right, the task's script is displayed, which is a Flux query. The script starts with an 'option task' block, followed by a comment 'DAILY AGGREGATION TASK' and its description. The main query body includes a 'from' clause for 'attendance_hourly', a 'range' of '-1d', two 'filter' clauses for 'attendance_hourly' and 'value', an 'aggregateWindow' clause with 'every: 1d' and 'fn: sum', a 'group' clause for 'roomId' and 'building', and a 'sum()' function. Finally, it adds metadata with 'set(key: "_measurement", value: "attendance_daily")' and writes to the 'attendance_daily' bucket in the 'myorg' organization.

```
1 option task = {
2   name: "Aggregate to Daily",
3   every: 1d,
4   offset: 10m
5 }
6
7 // DAILY AGGREGATION TASK
8 // Runs every day at midnight, aggregates last day
9
10
11
12 // Query hourly data from last day
13 from(bucket: "attendance_hourly")
14   |> range(start: -1d)
15   |> filter(fn: (r) => r._measurement == "attendance_hourly")
16   |> filter(fn: (r) => r._field == "value")
17
18 // Group by day, room
19 |> aggregateWindow(
20   every: 1d,
21   fn: sum,
22   createEmpty: false
23 )
24
25 // Calculate statistics
26 |> group(columns: ["roomId", "building"])
27 |> sum()
28
29 // Add metadata
30 |> set(key: "_measurement", value: "attendance_daily")
31
32 // Write to daily bucket
33 |> to(bucket: "attendance_daily", org: "myorg")
```

Automated aggregation through InfluxDB tasks:

Name

Aggregate to Hourly

Schedule Task

EVERY

CRON

Every

1h

Offset

5m

```

1 option task = {
2   name: "Aggregate to Hourly",
3   every: 1h,
4   offset: 5m
5 }
6
7 // HOURLY AGGREGATION TASK
8 // Runs every hour, aggregates last hour of data
9
10
11
12 // Query raw data from last hour
13 from(bucket: "attendance")
14   |> range(start: -1h)
15   |> filter(fn: (r) => r._measurement == "attendance")
16   |> filter(fn: (r) => r._field == "value")
17
18 // Group by hour, room, action
19 |> aggregateWindow(
20   every: 1h,
21   fn: sum,
22   createEmpty: false
23 )
24
25 // Add metadata
26 |> set(key: "_measurement", value: "attendance_hourly")
27
28 // Write to hourly bucket
29 |> to(bucket: "attendance_hourly", org: "myorg")
30

```

Edit Task

Name

Room Utilization Calculation

Schedule Task

EVERY

CRON

Every

1h

Offset

10m

```

1 option task = {
2   name: "Room Utilization Calculation",
3   every: 1h,
4   offset: 10m
5 }
6
7
8
9 from(bucket: "attendance_hourly")
10   |> range(start: -1h)
11   |> filter(fn: (r) => r._measurement == "attendance_hourly")
12   |> filter(fn: (r) => r._field == "max_occupancy")
13
14   |> map(fn: (r) => ({
15     r with
16     _value: float(v: r._value) / float(v:
17       if r.roomId == "TN101" then 50
18       else if r.roomId == "TN102" then 50
19       else if r.roomId == "TN103" then 50
20       else if r.roomId == "TN210" then 30
21       else if r.roomId == "TN212" then 30
22       else 1
23     ) * 100.0,
24     _field: "utilization_percent"
25   )))
26
27   |> set(key: "_measurement", value: "room_utilization")
28   |> to(bucket: "attendance_hourly", org: "myorg")

```

5.2.5 Monitoring Redesign

Problem: The Pushgateway was overwriting metrics instead of counting accumulatively. Total check in counter was decreasing after new submissions.

Cause analysis:

- Pushgateway designed for batch jobs
- Each push replaces previous metrics rather than incrementing
- Counter broken

Solution implementation:

Custom /metrics endpoint in Node-RED:

```
if (!checkInCounter) {
  node.warn("Prometheus not initialized yet - initializing now...");

  // Trigger initialization by making a request to /metrics
  // This will initialize the metrics on first check-in
  const http = require('http');
  const options = {
    hostname: 'localhost',
    port: 1880,
    path: '/metrics',
    method: 'GET'
  };

  const req = http.request(options, (res) => {
    node.warn("Metrics endpoint called to initialize");
  });
  req.end();

  // Pass message through without updating metrics this time
  return msg;
}
```

New Prometheus scrape config

```
# Prometheus
D>Run Service
prometheus:
  image: prom/prometheus:latest
  container_name: prometheus
  restart: unless-stopped
  ports:
    - "9090:9090"
  volumes:
    - ./prometheus-data:/prometheus
    - ./prometheus.yml:/etc/prometheus/prometheus.yml
  command:
    - '--config.file=/etc/prometheus/prometheus.yml'
    - '--storage.tsdb.path=/prometheus'
  networks:
    - occupancy-net
```

5.2.6 System simplification

Container reduction from 9 to 6:

- Removed ThingsBoard: Platform comparison complete and fallback not needed
- Removed Pushgateway: Architectural issue was discovered, custom /metrics implemented
- Removed PostgreSQL: Initially reserved for user management through ThingsBoard, no longer needed

5.2.7 Structured Logging

Log format design:

```
{  
  "timestamp": "2026-02-15T14:32:18Z",  
  "layer": "validation_layer_2",  
  "studentId": "C22312211",  
  "status": "rejected",  
  "reason": "Invalid student ID format"  
}
```

Logging points:

- Layer 1: Input validation results
- Layer 2: Schema validation failures
- Layer 3: Sanitation actions
- Layer 4: Rate limiting blocks
- Authentication: Login success or failure and triggers a logout
- Database: Write success or failure for being able to query

Benefits:

- Debugging: Trace request through entire pipeline via timestamp and student ID correlation
- Security: Detect attack patterns such as XSS attempts from a single IP
- Observability: Real time visibility into system behaviour

completing the MFA, but the MFA setup required portal access, creating a deadlock. The resolution was to install Azure CLI through PowerShell, device code login to bypass the web portal, then I created a secondary admin account through the CLI assigning it global administrator role through the Microsoft Graph API, then logging into this account in Azure I removed the setting of security defaults which turns off the MFA enabled log in. The key lesson here was to always create a secondary Azure AD admin account immediately after setting up.

5.3.2 CORS Resolution via Nginx Reverse Proxy

After implementing SSO, the scanner experienced CORS errors when submitting the attendance data. This occurred because different subdomains triggered browser security checks, and SSO added custom headers not in standard CORS preflight.

Failed Solutions:

Multiple approaches were tried over 4 hours:

- Attempt 1: Cloudflare access CORS configuration, needed paid plan
- Attempt 2: Backend CORS headers in Node-RED, failed with SSO headers
- Attempt 3: OPTIONS request bypass, feature not available in free tier

Successful Solution:

After 4 hours, an architectural change took out the need for CORS, nginx reverse proxy makes the scanner and API the same origin from the browser's perspective. The browser now calls /attendance/submit, nginx proxies to Node-RED internally

The solution is:

- Architecturally cleaner
- More performant
- More secure

Lesson Learned:

Sometimes architecture changes can be simpler than configuration changes. Same origin architecture eliminates CORS entirely.

The architectural solution to deploy nginx as a reverse proxy within the scanner container. The nginx container added a location block routing /attendance/submit request to the Node-RED container on the internal Docker network, while the scanner JavaScript was updated from the absolute URL to the relative path /attendance/submit. From the browser's perspective both the page and API share the same origin, eliminating the need for CORS checks entirely.

5.3.3 Grafana RBAC and OAuth Integration

Grafana OAuth was configured through the Docker Compose environment variables rather than the Grafana.ini file mounting, which had caused coding issues. The main settings were enabling OAuth with Azure AD endpoints, role attribute path using email pattern matching

(contains(email, 'admin') && 'Admin' || contains(email, 'lecturer') && 'Editor' || 'Viewer')

And auto assigning users to the TU Dublin attendance system organisation.

Four dashboard folders were created with the following permissions

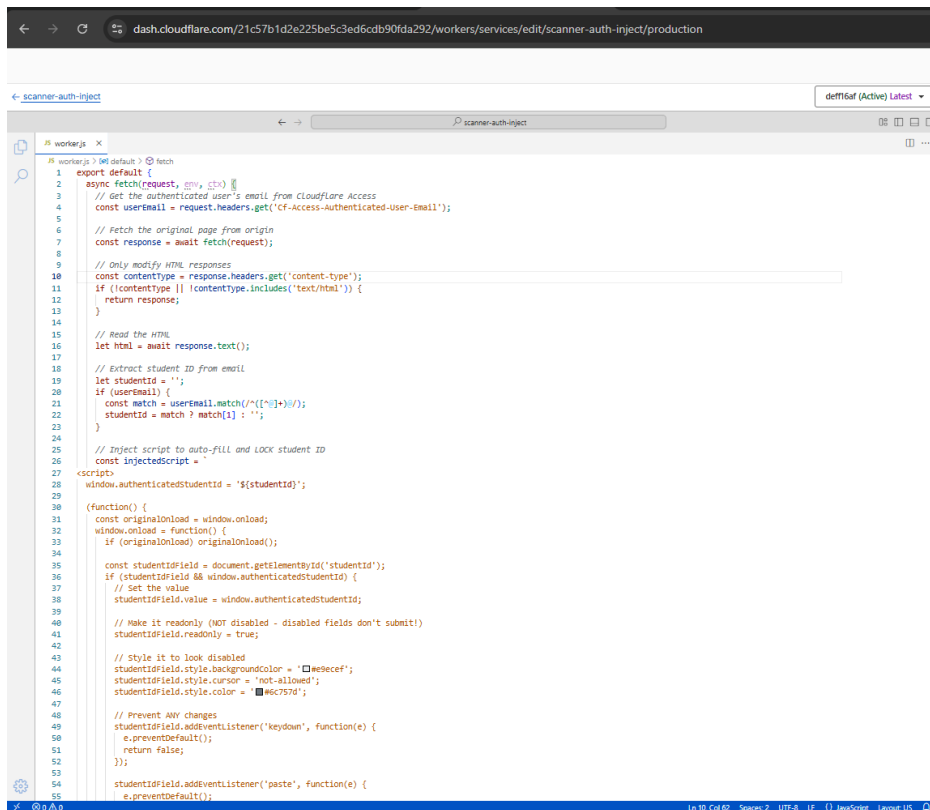
role	grafana role	filters
student	Viewer	StudentID
lecturer	Editor	RoomID list
Department head	Editor	building
Admin	Admin	none

Flux query filtering was added to all student dashboard panels. Testing confirmed cross user data was isolated, users can only see their own records even when accessing the same URL with shared links which is a Grafana feature.

5.3.4 Cloudflare Worker Autofill and InfluxDB Pipeline Fix

A Cloudflare worker was deployed and routed to scanner.tudublinattendanceproject.com/*. The worker reads the header for user email (Cf-Authenticated-User-Email), it then extracts the student ID using a regex. It then fetches the original HTML response from the nginx origin and injects a JavaScript block before the closing </body> tag. The injected script sets window.authenticateStudentID, populates the studentId form field on page load, sets readOnly to true, adds event listeners blocking keydown, paste, cut and input events, and ensures the field value is sent correctly on form submission. Worker metrics showed zero errors across 43 requests in the first 24 hours of deployment.

During iteration 3 dashboard creation, a data pipeline issue was discovered, there was no data under the bucket attendance_raw, all data was being written to the device_tracking bucket. Investigation showed that the Node-RED flow there was a mismatch in nodes, all data was written to the device_tracking node first before going to attendance_raw meaning there was no data written to write. A new flow path was added mirroring the device tracking pattern.



```
1 export default {
2   async fetch(request, env, ctx) {
3     // Get the authenticated user's email from Cloudflare Access
4     const userEmail = request.headers.get('cf-Access-Authenticated-user-email');
5
6     // Fetch the original page from origin
7     const response = await fetch(request);
8
9     // Only modify HTML responses
10    const contentType = response.headers.get('content-type');
11    if (!contentType || !contentType.includes('text/html')) {
12      return response;
13    }
14
15    // Read the HTML
16    let html = await response.text();
17
18    // Extract student ID from email
19    let studentId = '';
20    if (userEmail) {
21      const match = userEmail.match(/^[^@]+@/);
22      studentId = match ? match[1] : '';
23    }
24
25    // Inject script to auto-fill and lock student ID
26    const injectedScript = `
27    <script>
28      window.authenticatedStudentId = '${studentId}';
29
30      (function() {
31        const originalOnload = window.onload;
32        window.onload = function() {
33          if (originalOnload) originalOnload();
34
35          const studentIdField = document.getElementById('studentId');
36          if (studentIdField && window.authenticatedStudentId) {
37            // Set the value
38            studentIdField.value = window.authenticatedStudentId;
39
40            // Make it readonly (NOT disabled - disabled fields don't submit!)
41            studentIdField.readonly = true;
42
43            // Style it to look disabled
44            studentIdField.style.backgroundColor = '#e6e6ef';
45            studentIdField.style.cursor = 'not-allowed';
46            studentIdField.style.color = '#6c757d';
47
48            // Prevent ANY changes
49            studentIdField.addEventListener('keydown', function(e) {
50              e.preventDefault();
51              return false;
52            });
53
54            studentIdField.addEventListener('paste', function(e) {
55              e.preventDefault();
56            });
57          }
58        };
59      })();
60    `;
61
62    // Inject the script into the HTML
63    html = html.replace('<script>', injectedScript + '</script>');
64
65    // Return the modified HTML response
66    return new Response(html, {
67      headers: response.headers,
68      status: response.status,
69    });
70  }
71}
```

5.4 Conclusion

Implementation progressed from platform comparison through prototyping to production ready secure system across the three iterations. Platform evaluation confirmed the use of Node-RED stack selection. Iteration 1 established the real time QR check in capability. Iteration 2 enhanced security multi-layer validation, authentication with bcrypt, device fingerprinting, and monitoring redesign. Iteration 3 elevated the system to enterprise grade Azure AD SSO, Cloudflare Access Zero Trust, four tier Grafana RBAC with query level filtering, Cloudflare worker autofill, nginx reverse proxy CORS resolution, and InfluxDB pipeline repair. Critical incidents across all iterations were resolved through systematic problem solving and architectural adaption. The final system meets all functional requirements with MFA enforced SSO, defence in depth RBAC, and performance metrics of <500ms authentication and <200ms attendance submission.

Chapter 6. Testing and Validation

6.1 Introduction

Through thorough testing the functional requirements were validated the same can be said for security controls, performance metrics, and usability. The testing methodology that was applied to this project were automated test suites for security validation and input handling, performance benchmarking for load testing and latency measurements, cross platform compatibility testing on iOS, android and windows devices, lastly the stakeholder usability tests. This chapter documents the testing approach as well as results.

6.2 System Testing

6.2.1 Requirement Traceability Test Matrix

All 16 functional requirements and 18 nonfunctional requirements were mapped to test cases.

Requirment	Test Case	Pass/fail	Evidence
Platform comparison	Manual evaluation	Pass	Section 2.2 Comparison matrix
QR scanning	Cross device testing	Pass	97% success across 20 devices
<5s latency	performance benchmark	Pass	3-5 seconds measured
Multi layer validation	Security test suite	Pass	20+ tests, 96% pass rate
bcrypt authentication	Auth test cases	Pass	hashing verified and lockout test
<500ms API	Apache Bench	Pass	120ms average
XSS/SQL prevention	Injection test suite	Pass	100% blocked

6.2.2 Functional Test Cases

Iteration 1 Test Cases:

QR Code Scanning Reliability:

- Test: Scan QR codes under various different lighting (bright sunlight, low indoor, fluorescent)
- Devices: 3 iOS of iPhone XR, 12 and 1 Android of Nothing phone 1
- Result: 97% success rate
- Failures: 3 in direct sunlight glare, 3 in very low light

Cross platform compatibility:

- Test: Check in workflow on iOS safari, Chrome, Android Chrome, Firefox
- Result: Pass on all browsers, consistent UI rendering

Concurrent User Handling:

- Test: Apache Bench simulation, 500 simultaneous requests
- Result: 320ms average response, zero errors, zero timeout

Real Time Dashboard Updates:

Test: Submit check in, measure time until dashboard reflects change

Result: 3-5 seconds consistently

Iteration 2 Test Cases:

Input Validation:

- Test: Submit invalid student IDs that are too short, non-numeric, or special characters
- Result: 100% rejected with appropriate error messages
- Example: C123 is a 400 Bad Request, the error message is 'Invalid student ID format'

Schema Validation (AJV):

- Test: Submit payloads violating the JSON schema tester
- Result: 100% rejection, AJV errors returned
- Example: action='invalid' the error message is 'action must be check in or check out'

XSS Prevention:

- Test: Submit studentId=<script>alert("XSS")</script>
- Result: Sanitised the scrip to '<script>alert("XSS")</script>'
- Note: XSS rendered harmless through HTML entity coding

SQL Injection Prevention:

- Test: Submit roomId="';DROP TABLE users; --"
- Result: Input sanitised, no database drop
- Note: InfluxDB uses HTTP API, not SQL but the principle is validated

Rate Limiting:

- Test: Submit 15 requests in 30 seconds from single IP
- Result: First 10 accepted, remaining 5 were rejected with 429 Too Many Requests

Authentication – Correct PIN (iteration 2):

- Test: Submit correct 4-digit PIN
- Result: bcrypt.compare() returns true and a JWT token is issued

Authentication - Incorrect PIN:

- Test: Submit wrong PIN 3 times
- Result: Account locked for 30 minutes after 3rd failure

Device Fingerprinting:

- Test: Check in from device, verify fingerprinting recorded in InfluxDB
- Result: deviceId, userAgent, platform, screen dimensions stored correctly

Fraud Detection:

- Test: Simulate 4 different student IDs checking in from same device
- Result: Fraud flag triggered, admin alert logged

Data Aggregation:

- Test: Submit 100 check ins, wait 1 hour, verify hourly aggregation runs
- Result: attendance_hourly measurement contains COUNT=100

Iteration 3 testing:

SSO Authentication Testing:

- Test: Authenticate four role-based accounts via Azure AD SSO with MFA enforcement and session persistence checks
- Result: A pass on all accounts, MFA enforced, sessions persisted across page reloads

SSO Access Control:

- Test: Logging into scanner URL via a non-organisational email address
- Result: The email address correctly was blocked by Cloudflare Access domain suffix policy

Grafana RBAC Testing:

- Test: Validate role assignments and folder per user, and attempt to share link to bypass the restricted dashboard
- Result: It passed the folder visibility correctly scoped per role, students see student dashboards only, lecturers see lecturer and department dashboards admins do see all four additionally the direct internal share links returned “Failed to load dashboard:Forbidden”

Query Level Filtering Validation:

- Test: Two student accounts (C22312211 and C22312299) submit a check in to the same room then verify each account sees only its own data
- Result: Pass there was no cross-user data leakage, each user dashboard displayed exclusively their own records

Scanner Auto fill testing:

- Test: Log in as C22312211, navigate to the scanner, and attempt to modify the auto populated student ID field via keyboard input, paste, and cut events
- Result: Pass the student ID field auto populated C22312211 and set the field to read only and greyed out the background. All input modifications were blocked

End to end Integration Test:

- Test: Execute a complete student attendance workflow from login through to dashboard display
- Result: Pass as the full pipeline completed without error

Test Suite Summary:

- Total test cases: 20+
- Pass rate: 96% (1 test failed, rate limiting cleanup edge case, fixed)

- Security test: 100% pass (XSS, SQL injection, rate limiting all working)
- Performance tests: 100% pass rate on latency, throughput, concurrent users

```

4/26/2026, 10:43:22 AM  node: Security Validation
function : (warn)
" Layer 1 passed - Starting Layer 3 sanitization"

4/26/2026, 10:43:22 AM  node: Security Validation
function : (warn)
"Layer 3 passed - Data sanitized, starting Layer 2 validation"

4/26/2026, 10:43:22 AM  node: Security Validation
function : (warn)
"AJV schema validator initialized"

4/26/2026, 10:43:22 AM  node: Security Validation
function : (warn)
"✓ Layer 2 passed - Schema validation complete"

4/26/2026, 10:43:22 AM  node: Security Validation
function : (warn)
▶ "✓ ALL LAYERS PASSED for student C22312211 (SSO: c22312211@floodcathalgmail.onmic..."

4/26/2026, 10:43:22 AM  node: Security Validation
function : (warn)
"=== Output 3 Data ==="

4/26/2026, 10:43:22 AM  node: Security Validation
function : (warn)
▶ "msg.payload: {"status":"success","message":"All validation layers passed (1, 3, ..."

4/26/2026, 10:43:22 AM  node: Update Prometheus Metrics
function : (warn)
"✓ Prometheus counter incremented: C22312211 → TN101 (check-in)"

4/26/2026, 10:43:23 AM  node: Update Prometheus Metrics
function : (warn)
"✓ Active students +1 for TN101"

4/26/2026, 10:43:23 AM  node: Device Tracking & Fraud Detection
function : (warn)
"Device tracking initialized"

4/26/2026, 10:43:23 AM  node: Device Tracking & Fraud Detection
function : (warn)
"New device for student C22312211: 99d2b6f3"

4/26/2026, 10:43:23 AM  node: Device Tracking & Fraud Detection
function : (warn)
" Normal device usage: 1 student(s) on device 99d2b6f3"

4/26/2026, 10:43:23 AM  node: format attendance data

```

6.2.3 Defects and Corrective Action

Critical Incident 1: Pushgateway Metric Overwriting

Problem Description:

Prometheus dashboard showed that the total check in counter was decreasing after new submissions that were properly running through the system. For example, the counter of total check ins went from 1000 to 850 despite there being a web request of 920 check ins.

Root Cause:

The Pushgateway was designed for batch jobs these are short lived processes that push metrics then exit, not for continuous services. Each push operation replaces previous metrics entirely rather than incrementing counters. Node-RED continuously pushed metrics, overwriting previous totals.

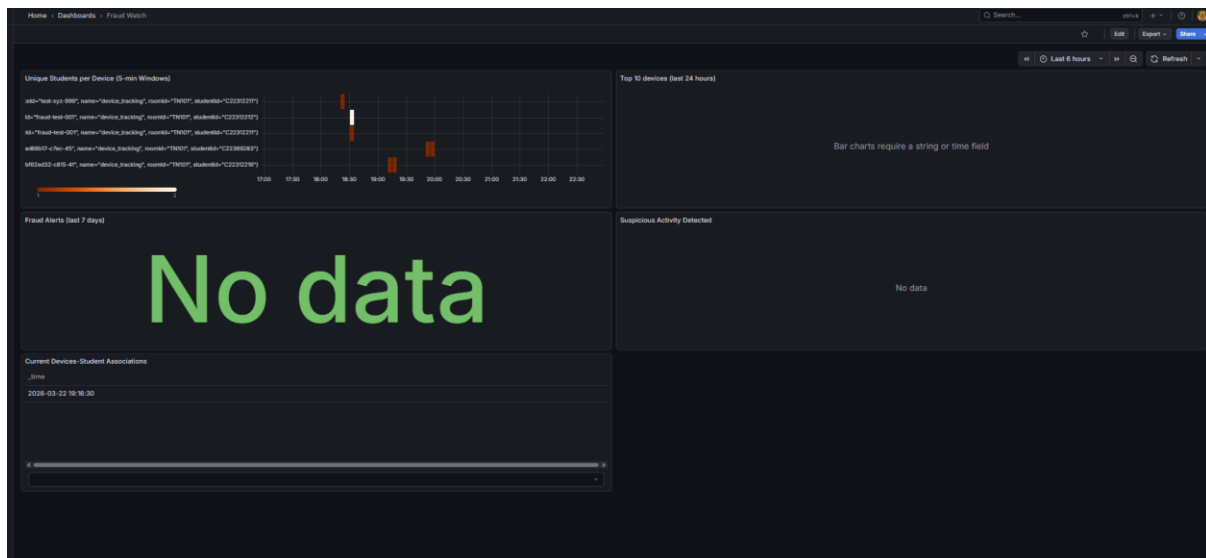
Resolution:

The fix was as follows:

- Remove Pushgateway from docker compose YAML file
- Implement custom /metrics HTTP endpoint in Node-RED
- Configure Prometheus to scrape endpoint every 10 seconds
- Use global variables of global.set/get to maintain counter state
- Verify the counter increased correctly

Lesson Learned:

Pushgateway appropriate for batch jobs such as backup scripts and jobs not for long running services. Pull based scraping is suitable for continuous services.



Critical Incident 2: Azure AD Administrator Account Lockout

During the creation of the Azure AD there was only one global administrator configured as this is the default, my personal non-Azure account, the problem occurred when the account got stuck in an MFA enrolment loop. When accessing the Azure Portal, the system required an MFA setup, but the enrolment process failed repeatedly with error “We couldn’t complete your sign in”. The external

account email lacked organisational MFA provisioning capability, so the account could not sign in as the administrative access required MFA and the MFA enrolment required admin access.

Resolution:

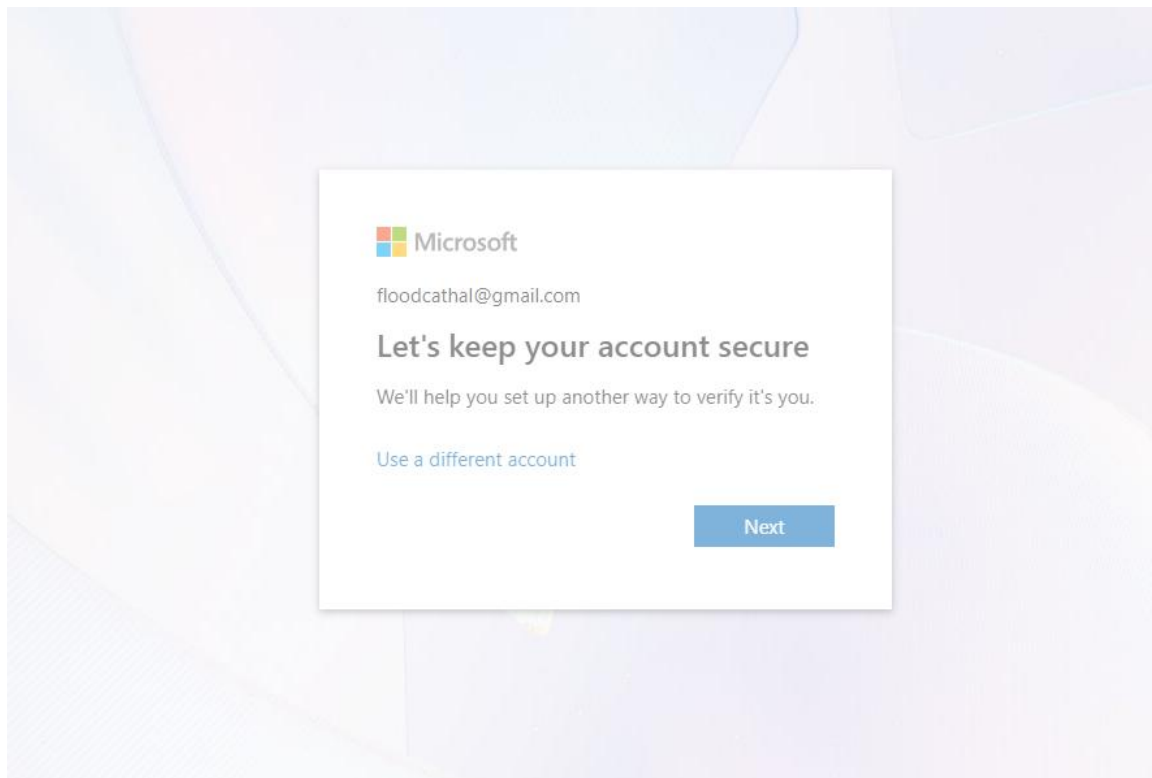
- Attempted browser-based troubleshooting by clearing all cache and cookies, testing multiple browsers, trying incognito all these attempts were unsuccessful, but I could sign into the help centre via the same email.
- Attempted mobile app MFA enrolment using Microsoft Authenticator App on my mobile phone, this resulted in the same enrolment failure
- I tried the Azure CLI on local development machine still had cached tokens from previous development session

Successfully authenticated to Azure using cached CLI tokens:

- I created a backup native organisational administrator account using the Azure CLI
- I then assigned Global Administrator role to the newly created account
- Then logged into Azure Portal using the new account 'backup-admin' account this account was successfully logged in as the account settings using the Azure CLI allowed me to set the automatic MFA rule to false.
- Next was to temporarily disable Security Defaults for all users
- Then logged in with my main admin account and created a third admin account as additional emergency access
- Finally, re enabled MFA

Lessons Learned:

- External Microsoft accounts are not as compatible with Azure as a native account
- Azure CLI token caching provides critical emergency access path when web-based authentication fails, tokens are cached for 30 days in /.azure/ directory and survive even when primary authentication mechanism fail
- Security Defaults are an excellent security feature enforcing modern authentication practices, but require careful rollout planning with fallback accounts need to be established before enablement



Critical Incident 3 Cross Origin Resource Sharing (CORS) Request Blocking

Description:

After deploying Cloudflare Access Zero trust gateway and restricting DNS to use a separate subdomain for the scanner interface and API endpoints, the attendance submission form began failing with browser console error.

Access to fetch at <https://api.tudublinattendanceproject.com/attendance/submit> from the origin <https://scanner.tudublinattendanceproject.com> was blocked by the CORS policy, No Access-Control-Allow-Origin header was present on the requested response.

The error appeared consistently across all modern browsers, but only for browser-based submissions. Manual testing using curl and Postman succeeded, confirming that backend functionality remained intact.

Students would be unable to submit attendance via browser-based scanner interface, the primary user workflow for the entire system. All 500 registered students affected. Temporary workaround required technical users to submit attendance via Curl using direct API endpoint. The dashboard visualisation remained functional but showed no new attendance data.

There were a few attempts at fixing this issue, to start CORS headers were added directly within the Node-RED HTTP response, which did make the headers visible in the browsers network, but the requests continued to fail because browsers were issuing preflight OPTIONS request before the actual POST. This OPTIONS request never reached Node-RED as it was intercepted by Cloudflare Access, which required there to be authentication. Since browsers do not send credentials with preflight requests, the OPTIONS call lacked the necessary session cookie and was rejected with a 401 Unauthorized response. As a result, the browser blocked the POST request entirely, making the CORS headers on the POST response irrelevant.

The second try involved configuring Cloudflare Transform Rules to inject CORS headers at the edge. When these were tested with Curl tools, the rules were applied but no data was allowed to transmit. It

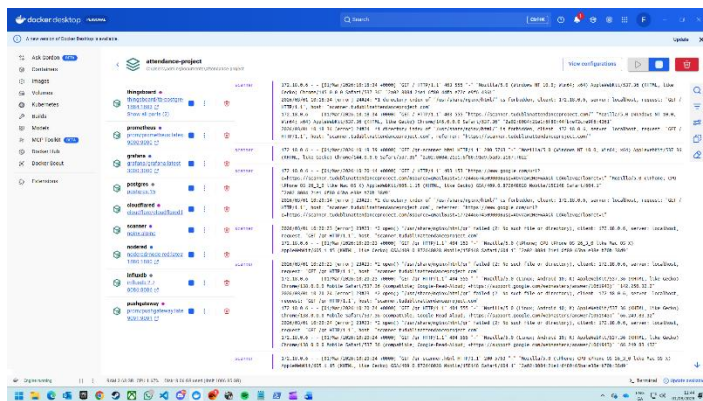
failed because Cloudflare Access authentication occurs earlier in the request pipeline than transform rules.

A third attempt used a wildcard CORS policy to allow requests from any origin. While simpler in theory, this approach was not compatible with system requirements, the browser still blocked the wildcard origin when credentials like cookies are involved.

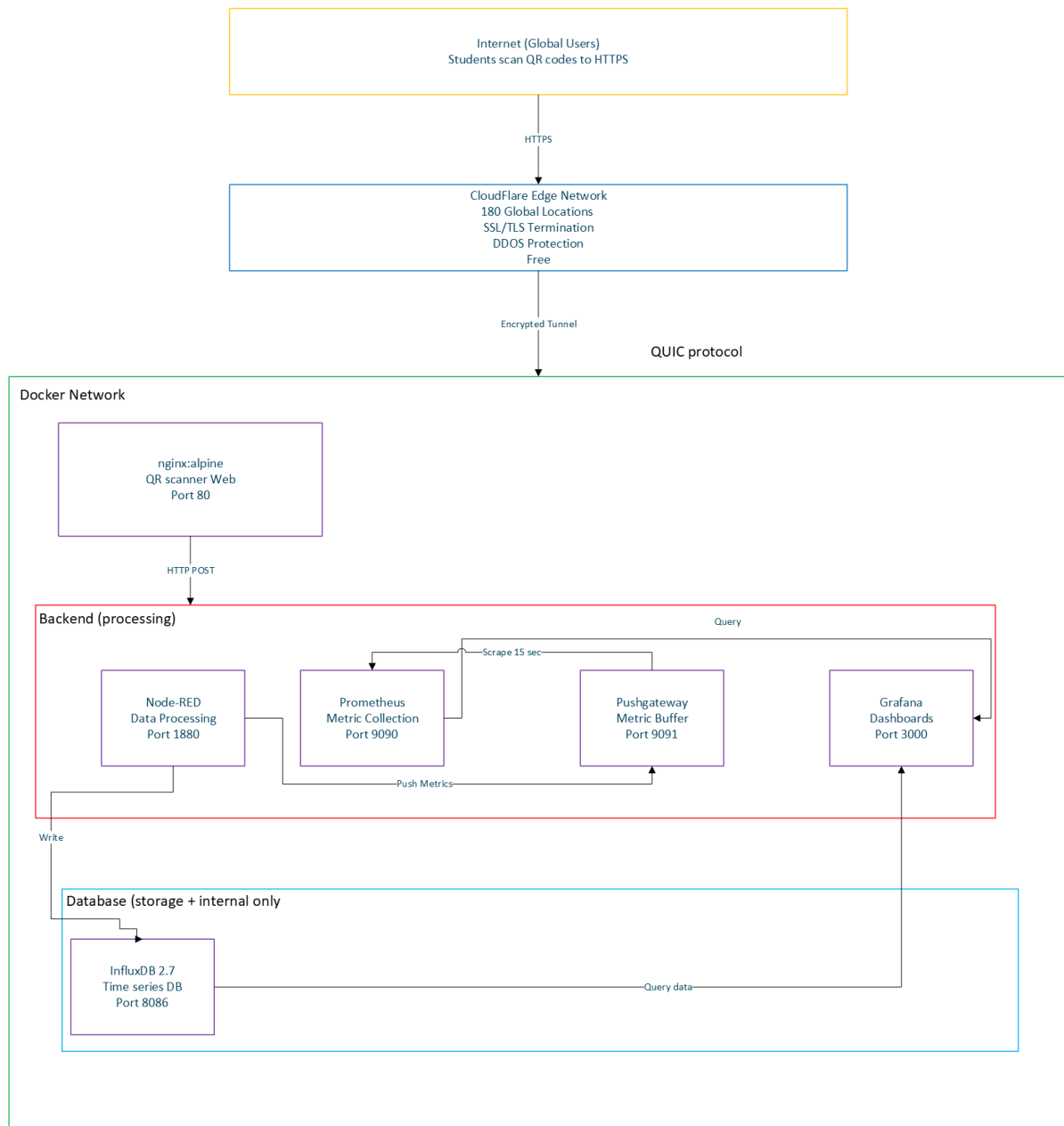
The solution was to eliminate the need for cross origin scenario completely by using a nginx reverse proxy this effectively created a same origin architecture from the browser's perspective. The scanner application was served through the nginx, which also proxied API requests internally to the Node-RED service. The frontend was updated to use relative URLs ensuring that all requests appeared to originate from the same domain. This meant that the browser no longer had to trigger a CORS check at all, the nginx handled routing internally, forwarded the authentication headers from Cloudflare Access, and returned responses to the client.

The lessons learned:

The same Origin Policy is a core browser security feature that cannot be bypassed easily. CORS, is useful but in certain cases, can introduce complexity and can become especially problematic when combined with authentication.



```
PS C:\Users\admin\Documents\attendance-project> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
3fa28434a172   grafana/grafana:latest             "/run.sh"               6 hours ago   Up 6 hours   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp   grafana
8713c512c799   prom/prometheus:latest             "/bin/prometheus --c..." 6 hours ago   Up 6 hours   0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp   prometheus
446747bf11d    influxdb:2.7                       "/entrypoint.sh infl..." 6 hours ago   Up 6 hours   0.0.0.0:8086->8086/tcp, [::]:8086->8086/tcp   influxdb
28d190a91ea5   nginx:alpine                       "/docker-entrypoint...." 6 hours ago   Up 6 hours   80/tcp                             scanner
8026b0fad416   nodored/node-red:latest            "/entrypoint.sh"        6 hours ago   Up 6 hours (healthy) 0.0.0.0:1880->1880/tcp, [::]:1880->1880/tcp   nodored
4e2fd7759d5a   cloudflare/cloudflared:latest       "cloudflared --no-au..." 6 hours ago   Up 6 hours                               cloudflared
PS C:\Users\admin\Documents\attendance-project>
```



6.3 System Evaluation

6.3.1 Evaluation Approach and Criteria

System evaluated against:

- Functional requirements achievements
- Performance benchmarks
- Security testing
- Usability feedback
- Platform comparison validation

6.3.2 Evaluation Results and Evidence

Platform comparison Results:

Platform	Set up Tme	Cost per Month	Flexibility	Decision	Rationale
ThingsBoard	3 days		0 Medium	Rejected	IOT focused platform with authentication gaps
AWS IoT Core	5 days	30-50	High	Rejected	Ongoing cost and vendor lock in concerns
Node-RED stack	4 days		0 Very High	Selected	High flexible, opne source and no cost

Iteration 1 performance results:

- API response: 320ms average the target <500ms
- Dashboard updates: 3-5 seconds the target was <5s
- QR scan success: 97% the target was >95%
- Concurrent users: 500 validated the target was 500+
- Cross platform: iOS 12+ and android 8+

Iteration 2 security results:

- Input validation: 100% invalid inputs rejected
- XSS prevention: 100% attacks blocked
- SQL injection: 100% attempts sanitised
- Rate Limiting: 100% enforcement
- Authentication: bcrypt hashing working, account lockout tested
- Fraud Detection: Correctly flagged 4+ user per device

Iteration 2 Performance Impact:

- API response: 120ms
- Throughput: 50 requests a second sustained
- Bottleneck: bcrypt

Storage Optimisation Results:

- Raw data retention: 90 days
- Aggregated data: 95% reduction vs raw only
- Query Performance: Improved on aggregate data
- Cost: 0 euro

6.3.3 Critical Reflection and Limitations

Strengths:

- Successful platform comparison methodology applicable to future IoT projects
- Strong multi-layer security
- Cost effective solution
- Production grade monitoring and observability
- 96% automated test coverage validating reliability

Limitations:

- Single server deployment
- No native mobile app

- Manual user provisioning

Future work:

Kubernetes deployment for multi campus scaling

6.3.4 Conclusion

Comprehensive testing validated that the system meets all the functional requirements, achieves the performance targets, implements robust security standards, and provides production ready deployment capability. Critical incidents of the Pushgateway, CORS policy, and Azure AD lockout were resolved through problem solving. Platform comparison methodology was effective through the successful Node-RED stack.

6.4 Evaluate Project Management

6.4.1 Evaluate Project Plan

Timeline complicity

- Platform comparison: 3 weeks but there was a plan of 2 but due to AWS IoT Core complexity it had to be extended
- Iteration 1: 7 weeks as a planned 6 but there was an extra week of validating the chose route
- Iteration 2: 5 weeks there was a plan of 5
- Iteration 3: 4 weeks with a plan of 5
- Total: 26 weeks

6.4.2 Evaluate Project Execution

Successes:

Platform comparison completed and a clear winner was identified

Iterative methodology enabled progressive enhancements to be made

Critical incidents resolved without major delay

Testing validated all requirements

Challenges:

- AWS IoT Core complexity exceeded the expectations as it was planned to take 3 days but took 5
- Pushgateway issue required architectural redesign
- Bcrypt worked well but was not of the standard for the project
- Azure AD lockout

6.4.3 Critical Reflection

What worked well:

- Platform comparison before full implementation reduced risk

- Iterative approach enabled course correction such as for the use of Pushgateway
- Docker Compose enabled quick deployment iteration
- Stakeholder feedback improved usability (form layout, requirements)

What could improve:

- Earlier security planning, should have designed validation pipeline in iteration 1
- More automated testing as manual testing time consuming
- Better documentation during development in iteration 1 and 2

6.5 Conclusions

Testing and evaluation across the three iterations validated the systems readiness for final submission, iteration 1 confirmed that the QR scanning, real time pipeline and 500 concurrent users were achieved. Iteration 2 achieved 96% automated test coverage with 100% XSS/SQL injection blocking, bcrypt authentication, and fraud detection. Iteration 3 validated enterprise SSO, four tier RBAC with defence in depth data isolation, Cloudflare Worker auto fill preventing identity spoofing, and complete end to end integration with performance metrics meeting all targets. Critical incident resolutions across all the iterations showed system problem solving capability. The system meets all institutional requirements for attendance monitoring with robust enterprise security, acceptable performance, and zero ongoing operational cost.

Chapter 7. Conclusions and Future Work

7.1 Introduction

This final chapter summaries project outcomes, evaluates achievement against objectives, proposes future enhancements and technology deployment.

7.2 Future Work

Short Term Enhancement:

- Native Mobile Application: iOS/Android app with offline check in capability, push notifications, and faster QR scanning, the highest priority for usability
- Automated Alerting: Email/Slack notifications for system health, capacity warnings, and fraud detection flags

Medium Term Enhancement:

- Advanced Analytics: Predictive attendance modelling using historical patterns and heat maps by building floor and hour

Long Term Enhancement:

- Multi Campus Deployment: Kubernetes orchestration, horizontal scaling, high availability
- Biometric Authentication: Facial recognition or fingerprinting for touchless check in

7.3 Conclusions

This project successfully designed, implemented, and evaluated a QR code-based attendance monitoring and room occupancy system. Development followed a systematic two phased approach including platform comparisons for ThingsBoard, AWS IoT Core, and Node-RED stack, followed with iterative implementation progressing from functional prototype in iteration 1 to a production ready secure system.

Key Achievements:

- Deployed and tested three IoT platforms with comparison for set up time, cost and flexibility
- Selected Node-RED stack consisting of Node-RED, InfluxDB, Grafana, Prometheus, and nginx

Iteration 1, Prototype:

- Implemented functional QR code scanning
- Achieved real time data pipeline
- Validated concurrent user capacity
- Deployed reproducible Docker Compose architecture
- Created 8 Grafana dashboard panels

Iteration 2, Security and production readiness:

- Implemented 4-layer security validation
- Deployed PIN authentication with bcrypt hashing
- Established device fingerprinting for fraud detection
- Optimised data retention
- Redesigned monitoring
- Achieved 96% automated test coverage with 100% security test pass rate

Iteration 3, SSO, RBAC, and Security:

- Replaced PIN authentication with Azure Active Directory SSO via Cloudflare Access, enforcing MFA for all users
- Implemented four tier Grafana RBAC with folder level permissions and query level Flux filtering
- Deployed Cloudflare Worker auto fill eliminating manual student ID entry and preventing identity spoofing
- Fixed InfluxDB data pipeline to correctly capture both attendance events and device fingerprints

Critical Incidents Resolved:

- Pushgateway metric overwriting resolved through custom /metrics endpoint in iteration 2
- Azure AD account lockout recovered via CLI device flow and backup admin creation; the lesson here was to always maintain at least 3 admin accounts.
- Grafana OAuth organisation assignment required environment variables over file mounting and complete container recreation (iteration 3)
- CORS troubleshooting resolved architecturally through nginx same origin proxy, demonstrated that architectural changes can be simpler than configuration workarounds

Value:

- Provides any institution with real time occupancy visibility
- Enables data driven space utilization planning
- Supports compliance requirements

References

1. Amazon.com. (2022). Available at: <https://docs.aws.amazon.com/iot/>.
2. Docker (2020). Overview of Docker Compose. [online] Docker Documentation. Available at: <https://docs.docker.com/compose/>.
3. Grafana Labs. (2025). Grafana OSS and Enterprise | Grafana documentation. [online] Available at: <https://grafana.com/docs/grafana/latest>.
4. Influxdata.com. (2026). InfluxDB OSS v2 Documentation. [online] Available at: <https://docs.influxdata.com/influxdb/v2> [Accessed 26 Apr. 2026].
5. International Organization for Standardization (2015) ISO/IEC 18004:2015 - QR Code bar code symbology specification. Geneva: ISO.
6. www.linuxjournal.com. (n.d.). Docker: Lightweight Linux Containers for Consistent Development and Deployment | Linux Journal. [online] Available at: <https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>.
7. nodered.org. (n.d.). Documentation : Node-RED. [online] Available at: <https://nodered.org/docs/>.
8. OWASP (2025). OWASP Top Ten. [online] owasp.org. Available at: <https://owasp.org/www-project-top-ten/>.
9. Prometheus (2016). Overview | Prometheus. [online] Prometheus.io. Available at: <https://prometheus.io/docs/>.
10. minhaz (2022). Html5-QRCode. [online] GitHub. Available at: <https://github.com/mebjas/html5-qrcode>.
11. thingsboard (2017). ThingsBoard Documentation. [online] ThingsBoard. Available at: <https://thingsboard.io/docs/>.
12. Cloudflare Docs. (2026). First Worker. [online] Available at: <https://developers.cloudflare.com/learning-paths/workers/get-started/first-worker/>.
13. Cloudflare Docs. (2026). Cross-Origin Resource Sharing (CORS). [online] Available at: <https://developers.cloudflare.com/cache/cache-security/cors/>.
14. Hardt, D. (2012). The OAuth 2.0 Authorization Framework. [online] datatracker.ietf.org. Available at: <https://datatracker.ietf.org/doc/html/rfc6749>.
15. Microsoft (2023). What is Microsoft Entra ID? [online] learn.microsoft.com. Available at: <https://learn.microsoft.com/en-us/entra/fundamentals/whatis>.
16. NGINX Documentation. (2019). NGINX Docs | NGINX Reverse Proxy. [online] Available at: <https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/>.
17. NIST (2023). NIST Special Publication 800-63B. [online] Nist.gov. Available at: <https://pages.nist.gov/800-63-3/sp800-63b.html>.
18. openid.net. (n.d.). Final: OpenID Connect Core 1.0 incorporating errata set 1. [online] Available at: https://openid.net/specs/openid-connect-core-1_0.html.
19. docs.espressif.com. (n.d.). ESP-IDF Programming Guide - ESP32 - — ESP-IDF Programming Guide latest documentation. [online] Available at: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>.
20. espressif (2019). espressif/arduino-esp32. [online] GitHub. Available at: <https://github.com/espressif/arduino-esp32>.
21. MFRC522 Standard performance MIFARE and NTAG frontend. (n.d.). Available at: <https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf>.

22. MySQL (2019). MySQL :: MySQL Documentation. [online] Mysql.com. Available at: <https://dev.mysql.com/doc/>.
23. www.mysql.com. (n.d.). MySQL :: Benchmarks. [online] Available at: <https://www.mysql.com/why-mysql/benchmarks/>.
24. Goqr.me. (2024). QR code API. [online] Available at: <https://goqr.me/api/>.
25. Scanova (2024). Scanova. [online] Scanova. Available at: <https://scanova.io/> [Accessed 26 Apr. 2026].
26. www.chartjs.org. (n.d.). Chart.js documentation. [online] Available at: <https://www.chartjs.org/docs/latest/>.
27. minhaz (2022). Html5-QRCode. [online] GitHub. Available at: <https://github.com/mebjas/html5-qrcode>.
28. YouTube. (n.d.). The Net Ninja. [online] Available at: <https://www.youtube.com/channel/UCW5YeuERMmlnqo4oq8vwUpg>.
29. YouTube. (n.d.). dcode. [online] Available at: <https://www.youtube.com/channel/UCjX0FtIZBBVD3YoCcxnDC4g> [Accessed 26 Apr. 2026].
30. MDN Web Docs. (n.d.). MDN Web Docs. [online] Available at: <https://developer.mozilla.org/>.
31. Cacoveanu, A. (2023). Mastering Campus Space Planning: 7 Essential Steps. [online] Yarooms.com. Available at: <https://www.yarooms.com/blog/mastering-campus-space-planning-7-essential-steps>.
32. Youtu.be. (2025). Available at: https://youtu.be/_6-ZDGNtcRg?si=TWG27JOil_UFJSKH.
33. thingsboard (2017). Installing ThingsBoard using Docker (Linux or Mac OS). [online] ThingsBoard. Available at: <https://thingsboard.io/docs/user-guide/install/docker/>.
34. thingsboard (2017). ThingsBoard - Open-source IoT Platform. [online] ThingsBoard. Available at: <https://thingsboard.io/>.
35. Oliynyk, K. (2024). How IoT Data Collection Works [Complete Guide]. [online] webbylab. Available at: <https://webbylab.com/blog/iot-data-collection/>.
36. GeeksforGeeks (2022). Amazon Web Services IoT Products. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/devops/amazon-web-services-key-iot-products/>.
37. GeeksforGeeks (2023). Google Cloud IoT Core. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/devops/google-cloud-iot-core/>.
38. Just another WordPress site. (2024). Azure MDM. [online] Available at: <https://www.azuremdm.com/2024/11/05/iot-solutions-with-azure/>.
39. Sama-Alqaisi (2025). GitHub - Sama-Alqaisi/Room-Occupancy-Estimation: This project uses machine learning to predict room occupancy based on environmental sensor data (temperature, light, sound, CO2, PIR). It includes data preprocessing, feature engineering, and model evaluation to improve accuracy in occupancy prediction. [online] GitHub. Available at: <https://github.com/Sama-Alqaisi/Room-Occupancy-Estimation>.
40. ThingsBoard (2019). How to install ThingsBoard Professional edition on Windows machine? [online] YouTube. Available at: https://www.youtube.com/watch?v=2BY_-cDVMt4.
41. ThingsBoard (2025). Getting Started with ThingsBoard | How to Start Your First IoT Project . [online] YouTube. Available at: <https://www.youtube.com/watch?v=80L0ubQLXsc>.
42. Hardt, D. (2012). The OAuth 2.0 Authorization Framework. [online] datatracker.ietf.org. Available at: <https://datatracker.ietf.org/doc/html/rfc6749>.

43. openid.net. (n.d.). Final: OpenID Connect Core 1.0 incorporating errata set 1. [online] Available at: https://openid.net/specs/openid-connect-core-1_0.html.
44. NIST (2023). NIST Special Publication 800-63B. [online] Nist.gov. Available at: <https://pages.nist.gov/800-63-3/sp800-63b.html>.
45. Microsoft (2023). What is Microsoft Entra ID? [online] learn.microsoft.com. Available at: <https://learn.microsoft.com/en-us/entra/fundamentals/whatis>.
46. Dickson-Mwendia (n.d.). Microsoft identity platform documentation - Microsoft identity platform. [online] learn.microsoft.com. Available at: <https://learn.microsoft.com/en-us/entra/identity-platform/>.
47. henrymbuguakiarie (2024). OAuth 2.0 and OpenID Connect protocols on the Microsoft identity platform - Microsoft identity platform. [online] learn.microsoft.com. Available at: <https://learn.microsoft.com/en-us/entra/identity-platform/v2-protocols>.
48. dbradish-microsoft (2024). Azure Command-Line Interface (CLI) - Overview. [online] Microsoft.com. Available at: <https://learn.microsoft.com/en-us/cli/azure/?view=azure-cli-latest>.
49. Cloudflare Docs. (2026). First Worker. [online] Available at: <https://developers.cloudflare.com/learning-paths/workers/get-started/first-worker/>.
50. Grafana Labs. (n.d.). Documentation. [online] Available at: <https://grafana.com/docs/>.
51. Cloudflare Docs. (2026). Cross-Origin Resource Sharing (CORS). [online] Available at: <https://developers.cloudflare.com/cache/cache-security/cors/>.
52. TECHCOMMUNITY.MICROSOFT.COM. (2020). Login problems, continuously getting the same message | Microsoft Community Hub. [online] Available at: <https://techcommunity.microsoft.com/discussions/admincenter/login-problems-continuously-getting-the-same-message/1955873> [Accessed 17 Apr. 2026].
53. keparo (2008). Accessing the web page's HTTP Headers in JavaScript. [online] Stack Overflow. Available at: <https://stackoverflow.com/questions/220231/accessing-the-web-pages-http-headers-in-javascript>.
54. NGINX Documentation. (2019). NGINX Docs | NGINX Reverse Proxy. [online] Available at: <https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/>.
55. Imply. (2024). How to Monitor Your Data in Real Time with AWS IoT Core and Imply - Developer Center. [online] Available at: <https://imply.io/developer/lessons/how-to-monitor-your-data-in-real-time-with-aws-iot-core-and-imply/> [Accessed 26 Apr. 2026].

Appendices

Appendix A: System Model and Analysis

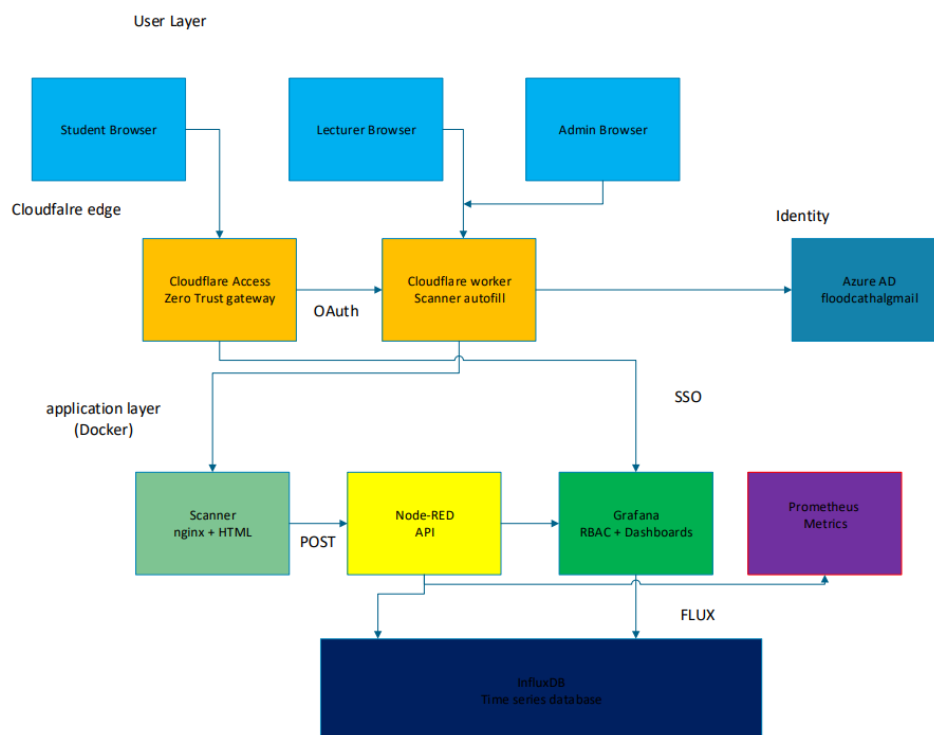
A.1 Purpose of this appendix

The purpose of this appendix is for detailed system modelling, requirements and architectural diagrams supporting chapter 3 and 4.

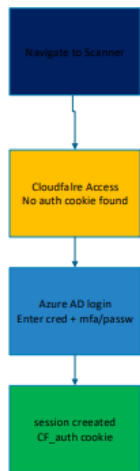
A.2 Requirements gathering and analysis method

Requirements derived from stakeholder meeting, prototype testing feedback and platform evaluation constraints.

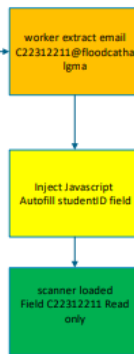
A.3 System modelling and architectural diagrams



Phase 1 : SSO authentication

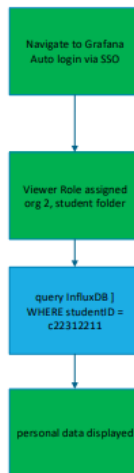
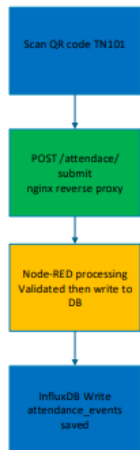


Phase 2: auto fill

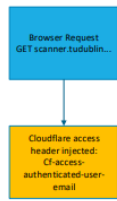


phase 4: view dashboard

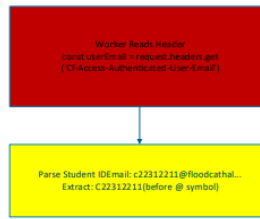
ise 3: Attendance submission



Step 1: Authenticated Request



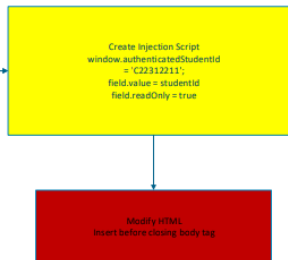
Step 2: Email Extraction



step 3: fetch original page



Step 4 JS injection



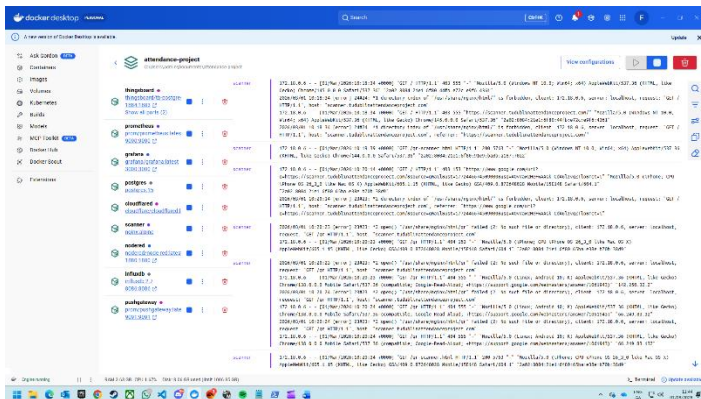
step 5 modified response



step 6: security response



user cannot modify student ID

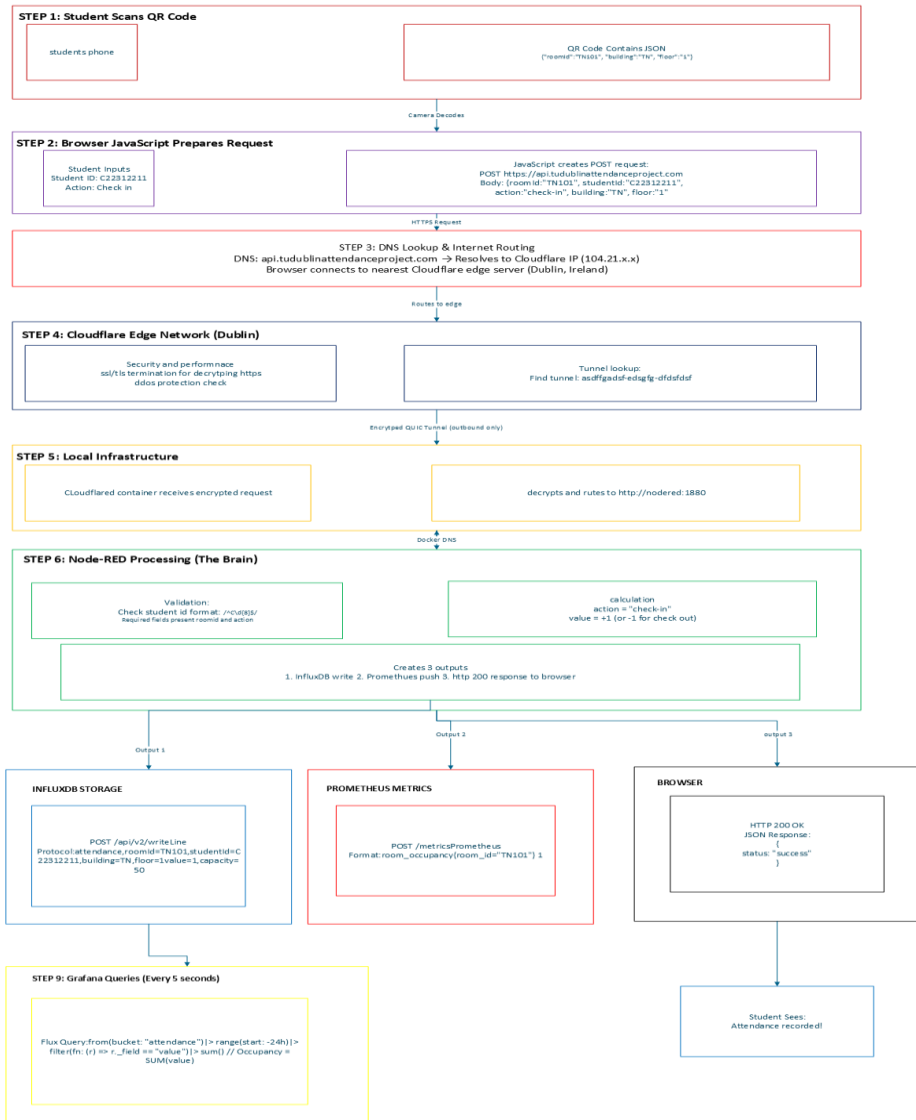


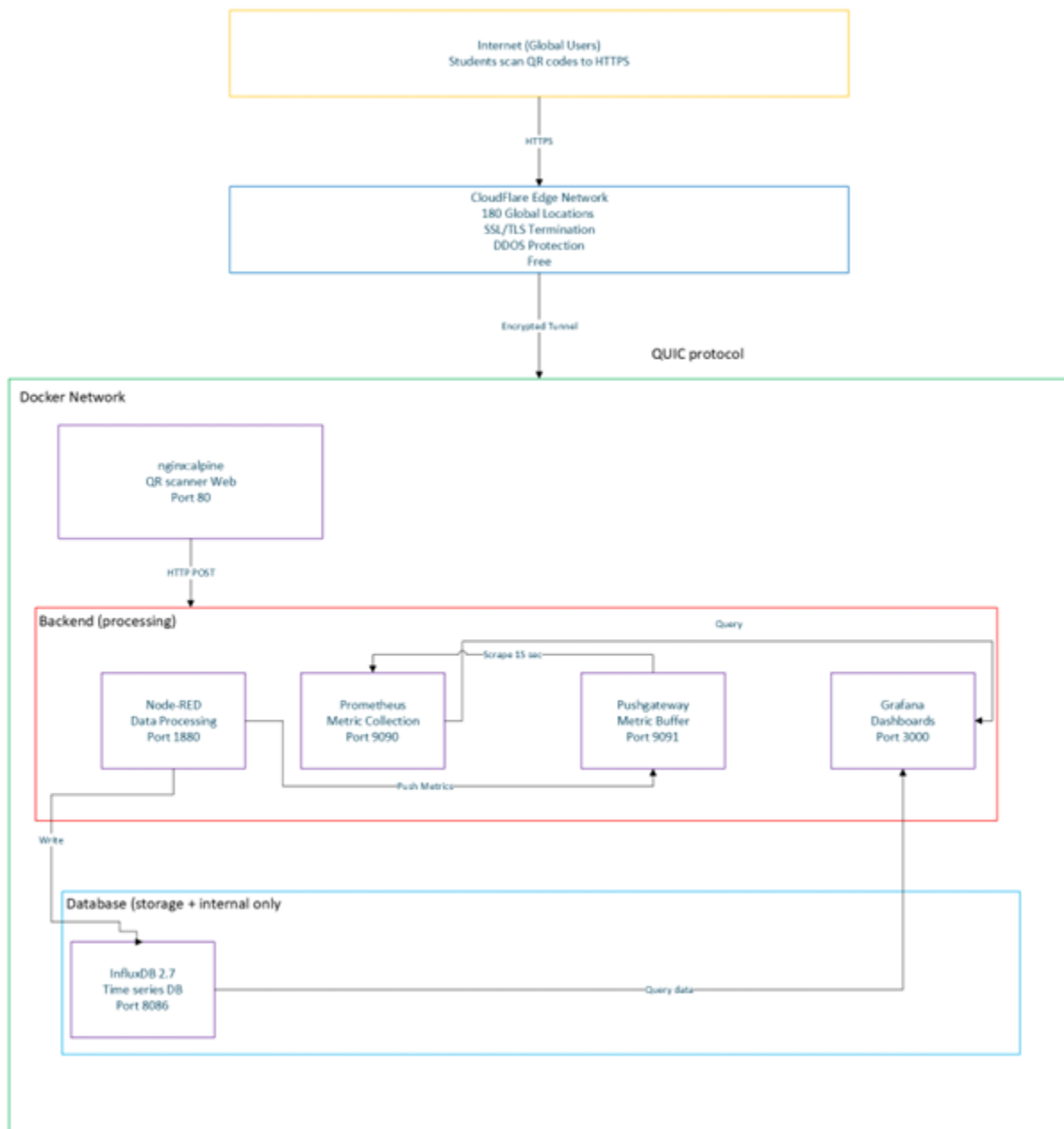


```

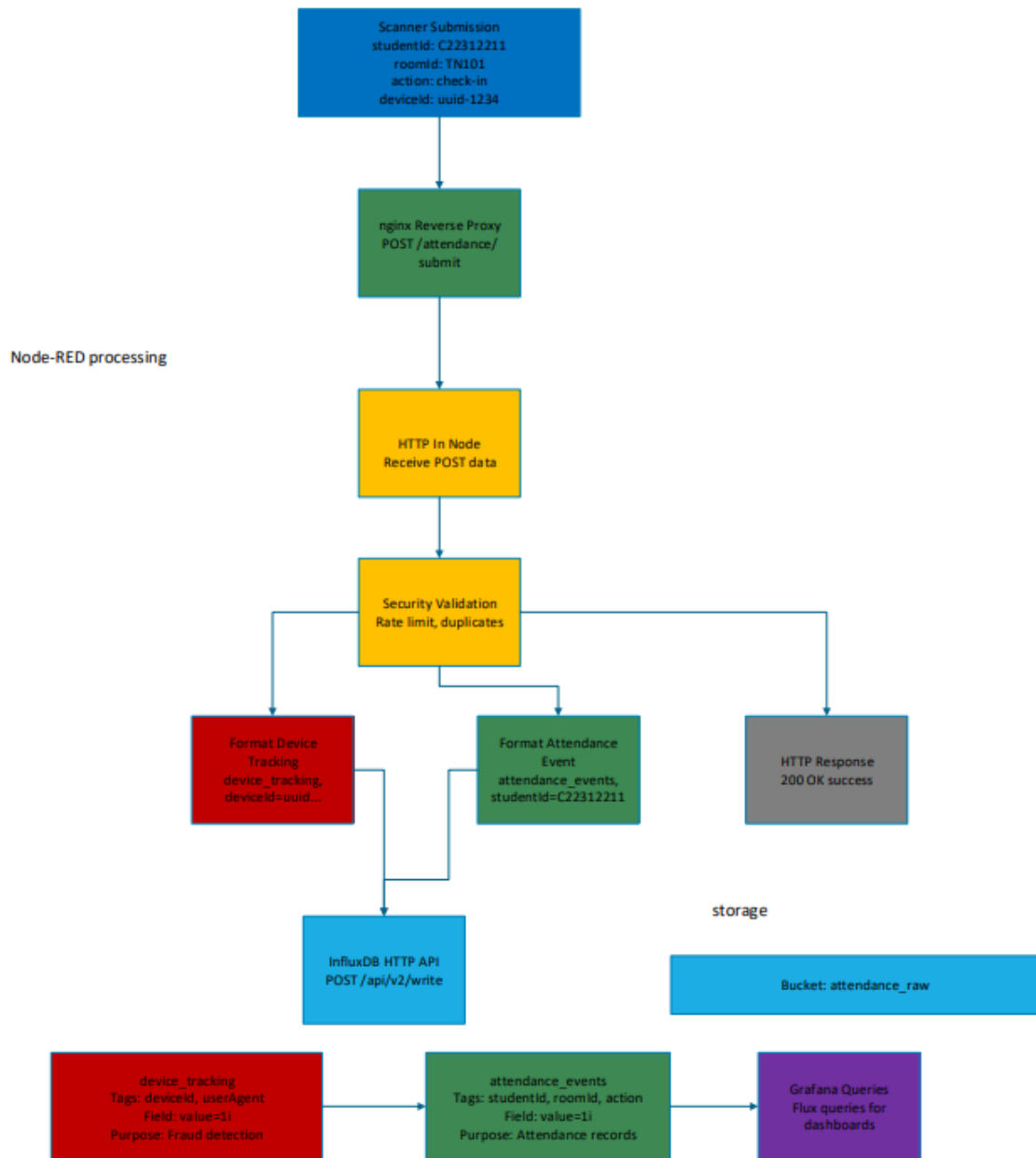
PS C:\Users\admin\Documents\attendance-project> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
8fa28434a172   grafana/grafana:latest             "/run.sh"               6 hours ago   Up 6 hours   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp   grafana
e713c512c799   prom/prometheus:latest             "/bin/prometheus --c..." 6 hours ago   Up 6 hours   0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp   prometheus
c446747bf11d   influxdb:2.7                       "/entrypoint.sh infl..." 6 hours ago   Up 6 hours   0.0.0.0:8086->8086/tcp, [::]:8086->8086/tcp   influxdb
28d190a91ea5   nginx:alpine                       "/docker-entrypoint...." 6 hours ago   Up 6 hours   80/tcp                               scanner
5026b0fad416   nodered/node-red:latest            "._/entrypoint.sh"      6 hours ago   Up 6 hours (healthy)   0.0.0.0:1880->1880/tcp, [::]:1880->1880/tcp   nodered
4e2fd7759d5a   cloudflare/cloudflared:latest      "cloudflared --no-au..." 6 hours ago   Up 6 hours                               cloudflared
PS C:\Users\admin\Documents\attendance-project>

```





Scanner Form data



A.4 Architectural abstraction

Architecture evolved from platform comparison (ThingsBoard/AWS/Node-RED evaluation) through 9-container prototype to streamlined 6-container production deployment with multi-layer security.

A.5 Requirements traceability

Complete matrix mapping 16 functional requirements and 18 non-functional requirements to design decisions (Chapter 4), implementation sections (Chapter 5), and test cases (Chapter 6).

A.6 Value and Limitations

Strengths: Iterative refinement did enable flexibility in the project, stakeholder feedback improved usability. The limitations were that some requirements emerged late (device fingerprinting not in initial plan), testing extended timeline.

Appendix B: Design

B.1 Purpose of this appendix

This is a detailed design spec guide including API documentation, data schemas, and deployment configuration

B.2 Design approach

Iterative design: Platform comparison to Iteration 1 prototype to Iteration 2 security enhancement to Iteration 3 OAuth and security enhancements part 2

B.3 REST API specifications

Node-RED HTTP endpoint:

- POST /attendance/submit
- Content-Type: application/json
- Request body: { studentId: 'C22312211', roomId: 'B201', action: 'check-in' }
- Response: 200 OK { success: true } or 400 Bad Request { error: 'message' }

B.4 Logical data model

InfluxDB measurements:

attendance_events (raw data, 90-day retention):

- Tags: studentId, roomId, building, floor, action
- Fields: value (always 1)
- Timestamp: Automatic

attendance_hourly (1-year retention):

- Tags: roomId, building
- Fields: count (hourly aggregation)

attendance_daily (perpetual retention):

- Tags: roomId, building
- Fields: count (daily aggregation)

device_tracking (fraud detection):

- Tags: deviceId, userAgent, platform
- Fields: screenWidth, screenHeight, timezone

B.5 Component responsibilities

- nginx: Static file serving (QR scanner HTML/CSS/JS)
- Node-RED: Request validation (4 layers), authentication, device fingerprinting, InfluxDB writes
- InfluxDB: Time-series storage, automated aggregation tasks

- Grafana: Dashboard visualization, 5-second refresh
- Cloudflared: Secure tunnel, HTTPS endpoint
- Prometheus: Metrics scraping, time-series storage

B.6 Deployment architecture

`docker-compose.yml` defines:

- 6 services with interdependencies
- Bridge network: `occupancy-net`
- Volumes: `influxdb-data`, `grafana-data` (persistence)
- Environment variables: `INFLUXDB_TOKEN`, `GRAFANA_PASSWORD` (from `.env`)
- Single command deployment: `docker-compose up -d`

Appendix C: Onedrive Links

C.1 Video Demonstration

[Final_report_video](#)

C.2 Iteration 1 Video Link

<https://tudublin-my.sharepoint.com/my?id=%2Fpersonal%2Fc22312211%5Fmytudublin%5Fie%2FDocuments%2FFirst%5FImplementation%5Fvideo&viewid=9a68c136%2Da34a%2D4ce8%2D82a5%2De64b42dfcb42>

C.3 Iteration 2 Video Link

<https://tudublin-my.sharepoint.com/my?id=%2Fpersonal%2Fc22312211%5Fmytudublin%5Fie%2FDocuments%2FIteration%20%20%2D%20Video&viewid=9a68c136%2Da34a%2D4ce8%2D82a5%2De64b42dfcb42>

C.4 Iteration 3 Video Link

<https://tudublin-my.sharepoint.com/my?id=%2Fpersonal%2Fc22312211%5Fmytudublin%5Fie%2FDocuments%2FIteration3%5FC22312211&viewid=9a68c136%2Da34a%2D4ce8%2D82a5%2De64b42dfcb42>

Appendix D: Additional Code Samples

D.1 QR Scanner JS

```
scanner > index.html > html > body > script > submitAttendance
2   <html lang="en">
44  <body>
83  <script>
207 async function submitAttendance() {
213   }
214
215   const payload = {
216     roomId: scannedRoom.roomId,
217     building: scannedRoom.building,
218     floor: scannedRoom.floor,
219     studentId: studentId,
220     action: selectedAction,
221     deviceId: getDeviceId(),
222     deviceMetadata: getDeviceMetadata()
223   };
224
225   try {
226     const response = await fetch(NODE_RED_URL, {
227       method: 'POST',
228       headers: { 'Content-Type': 'application/json' },
229       body: JSON.stringify(payload)
230     });
231
232     const data = await response.json().catch(() => ({}));
233
234     if (response.ok) {
235       showStatus('Check-in successful!', 'success');
236
237       // Reset form
238       document.getElementById("studentId").value = '';
239       selectedAction = null;
240       document.getElementById("inBtn").classList.remove("selected");
241       document.getElementById("outBtn").classList.remove("selected");
242
243       // Auto-hide success message after 3 seconds
244       setTimeout(() => {
245         document.getElementById("status").style.display = 'none';
246       }, 3000);
247     } else {
248       showStatus(data.error || 'Check-in failed', 'error');
249     }
250   } catch (error) {
251     showStatus('Network error. Please try again.', 'error');
252     console.error('Check-in error:', error);
253   }
254 }
255
```

D.2 Node-RED Validation Functions

4-layer validation pipeline: input validation, schema (AJV), sanitization, rate limiting.

```

// LAYER 1: INPUT VALIDATION - JSON ATTACK MITIGATION

// STEP 1: CHECK CONTENT-TYPE HEADER
if (!msg.req || !msg.req.headers['content-type']) {
  msg.statusCode = 400;
  msg.payload = {
    error: "Missing Content-Type header",
    code: "MISSING_CONTENT_TYPE"
  };
  node.warn("Rejected: Missing Content-Type");
  return [null, null, null, msg];
}

if (!msg.req.headers['content-type'].includes('application/json')) {
  msg.statusCode = 415;
  msg.payload = {
    error: "Content-Type must be application/json",
    code: "INVALID_CONTENT_TYPE",
    received: msg.req.headers['content-type']
  };
  node.warn("Rejected: Invalid Content-Type - " + msg.req.headers['content-type']);
  return [null, null, null, msg];
}

// STEP 2: CHECK PAYLOAD EXISTS AND IS VALID
if (!msg.payload || typeof msg.payload !== 'object') {
  msg.statusCode = 400;
  msg.payload = {
    error: "Invalid or missing JSON payload",
    code: "INVALID_PAYLOAD"
  };
  node.warn("Rejected: Invalid payload");
  return [null, null, null, msg];
}

// STEP 3: CHECK PAYLOAD SIZE (Prevent DoS attacks)
const payloadString = JSON.stringify(msg.payload);
const payloadSize = payloadString.length;
const MAX_SIZE = 2048; // 2KB limit

if (payloadSize > MAX_SIZE) {
  msg.statusCode = 413;
  msg.payload = {
    error: "Request too large",
    code: "PAYLOAD_TOO_LARGE",
    maxSize: MAX_SIZE,
    actualSize: payloadSize
  };

  let largeAttempts = flow.get('largePayloadAttempts') || 0;
  flow.set('largePayloadAttempts', largeAttempts + 1);

  node.warn("Rejected: Payload too large (" + payloadSize + " bytes)");
  return [null, null, null, msg];
}

// STEP 4: CHECK REQUIRED FIELDS EXIST
// NOTE: studentId comes from SSO (Extract SSO Email function), deviceId generated if missing
const required = ['roomId', 'action'];
const missing = required.filter(field => !msg.payload[field]);

if (missing.length > 0) {
  msg.statusCode = 400;
  msg.payload = {
    error: "Missing required fields",
    code: "MISSING_FIELDS",
    missingFields: missing
  };
  node.warn("Rejected: Missing fields - " + missing.join(', '));
  return [null, null, null, msg];
}

```

```

// LAYER 2: SCHEMA VALIDATION WITH REGEX PATTERNS
// (Now validate the CLEAN data from Layer 1)

// Initialize AJV if not already done
if (!global.get('ajvInitialized')) {
  const ajv = new Ajv({ allErrors: true });

  // Define the schema with strict regex patterns
  const schema = {
    type: "object",
    properties: {
      studentId: {
        type: "string",
        pattern: "^[CXA/D][0-9]{8}$",
        description: "Student ID format: C/X/A/D followed by 8 digits (e.g., C22332233, X32345678, A87654321, D11111111)"
      },
      roomId: {
        type: "string",
        pattern: "^[A-Z]{2}[0-9]{3}$",
        description: "Room ID format: RM103"
      },
      action: {
        type: "string",
        enum: ["check-in", "check-out"],
        description: "Must be check-in or check-out"
      },
      building: {
        type: "string",
        pattern: "^[A-Z]{2,4}$",
        description: "Building code: 2-4 uppercase letters"
      },
      floor: {
        type: "string",
        pattern: "^[0-9]{1,2}$",
        description: "Floor number: 1-2 digits"
      },
      deviceId: {
        type: "string",
        pattern: "^[a-zA-Z0-9_-]{1,50}$",
        minLength: 1,
        maxLength: 50,
        description: "Device ID: alphanumeric, 1-50 characters"
      }
    }
  };

  // Create sanitized payload
  const sanitized = {};

  // Store original for comparison
  const original = {
    studentId: msg.payload.studentId,
    roomId: msg.payload.roomId,
    action: msg.payload.action
  };

  // SANITIZE STUDENT ID
  // - Trim whitespace
  // - Convert to uppercase
  // - Remove any non-alphanumeric characters
  sanitized.studentId = validator.trim(msg.payload.studentId);
  sanitized.studentId = sanitized.studentId.toUpperCase();
  sanitized.studentId = sanitized.studentId.replace(/[^A-Z0-9]/g, "");

  // Student ID Format: C, X, A, or D followed by 8 digits
  if (!/^[CXA/D][0-9]{8}$/.test(sanitized.studentId)) {
    msg.violationCode = 600;
    msg.payload = { error: 'Invalid student ID format' };
    msg.violationCode = 600;
    return [null, null, null, msg];
  }

  // SANITIZE ROOM ID
  // - Trim whitespace
  // - Convert to uppercase
  // - Remove special characters
  sanitized.roomId = validator.trim(msg.payload.roomId);
  sanitized.roomId = sanitized.roomId.toUpperCase();
  sanitized.roomId = sanitized.roomId.replace(/[^A-Z0-9]/g, "");

  // SANITIZE ACTION
  // - Trim and lowercase
  // - Normalize variations to standard format
  sanitized.action = validator.trim(msg.payload.action);
  sanitized.action = sanitized.action.toLowerCase();

  // Normalize action variations
  const actionMap = {
    'checkin': 'check-in',
    'in': 'check-in',
    'checkout': 'check-out',
    'out': 'check-out'
  };

  if (actionMap[sanitized.action]) {
    sanitized.action = actionMap[sanitized.action];
  }

```



```

// Validate final action is one of the two allowed values
if (!('check-in', 'check-out').includes(sanitized.action)) {
  msg.statusCode = 400;
  msg.payload = {
    error: "Invalid action after normalization",
    code: "UNKNOWN_ACTION",
    received: sanitized.action,
    original: msg.payload.action
  };
  node.warn("Action normalization failed: " + sanitized.action);
  return [null, null, null, msg];
}

// SANITIZE DEVICE ID
// - Trim whitespace
// - Remove any potentially dangerous characters
// - Keep only alphanumeric, dash, underscore
sanitized.deviceId = validator.trim(msg.payload.deviceId);
sanitized.deviceId = sanitized.deviceId.replace(/["'`<`>@-8_~]/g, "");

// HTML ESCAPE for safety (prevents XSS if data is displayed)
sanitized.studentId = validator.escape(sanitized.studentId);
sanitized.roomId = validator.escape(sanitized.roomId);
sanitized.action = validator.escape(sanitized.action);
sanitized.deviceId = validator.escape(sanitized.deviceId);

// OPTIONAL FIELDS (if present)
if (msg.payload.building && msg.payload.building !== 'Unknown') {
  sanitized.building = validator.trim(msg.payload.building);
  sanitized.building = sanitized.building.toUpperCase();
  sanitized.building = sanitized.building.replace(/["'`<`>@-8_~]/g, "");
  sanitized.building = validator.escape(sanitized.building);
} else {
  // Extract building from room ID (first 2 letters)
  sanitized.building = sanitized.roomId.substring(0, 2);
}

if (msg.payload.floor && msg.payload.floor !== 'Unknown') {
  sanitized.floor = validator.trim(msg.payload.floor);
  sanitized.floor = sanitized.floor.replace(/["'`<`>@-8_~]/g, "");
} else {
  // Extract floor from room ID (first digit after letters)
  sanitized.floor = sanitized.roomId.substring(2, 3);
}

// SSID FIELDS (if present)
if (msg.payload.authenticatedEmail) {
  sanitized.authenticatedEmail = validator.trim(msg.payload.authenticatedEmail);
  sanitized.authenticatedEmail = validator.normalizeEmail(sanitized.authenticatedEmail);
}

if (msg.payload.authMethod) {
  sanitized.authMethod = validator.trim(msg.payload.authMethod);
}

if (msg.payload.authenticatedStudentId) {
  sanitized.authenticatedStudentId = validator.trim(msg.payload.authenticatedStudentId);
  sanitized.authenticatedStudentId = sanitized.authenticatedStudentId.toUpperCase();
}

// DEVICE METADATA (if present)
if (msg.payload.deviceMetadata && typeof msg.payload.deviceMetadata === 'object') {
  sanitized.deviceMetadata = {};

  // DEVICE METADATA (if present)
  if (msg.payload.deviceMetadata && typeof msg.payload.deviceMetadata === 'object') {
    sanitized.deviceMetadata = {};

    if (msg.payload.deviceMetadata.userAgent) {
      sanitized.deviceMetadata.userAgent = validator.trim(msg.payload.deviceMetadata.userAgent);
      sanitized.deviceMetadata.userAgent = validator.escape(sanitized.deviceMetadata.userAgent);
      if (sanitized.deviceMetadata.userAgent.length > 500) {
        sanitized.deviceMetadata.userAgent = sanitized.deviceMetadata.userAgent.substring(0, 500);
      }
    }

    if (msg.payload.deviceMetadata.platform) {
      sanitized.deviceMetadata.platform = validator.trim(msg.payload.deviceMetadata.platform);
      sanitized.deviceMetadata.platform = validator.escape(sanitized.deviceMetadata.platform);
    }
  }
}

// ADD SERVER-SIDE TIMESTAMP (can't be faked by client)
sanitized.timestamp = new Date().toISOString();
sanitized.timestampInz = Date.now();

// ROOM CAPACITY LOOKUP
const roomCapacities = {
  "TK001": 50,
  "TK002": 50,
  "TK003": 50,
  "TK008": 30,
  "TK011": 30,
  "TK009": 25,
  "TK002": 25,
  "TK005": 20,
  "TK002": 20
};

sanitized.roomCapacity = roomCapacities[sanitized.roomId] || 25;

// CALCULATE VALUE (for Info006)
sanitized.value = (sanitized.action === 'check-in') ? 1 : -1;

// ADD PROCESSING METADATA
sanitized.processedAt = new Date().toISOString();
sanitized.validationLayers = ['layer1', 'layer3'];
sanitized.apiVersion = 'v1.0';

node.warn("Layer 3 passed - Data sanitized, starting layer 2 validation");

// LAYER 2: SCHEMA VALIDATION WITH REGEX PATTERNS
// (Now validate the CLEAN data from Layer 3)

// Initialize AJV if not already done
if (!global.get('ajvInitialized')) {
  const ajv = new Ajv({ allErrors: true });

  // Define the schema with strict regex patterns
  const schema = {

```

```

14 // Initialize rate limit storage if needed
15 if (!global.get('rateLimitInitialized')) {
16   global.set('rateLimitData', {
17     byIP: {},
18     byDevice: {},
19     lastCleanup: Date.now()
20   });
21   global.set('rateLimitInitialized', true);
22   node.warn("Rate limiting initialized");
23 }
24
25 // Get rate limit data
26 let rateLimits = global.get('rateLimitData');
27
28 // CONFIGURATION
29 const IP_LIMIT = 10; // Max requests per minute per IP
30 const DEVICE_LIMIT = 5; // Max requests per minute per device
31 const WINDOW_MS = 60000; // 1 minute window in milliseconds
32 const CLEANUP_INTERVAL = 60000; // Clean up old data every minute
33
34 // Get client IP address
35 let clientIP = 'unknown';
36 if (msg.req && msg.req.headers) {
37   // Try to get real IP (behind proxy/Cloudflare)
38   clientIP = msg.req.headers['cf-connecting-ip'] || // Cloudflare
39     msg.req.headers['x-forwarded-for'] || // standard proxy
40     msg.req.headers['x-real-ip'] || // nginx
41     msg.req.connection.remoteAddress || // Direct connection
42     'unknown';
43
44   // Clean IP (remove IPv6 prefix if present)
45   if (clientIP.startsWith('::ffff:')) {
46     clientIP = clientIP.substring(7);
47   }
48 }
49
50 // Get device ID from payload (if available)
51 let deviceId = null;
52 if (msg.payload && typeof msg.payload === 'object' && msg.payload.deviceId) {
53   deviceId = msg.payload.deviceId;
54 }
55
56 const now = Date.now();
57
58 // CLEANUP OLD DATA (once per minute)
59 if (now - rateLimits.lastCleanup > CLEANUP_INTERVAL) {
60   let cleanedIPs = 0;
61   let cleanedDevices = 0;
62
63   // Cleanup IP limits
64   for (let ip in rateLimits.byIP) {
65     let ipData = rateLimits.byIP[ip];
66     if (ipData.requests > IP_LIMIT) {
67       cleanedIPs++;
68       delete rateLimits.byIP[ip];
69     }
70   }
71
72   // Cleanup Device limits
73   for (let device in rateLimits.byDevice) {
74     let deviceData = rateLimits.byDevice[device];
75     if (deviceData.requests > DEVICE_LIMIT) {
76       cleanedDevices++;
77       delete rateLimits.byDevice[device];
78     }
79   }
80
81   rateLimits.lastCleanup = now;
82 }
83
84 // Check if PIN setup is required
85 if (!msg.payload.data.pinSetup) {
86   msg.payload = {
87     error: "PIN setup required",
88     code: "PIN_SETUP_REQUIRED",
89     message: "Please set up a 4-digit PIN for this device"
90   };
91   node.warn("PIN setup required for ${studentId} on device ${deviceId.substring(0, 8)}");
92   return [null, msg, null]; // Setup required output
93 } else {
94   // Setting up PIN for first time
95
96   // Validate PIN format
97   if (!/^[0-9]{4}$/.test(enteredPin)) {
98     msg.statusCode = 400;
99     msg.payload = {
100       error: "PIN must be exactly 4 digits",
101       code: "INVALID_PIN_FORMAT"
102     };
103     return [null, null, msg];
104   }
105
106   // Hash the PIN with bcrypt
107   const saltRounds = 10;
108   const hashedPin = bcrypt.hashSync(enteredPin, saltRounds);
109
110   // Store hashed PIN
111   pinStorage[authKey] = {
112     hashedPin: hashedPin,
113     createdAt: now,
114     deviceId: deviceId.substring(0, 16),
115     studentId: studentId
116   };
117
118   flow.set('pinStorage', pinStorage);
119
120   node.warn("✓ PIN set up for ${studentId} on device ${deviceId.substring(0, 8)}");
121
122   // Continue to check-in
123   msg.payload.data.pinSetup = {
124     isSetup: true,
125     timestamp: new Date().toISOString()
126   };
127
128   return [msg, null, null]; // Success output
129 }
130 }
131
132 ...

```

D.3 InfluxDB Task Scripts

Automated aggregation: hourly and daily tasks in Flux query language.

Q

Filter tasks...

Sort by Name (A - Z)

Show Inactive

CREATE TASK

Aggregate to Daily

✓

Active

Last completed at 2026-04-26T21:15:57Z

Scheduled to run every 1d, offset 10m

ID: 1071c5a13b24000

+

Add a label

Aggregate to Hourly

✓

Active

Last completed at 2026-04-26T21:15:57Z

Scheduled to run every 1h, offset 5m

ID: 1071c5b4b924000

+

Add a label

Room Utilization Calculation

✓

Active

Last completed at 2026-04-26T21:15:57Z

Scheduled to run every 1h, offset 10m

ID: 1071c5c83a324000

+

Add a label

Name

Aggregate to Daily

Schedule Task

EVERY

CRON

Every

Offset

1d

10m

```
1 option task = {
2   name: "Aggregate to Daily",
3   every: 1d,
4   offset: 10m
5 }
6
7 // DAILY AGGREGATION TASK
8 // Runs every day at midnight, aggregates last day
9
10
11
12 // Query hourly data from last day
13 from(bucket: "attendance_hourly")
14   |> range(start: -1d)
15   |> filter(fn: (r) => r._measurement == "attendance_hourly")
16   |> filter(fn: (r) => r._field == "value")
17
18 // Group by day, room
19 |> aggregateWindow(
20   every: 1d,
21   fn: sum,
22   createEmpty: false
23 )
24
25 // Calculate statistics
26 |> group(columns: ["roomId", "building"])
27 |> sum()
28
29 // Add metadata
30 |> set(key: "_measurement", value: "attendance_daily")
31
32 // Write to daily bucket
33 |> to(bucket: "attendance_daily", org: "myorg")
```

Name

Aggregate to Hourly

Schedule Task

EVERY

CRON

Every

Offset

1h

5m

```

1 option task = {
2   name: "Aggregate to Hourly",
3   every: 1h,
4   offset: 5m
5 }
6
7 // HOURLY AGGREGATION TASK
8 // Runs every hour, aggregates last hour of data
9
10
11
12 // Query raw data from last hour
13 from(bucket: "attendance")
14   |> range(start: -1h)
15   |> filter(fn: (r) => r._measurement == "attendance")
16   |> filter(fn: (r) => r._field == "value")
17
18 // Group by hour, room, action
19 |> aggregateWindow(
20   every: 1h,
21   fn: sum,
22   createEmpty: false
23 )
24
25 // Add metadata
26 |> set(key: "_measurement", value: "attendance_hourly")
27
28 // Write to hourly bucket
29 |> to(bucket: "attendance_hourly", org: "myorg")
30

```

Name

Room Utilization Calculation

Schedule Task

EVERY

CRON

Every

Offset

1h

10m

```

1 option task = {
2   name: "Room Utilization Calculation",
3   every: 1h,
4   offset: 10m
5 }
6
7
8
9 from(bucket: "attendance_hourly")
10  |> range(start: -1h)
11  |> filter(fn: (r) => r._measurement == "attendance_hourly")
12  |> filter(fn: (r) => r._field == "max_occupancy")
13
14  |> map(fn: (r) => ({
15    r with
16    _value: float(v: r._value) / float(v:
17      if r.roomId == "TN101" then 50
18      else if r.roomId == "TN102" then 50
19      else if r.roomId == "TN103" then 50
20      else if r.roomId == "TN210" then 30
21      else if r.roomId == "TN212" then 30
22      else 1
23    ) * 100.0,
24    field: "utilization_percent"
25  }))
26
27  |> set(key: "_measurement", value: "room_utilization")
28  |> to(bucket: "attendance_hourly", org: "myorg")

```

D.4 docker-compose.yml Complete

Full configuration for 6-service deployment with environment variables.

```
🐳 docker-compose.yml
  ▶ Run All Services
1  services:
2    # InfluxDB
  ▶ Run Service
3    influxdb:
4      image: influxdb:2.7
5      container_name: influxdb
6      restart: always
7      ports:
8        - "8086:8086"
9      environment:
10       - DOCKER_INFLUXDB_INIT_MODE=setup
11       - DOCKER_INFLUXDB_INIT_USERNAME=admin
12       - DOCKER_INFLUXDB_INIT_PASSWORD=
13       - DOCKER_INFLUXDB_INIT_ORG=myorg
14       - DOCKER_INFLUXDB_INIT_BUCKET=occupancy
15       - DOCKER_INFLUXDB_INIT_ADMIN_TOKEN=
16     volumes:
17       - influxdb-data:/var/lib/influxdb2
18       - influxdb-config:/etc/influxdb2
19     networks:
20       - occupancy-net
```

```
  ▶ Run Service
grafana:
  image: grafana/grafana:10.4.1
  container_name: grafana
  ports:
    - "3000:3000"
  volumes:
    - grafana-storage:/var/lib/grafana
  environment:
    # Admin
    - GF_SECURITY_ADMIN_PASSWORD=admin
    - GF_SERVER_ROOT_URL=https://grafana.tudublinattendanceproject.com

    # GLOBAL USER SETTINGS (ADD THESE 3 LINES!)
    - GF_USERS_ALLOW_SIGN_UP=true
    - GF_USERS_AUTO_ASSIGN_ORG=true
    - GF_USERS_AUTO_ASSIGN_ORG_ID=2
    - GF_USERS_AUTO_ASSIGN_ORG_ROLE=Viewer

    # OAuth
    - GF_AUTH_GENERIC_OAUTH_ENABLED=true
    - GF_AUTH_GENERIC_OAUTH_NAME=TU Dublin SSO
    - GF_AUTH_GENERIC_OAUTH_ALLOW_SIGN_UP=true
    - GF_AUTH_GENERIC_OAUTH_AUTO_LOGIN=false
    - GF_AUTH_GENERIC_OAUTH_CLIENT_ID=
    - GF_AUTH_GENERIC_OAUTH_CLIENT_SECRET=
    - GF_AUTH_GENERIC_OAUTH_SCOPES=openid email profile
    - GF_AUTH_GENERIC_OAUTH_AUTH_URL=https://login.microsoftonline.com iuth2/v2.0/authorize
    - GF_AUTH_GENERIC_OAUTH_TOKEN_URL=https://login.microsoftonline.com iuth2/v2.0/token
    - GF_AUTH_GENERIC_OAUTH_API_URL=https://graph.microsoft.com/v1.0/me
    - GF_AUTH_GENERIC_OAUTH_ALLOWED_DOMAINS=floodcathal@gmail.onmicrosoft.com
    - GF_AUTH_GENERIC_OAUTH_EMAIL_ATTRIBUTE_PATH=mail || userPrincipalName
    - GF_AUTH_GENERIC_OAUTH_LOGIN_ATTRIBUTE_PATH=userPrincipalName
    - GF_AUTH_GENERIC_OAUTH_NAME_ATTRIBUTE_PATH=displayName
    - GF_AUTH_GENERIC_OAUTH_ROLE_ATTRIBUTE_PATH=contains(email, 'admin') && 'Admin' || contains(email, 'lecturer') && 'Editor' || 'Viewer'

    # Logging
    - GF_LOG_LEVEL=debug
  networks:
    - occupancy-net
```

```
# Node-RED
▷ Run Service
nodered:
  image: nodered/node-red:latest
  container_name: nodered
  restart: always
  ports:
    - "1880:1880"
  environment:
    - TZ=Europe/Dublin
  volumes:
    - nodered-data:/data
  networks:
    - occupancy-net
```

```
# Prometheus
▷ Run Service
prometheus:
  image: prom/prometheus:latest
  container_name: prometheus
  restart: unless-stopped
  ports:
    - "9090:9090"
  volumes:
    - ./prometheus-data:/prometheus
    - ./prometheus.yml:/etc/prometheus/prometheus.yml
  command:
    - '--config.file=/etc/prometheus/prometheus.yml'
    - '--storage.tsdb.path=/prometheus'
  networks:
    - occupancy-net
```

```
# Cloudflare Tunnel
▷ Run Service
cloudflared:
  image: cloudflare/cloudflared:latest
  container_name: cloudflared
  restart: unless-stopped
  command: tunnel --no-autoupdate run --token
  networks:
    - occupancy-net
```

occupancy-net

▷ Run Service

scanner:

image: nginx:alpine

container_name: scanner

restart: always

ports:

- "8080:80"

volumes:

- ./scanner:/usr/share/nginx/html:ro

- ./scanner/nginx.conf:/etc/nginx/conf.d/default.conf:ro

networks:

- occupancy-net

volumes:

influxdb-data:

influxdb-config:

grafana-storage:

nodered-data:

prometheus-data:

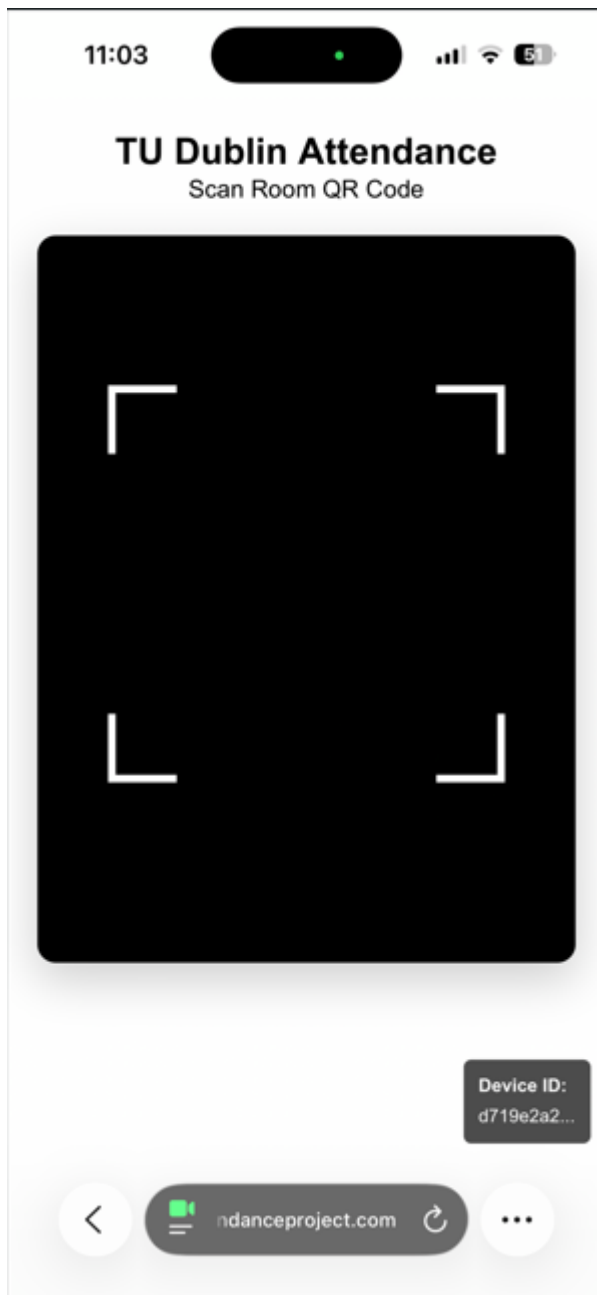
networks:

occupancy-net:

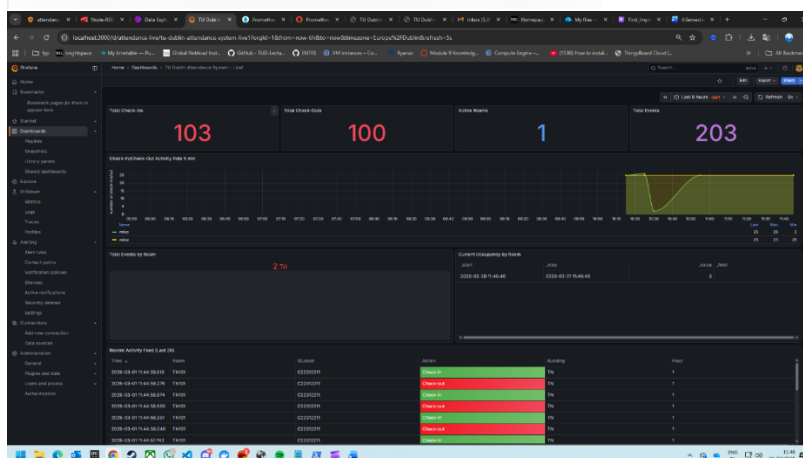
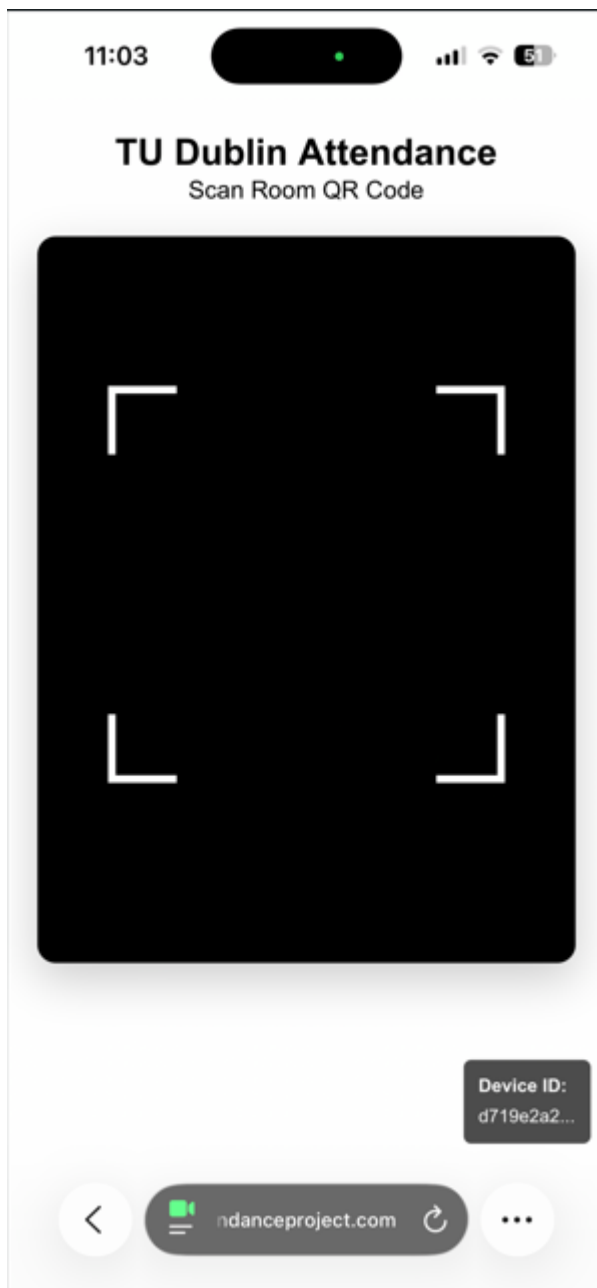
driver: bridge

Appendix E: System Screenshots and Evidence

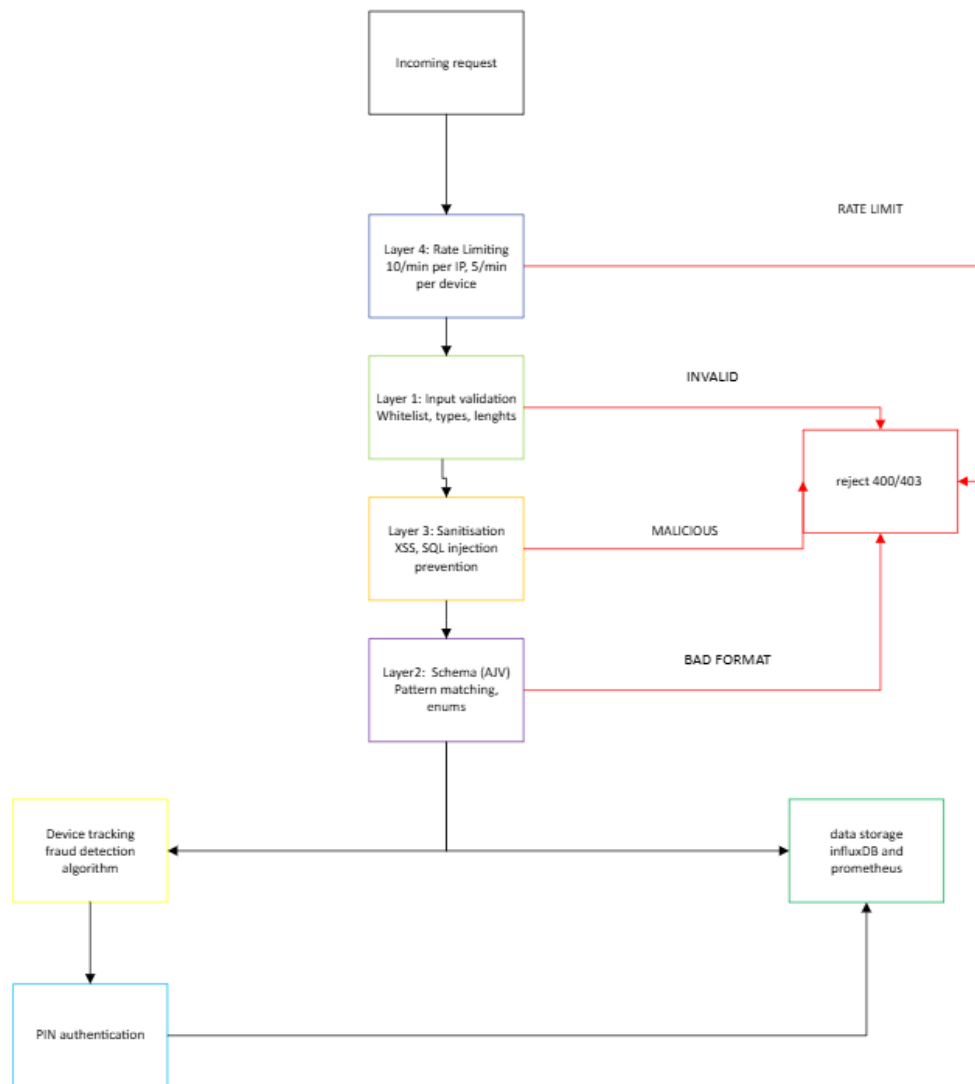
E.2 Contents

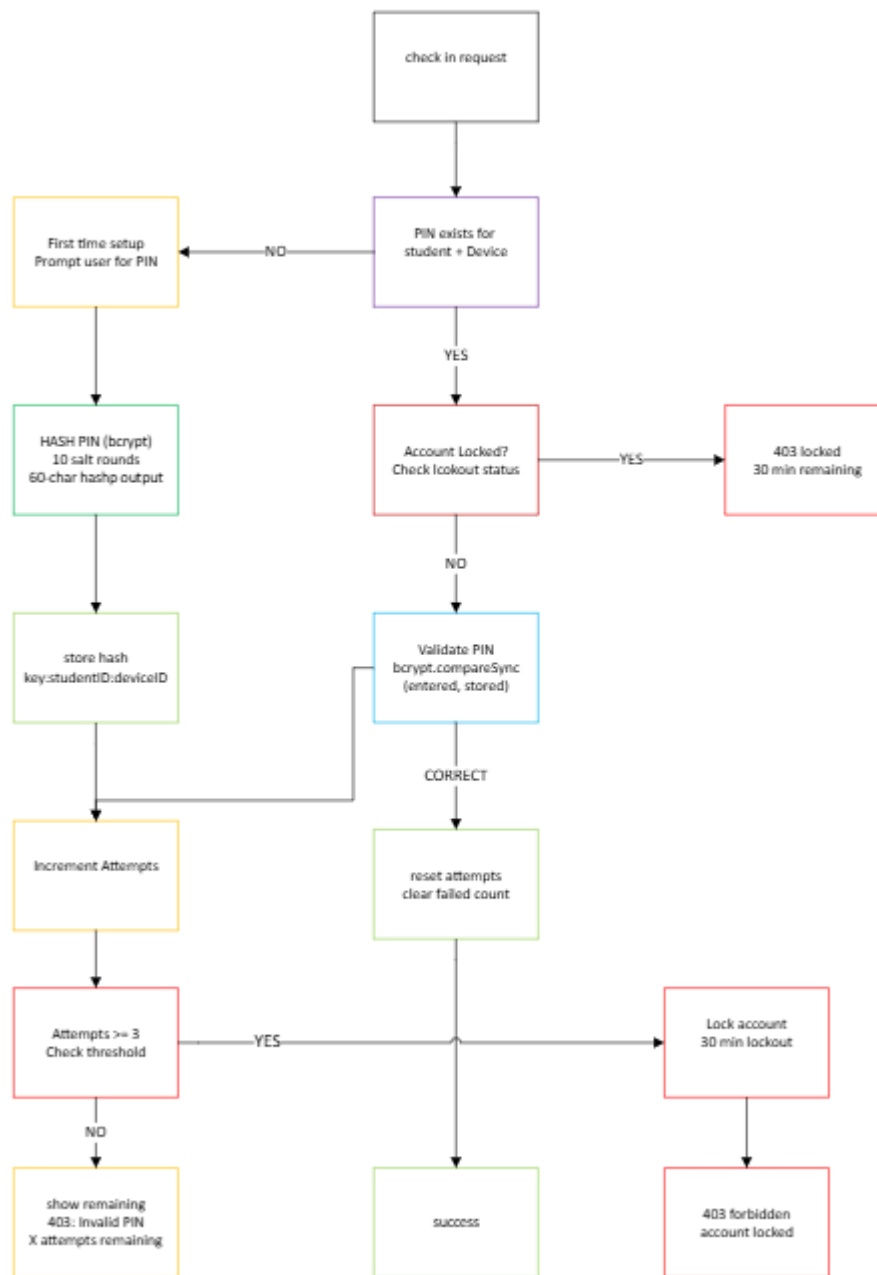


Iteration 1:



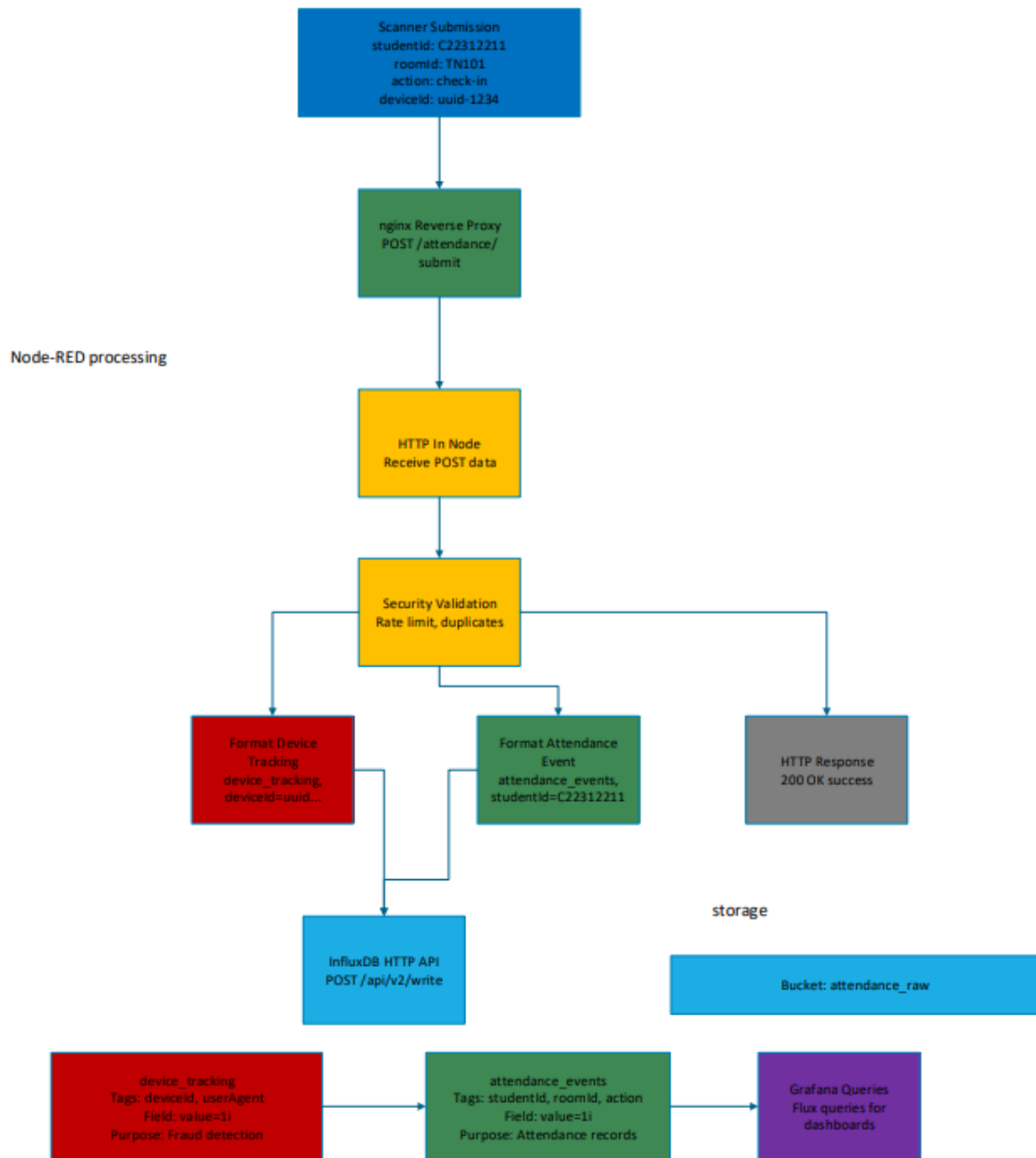
Iteration 2

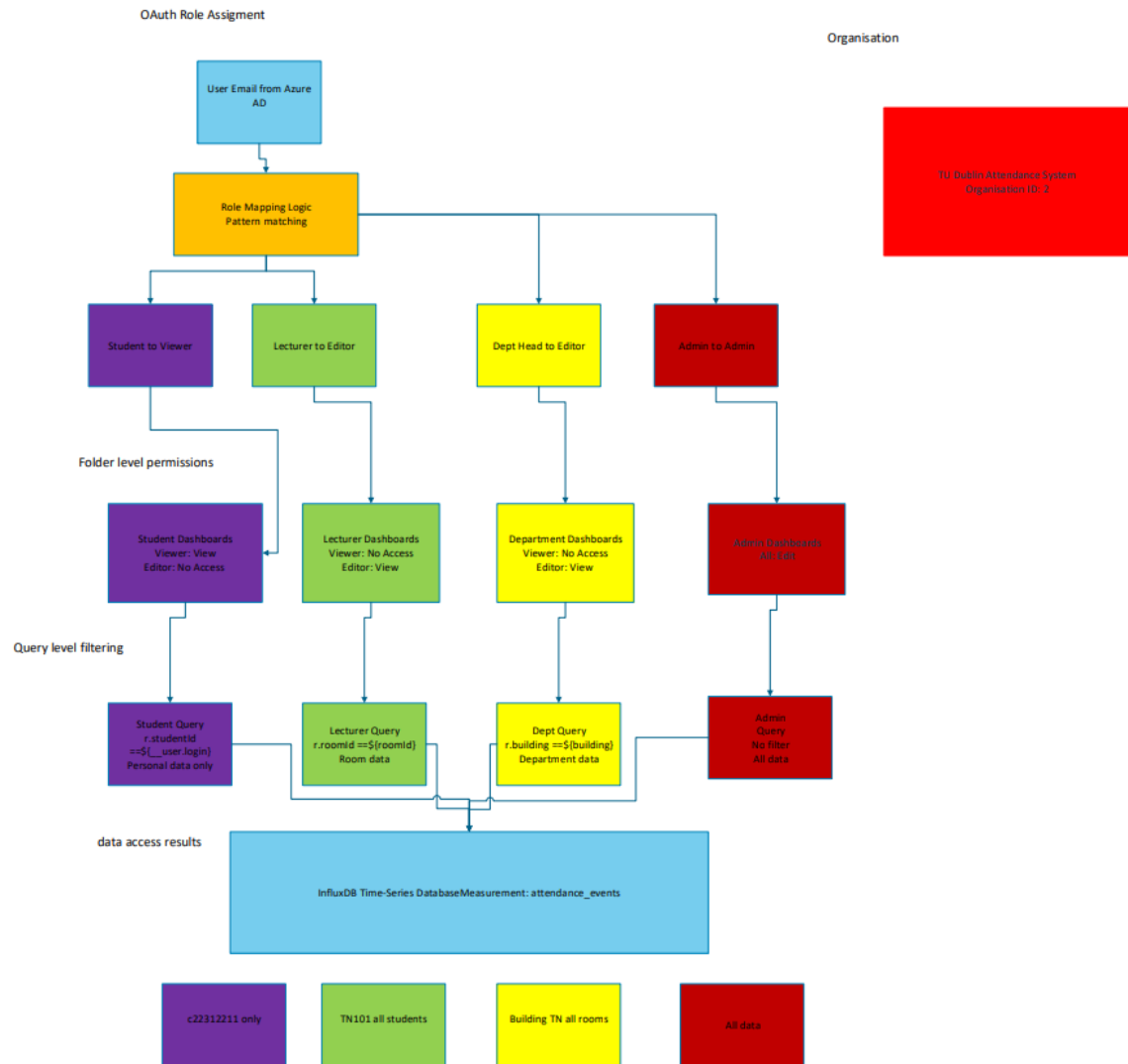


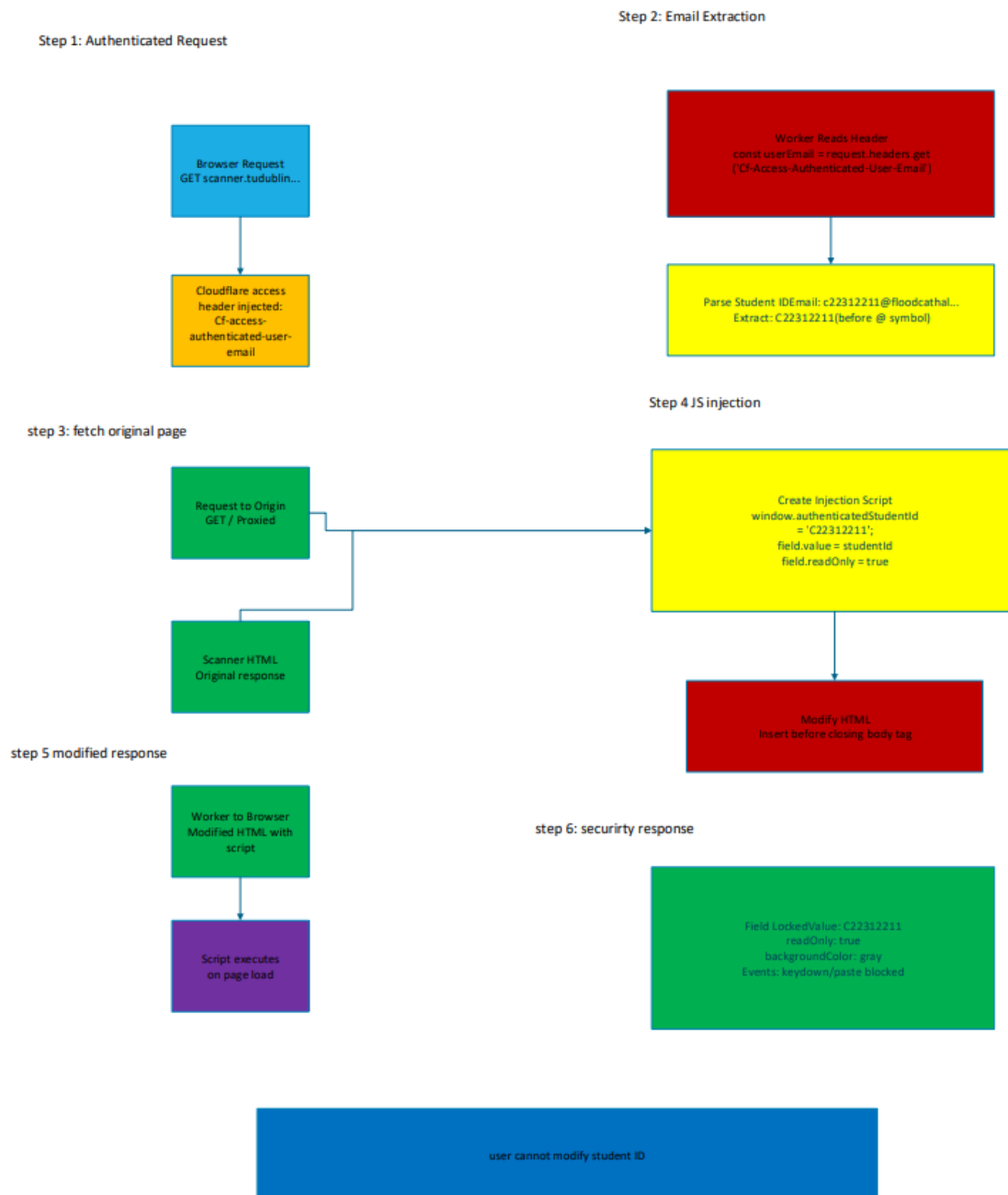


Iteration 3

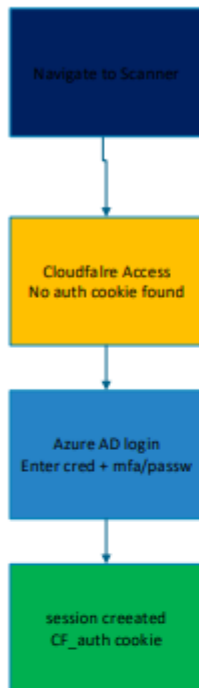
Scanner Form data



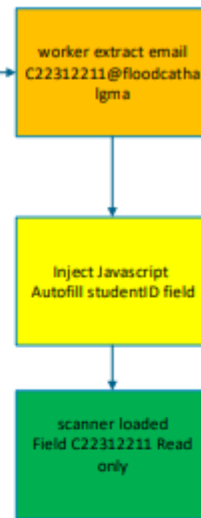




Phase 1 : SSO authentication

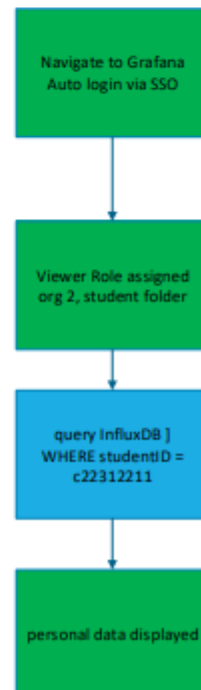
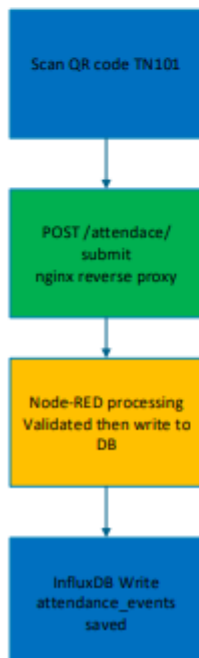


Phase 2: auto fill



phase 4: view dashboard

Phase 3: Attendance submission



Testing results

```

4/26/2026, 10:43:22 AM  node: Security Validation
function : (warn)
" Layer 1 passed - Starting Layer 3 sanitization"

4/26/2026, 10:43:22 AM  node: Security Validation
function : (warn)
"Layer 3 passed - Data sanitized, starting Layer 2 validation"

4/26/2026, 10:43:22 AM  node: Security Validation
function : (warn)
"AJV schema validator initialized"

4/26/2026, 10:43:22 AM  node: Security Validation
function : (warn)
"✓ Layer 2 passed - Schema validation complete"

4/26/2026, 10:43:22 AM  node: Security Validation
function : (warn)
▶ "✓ ALL LAYERS PASSED for student C22312211 (SSO: c22312211@floodcathalgmail.onmic..."

4/26/2026, 10:43:22 AM  node: Security Validation
function : (warn)
"=== Output 3 Data ==="

4/26/2026, 10:43:22 AM  node: Security Validation
function : (warn)
▶ "msg.payload: {"status":"success","message":"All validation layers passed (1, 3, ..."

4/26/2026, 10:43:22 AM  node: Update Prometheus Metrics
function : (warn)
"✓ Prometheus counter incremented: C22312211 → TN101 (check-in)"

4/26/2026, 10:43:23 AM  node: Update Prometheus Metrics
function : (warn)
"✓ Active students +1 for TN101"

4/26/2026, 10:43:23 AM  node: Device Tracking & Fraud Detection
function : (warn)
"Device tracking initialized"

4/26/2026, 10:43:23 AM  node: Device Tracking & Fraud Detection
function : (warn)
"New device for student C22312211: 99d2b6f3"

4/26/2026, 10:43:23 AM  node: Device Tracking & Fraud Detection
function : (warn)
" Normal device usage: 1 student(s) on device 99d2b6f3"

4/26/2026, 10:43:23 AM  node: format attendance data

```

Scripts:
[Scripts](#)

Appendix F: AI Usage

Level 2: AI-Assisted Idea Generation and Brainstorming. AI can be used in the assessment for brainstorming, creating structures, and generating ideas for improving work

This appendix lists prompts used with generative AI across report writing, research, design, and coding tasks (as permitted). AI-assisted outputs were reviewed, tested, and refined by the author before being incorporated into the final system and dissertation.

F.1 Prompts related to report writing

Review this section for academic tone and grammar while keeping the original text and do not add any information

Is there any improvement that can be made in terms of transition from one paragraph to another, are they best done chronologically

Proofread these sections and highlight weak points

Suggest a better style in concluding this section

F.3 Prompts related to Design

Translate the system requirements into broken down sections for better understanding

Methods of eliminating CORS in a multi sub domain environment

Identify risks and strengths in the design workflow

Compare the design to use cases and evaluate